Python 150 Interview Questions - Complete Solutions

Contents

SECTION A: BASICS, LOOPS & MATH (20 Questions)

Q1. Print all numbers from 1 to N

Question: Print all numbers from 1 to a given positive integer N.

Solution 1 – Brute Force Approach:

```
def print_numbers_bf(N):
    """Print all numbers from 1 to N using brute force."""
    for i in range(1, N + 1):
        print(i)
```

Logic: Use a simple for loop that iterates from 1 to N (inclusive). Each iteration prints the current number. This is the most straightforward approach.

Inefficiencies: No significant inefficiencies for this simple problem. It's O(N) time which is optimal for printing N numbers.

Solution 2 – Optimized Approach:

```
def print_numbers_opt(N):
    """Print all numbers from 1 to N using list comprehension."""
    print('\n'.join(map(str, range(1, N + 1))))
```

Optimization: Uses a single print statement with join, reducing function call overhead. Map converts numbers to strings efficiently.

Justification: Reduces I/O overhead by making only one system call to print instead of N calls.

Time Complexity:

- **Brute Force:** O(N)
- **Optimized:** O(N)

Space Complexity:

- **Brute Force:** O(1) - only loop variable
- **Optimized:** O(N) - creates list of strings

Best / Average / Worst Case Analysis:

- **Best Case:** O(N) - must visit all numbers
- **Average Case:** O(N)
- **Worst Case:** O(N)

Q2. Print all even numbers between 1 and N

Question: Print all even numbers from 1 to N (inclusive if N is even).

Solution 1 – Brute Force Approach:

```
def print_even_bf(N):
    """Print even numbers using modulo check."""
    for i in range(1, N + 1):
        if i % 2 == 0:
            print(i)
```

Logic: Iterate through all numbers 1 to N, check each with modulo operator, print if divisible by 2.

Inefficiencies: Checks every number including odd ones, performs modulo operation for all N numbers.

Solution 2 – Optimized Approach:

```
def print_even_opt(N):
    """Print even numbers using step size of 2."""
    for i in range(2, N + 1, 2):
        print(i)
```

Optimization: Start from 2 and increment by 2, directly accessing even numbers without checking.

Justification: Eliminates modulo operations and skips odd numbers entirely, halving the iterations.

Time Complexity:

- **Brute Force:** O(N)
- **Optimized:** O(N/2) O(N)

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(N) for brute force, O(N/2) for optimized
- **Average Case:** O(N) for brute force, O(N/2) for optimized
- **Worst Case:** O(N) for brute force, O(N/2) for optimized

Q3. Print all odd numbers between 1 and N

Question: Print all odd numbers from 1 to N (inclusive if N is odd).

Solution 1 – Brute Force Approach:

```
def print_odd_bf(N):
    """Print odd numbers using modulo check."""
    for i in range(1, N + 1):
        if i % 2 != 0:
            print(i)
```

Logic: Similar to even numbers but checks for non-divisibility by 2.

Inefficiencies: Same as Q2 - checks every number with modulo operation.

Solution 2 – Optimized Approach:

```
def print_odd_opt(N):
    """Print odd numbers using step size of 2."""
    for i in range(1, N + 1, 2):
        print(i)
```

Optimization: Start from 1 and increment by 2 to directly access odd numbers.

Justification: Same optimization principle as Q2 - eliminates unnecessary checks.

Time Complexity:

- **Brute Force:** O(N)
- **Optimized:** O(N/2) O(N)

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(N) for brute force, O(N/2) for optimized
- **Average Case:** O(N) for brute force, O(N/2) for optimized
- **Worst Case:** O(N) for brute force, O(N/2) for optimized

Q4. Sum of first N natural numbers

Question: Find the sum of the first N natural numbers (1 + 2 + ... + N).

Solution 1 – Brute Force Approach:

```python
def sum_natural_bf(N):
    """Sum using iterative addition."""
    total = 0
    for i in range(1, N + 1):
        total += i
    return total
```

Logic: Initialize total to 0, iterate through 1 to N, add each number to total.

Inefficiencies: Requires N additions, which is unnecessary for a mathematical formula.

Solution 2 – Optimized Approach:

```python
def sum_natural_opt(N):
    """Sum using mathematical formula."""
    return N * (N + 1) // 2
```

Optimization: Uses the arithmetic series formula: sum = n(n+1)/2

Justification: Converts O(N) time to O(1) time with a direct mathematical computation.

Time Complexity:

- **Brute Force:** O(N)
- **Optimized:** O(1)

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(N) for brute force, O(1) for optimized
- **Average Case:** O(N) for brute force, O(1) for optimized
- **Worst Case:** O(N) for brute force, O(1) for optimized

Q5. Factorial of a number

Question: Calculate factorial of n (n! = n × (n-1) × ... × 1).

Solution 1 – Brute Force Approach:

```
def factorial_bf(n):
    """Factorial using iterative multiplication."""
    if n < 0:
        return "Undefined"
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result
```

Logic: Handle negative case, start with 1, multiply by all numbers from 2 to n.

Inefficiencies: For large n, requires n-1 multiplications. No significant inefficiency for this problem.

Solution 2 – Optimized Approach:

```
import math

def factorial_opt(n):
    """Factorial using math.factorial or recursion with
       memoization."""
    if n < 0:
        return "Undefined"
    return math.factorial(n)
```

Optimization: Uses built-in math.factorial which is implemented in C and highly optimized.

Justification: Built-in functions use optimized algorithms and are faster than Python loops.

Time Complexity:

- **Brute Force:** O(n)
- **Optimized:** O(n) (but faster constant factor)

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) when n=0 or 1
- **Average Case:** O(n)
- **Worst Case:** O(n)

Q6. Check if a number is prime

Question: Determine if a given integer is prime (only divisible by 1 and itself).

Solution 1 – Brute Force Approach:

```
def is_prime_bf(n):
    """Check primality by testing all divisors."""
    if n <= 1:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
```

Logic: Check divisibility by all numbers from 2 to n-1. If any divides n, it's not prime.

Inefficiencies: Tests up to n-1 divisors when we only need to test up to sqrt(n).

Solution 2 – Optimized Approach:

```
def is_prime_opt(n):
    """Check primality up to sqrt(n)."""
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False

    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True
```

Optimization: 1. Check divisibility only up to sqrt(n) 2. Skip even numbers after checking 2 3. Use 6k±1 optimization: all primes ¿ 3 are of form 6k±1

Justification: Reduces operations from O(n) to O(n) with smart divisor checking.

Time Complexity:

- **Brute Force:** O(n)
- **Optimized:** O(n)

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) for even numbers or n1
- **Average Case:** O(n) for optimized
- **Worst Case:** O(n) for optimized when n is prime

Q7. Fibonacci series up to N terms

Question: Generate Fibonacci series up to N terms (0, 1, 1, 2, 3, 5, ...).

Solution 1 – Brute Force Approach:

```
def fibonacci_bf(N):
    """Fibonacci using array storage."""
    if N <= 0:
        return []
    if N == 1:
        return [0]

    fib = [0, 1]
    for i in range(2, N):
        fib.append(fib[i-1] + fib[i-2])
    return fib
```

Logic: Store all Fibonacci numbers in array, each new term is sum of previous two.

Inefficiencies: Stores entire sequence when we might only need last two terms.

Solution 2 – Optimized Approach:

```
def fibonacci_opt(N):
    """Fibonacci using two variables."""
    if N <= 0:
        return []

    result = []
    a, b = 0, 1
    for _ in range(N):
        result.append(a)
        a, b = b, a + b
    return result
```

Optimization: Uses only two variables to track last two terms instead of storing all.

Justification: Reduces space complexity from O(N) to O(1) for generating terms (though output is O(N)).

Time Complexity:

- **Brute Force:** O(N)
- **Optimized:** O(N)

Space Complexity:

- **Brute Force:** O(N) for storage array
- **Optimized:** O(1) for computation, O(N) for output

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) when N1
- **Average Case:** O(N)
- **Worst Case:** O(N)

Q8. Reverse digits of a number

Question: Reverse the digits of a given integer (e.g., 123 → 321).

Solution 1 – Brute Force Approach:

```python
def reverse_digits_bf(n):
    """Reverse digits using string conversion."""
    sign = -1 if n < 0 else 1
    n = abs(n)
    return sign * int(str(n)[::-1])
```

Logic: Convert to string, reverse using slicing, convert back to int. Handle negative sign.

Inefficiencies: String conversions have overhead. May overflow for large numbers.

Solution 2 – Optimized Approach:

```python
def reverse_digits_opt(n):
    """Reverse digits using mathematical operations."""
    sign = -1 if n < 0 else 1
    n = abs(n)
    reversed_num = 0

    while n > 0:
        digit = n % 10
        reversed_num = reversed_num * 10 + digit
        n //= 10

    return sign * reversed_num
```

Optimization: Use modulo and division to extract digits and build reversed number mathematically.

Justification: Avoids string conversion overhead and handles integer operations directly.

Time Complexity:

- **Brute Force:** O(d) where d is number of digits
- **Optimized:** O(d) where d is number of digits

Space Complexity:

- **Brute Force:** O(d) for string representation
- **Optimized:** O(1) constant space

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) for single-digit numbers
- **Average Case:** O(d)
- **Worst Case:** O(d) for multi-digit numbers

Q9. Check if a number is palindrome

Question: Check if a number reads the same forwards and backwards (e.g., 121).

Solution 1 – Brute Force Approach:

```python
def is_palindrome_bf(n):
    """Check palindrome using string conversion."""
    return str(n) == str(n)[::-1]
```

Logic: Convert to string, compare with its reverse.

Inefficiencies: Creates two strings and compares entire strings.

Solution 2 – Optimized Approach:

```python
def is_palindrome_opt(n):
    """Check palindrome mathematically."""
    if n < 0:
        return False

    original = n
    reversed_num = 0

    while n > 0:
        digit = n % 10
        reversed_num = reversed_num * 10 + digit
        n //= 10

    return original == reversed_num
```

Optimization: Reverse the number mathematically and compare with original.

Justification: Avoids string conversion and only reverses half the number in theory (though implementation reverses fully).

Time Complexity:

- **Brute Force:** O(d) where d is number of digits
- **Optimized:** O(d) where d is number of digits

Space Complexity:

- **Brute Force:** O(d) for strings
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) for single-digit numbers
- **Average Case:** O(d)
- **Worst Case:** O(d) for large numbers

Q10. Count number of digits in a number

Question: Count the number of digits in a given integer.

Solution 1 – Brute Force Approach:

```python
def count_digits_bf(n):
    """Count digits using string conversion."""
    n = abs(n)
    return len(str(n))
```

Logic: Convert to string, use len() function.

Inefficiencies: String conversion overhead.

Solution 2 – Optimized Approach:

```python
def count_digits_opt(n):
    """Count digits using mathematical operations."""
    if n == 0:
        return 1

    n = abs(n)
    count = 0
    while n > 0:
        count += 1
        n //= 10
    return count
```

Optimization: Use integer division to count digits mathematically.

Justification: More efficient than string conversion for pure counting.

Time Complexity:

- **Brute Force:** O(d) where d is number of digits
- **Optimized:** O(d) where d is number of digits

Space Complexity:

- **Brute Force:** O(d) for string
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) for n=0
- **Average Case:** O(d)
- **Worst Case:** O(d) for large numbers

Q11. Sum of digits of a number

Question: Find the sum of all digits of a given integer.

Solution 1 – Brute Force Approach:

```python
def sum_digits_bf(n):
    """Sum digits using string conversion."""
    n = abs(n)
    return sum(int(digit) for digit in str(n))
```

Logic: Convert to string, iterate through characters, convert each to int and sum.

Inefficiencies: Multiple type conversions and string creation.

Solution 2 – Optimized Approach:

```python
def sum_digits_opt(n):
    """Sum digits using modulo operations."""
    n = abs(n)
    total = 0
    while n > 0:
        total += n % 10
        n //= 10
    return total
```

Optimization: Extract digits using modulo and division, sum directly.

Justification: Avoids string conversion and works directly with integers.

Time Complexity:

- **Brute Force:** O(d) where d is number of digits
- **Optimized:** O(d) where d is number of digits

Space Complexity:

- **Brute Force:** O(d) for string
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) for single-digit numbers
- **Average Case:** O(d)
- **Worst Case:** O(d) for large numbers

Q12. Check if a number is Armstrong number

Question: Check if a number is Armstrong (sum of its digits each raised to power of number of digits equals the number itself).

Solution 1 – Brute Force Approach:

```python
def is_armstrong_bf(n):
    """Check Armstrong using string conversion."""
    if n < 0:
        return False

    digits = str(n)
    power = len(digits)
    total = sum(int(d) ** power for d in digits)
    return total == n
```

Logic: Convert to string to get digits and count, compute sum of digits$^{p}ower, compare$.

Inefficiencies: String conversion and multiple power calculations.

Solution 2 – Optimized Approach:

```python
def is_armstrong_opt(n):
    """Check Armstrong using mathematical operations."""
    if n < 0:
        return False
    if n == 0:
        return True

    # Count digits
    temp = n
    num_digits = 0
    while temp > 0:
        num_digits += 1
        temp //= 10

    # Calculate sum
    temp = n
    total = 0
    while temp > 0:
        digit = temp % 10
        total += digit ** num_digits
        temp //= 10

    return total == n
```

Optimization: Count digits mathematically first, then extract digits and compute power sum.

Justification: Avoids string conversion and reuses digit count.

Time Complexity:

- **Brute Force:** O(d) where d is number of digits
- **Optimized:** O(d) where d is number of digits

Space Complexity:

- **Brute Force:** O(d) for string
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) for n=0 or single-digit numbers
- **Average Case:** O(d)
- **Worst Case:** O(d) for large numbers

Q13. GCD of two numbers

Question: Find the Greatest Common Divisor of two integers.

Solution 1 – Brute Force Approach:

```python
def gcd_bf(a, b):
    """GCD by checking all possible divisors."""
    a, b = abs(a), abs(b)
    if a == 0:
        return b
    if b == 0:
        return a

    gcd = 1
    for i in range(1, min(a, b) + 1):
        if a % i == 0 and b % i == 0:
            gcd = i
    return gcd
```

Logic: Check all numbers from 1 to min(a,b), update gcd when common divisor found.

Inefficiencies: Checks all numbers, even non-divisors. O(min(a,b)) time.

Solution 2 – Optimized Approach:

```python
def gcd_opt(a, b):
    """GCD using Euclidean algorithm."""
    a, b = abs(a), abs(b)
    while b != 0:
        a, b = b, a % b
    return a
```

Optimization: Euclidean algorithm: gcd(a,b) = gcd(b, a mod b). Much faster convergence.

Justification: Reduces time complexity from O(min(a,b)) to O(log(min(a,b))).

Time Complexity:

- **Brute Force:** O(min(a,b))
- **Optimized:** O(log(min(a,b)))

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) when b=0
- **Average Case:** O(log(min(a,b))) for Euclidean
- **Worst Case:** O(min(a,b)) for brute force, O(log(min(a,b))) for Euclidean

Q14. LCM of two numbers

Question: Find the Least Common Multiple of two integers.

Solution 1 – Brute Force Approach:

```python
def lcm_bf(a, b):
    """LCM by checking multiples."""
    a, b = abs(a), abs(b)
    if a == 0 or b == 0:
        return 0

    max_num = max(a, b)
    while True:
        if max_num % a == 0 and max_num % b == 0:
            return max_num
        max_num += 1
```

Logic: Start from max(a,b), increment until finding a number divisible by both.

Inefficiencies: Could require many iterations. In worst case, lcm = a*b, so O(a*b) time.

Solution 2 – Optimized Approach:

```python
def gcd(a, b):
    """Helper function for GCD."""
    while b != 0:
        a, b = b, a % b
    return a

def lcm_opt(a, b):
    """LCM using formula: lcm(a,b) = a*b / gcd(a,b)."""
    a, b = abs(a), abs(b)
    if a == 0 or b == 0:
        return 0
    return (a * b) // gcd(a, b)
```

Optimization: Use mathematical relationship: lcm(a,b) × gcd(a,b) = a × b

Justification: Reduces from potentially O(a*b) to O(log(min(a,b))) using efficient GCD.

Time Complexity:

- **Brute Force:** O(a*b) in worst case
- **Optimized:** O(log(min(a,b)))

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) when a=b
- **Average Case:** O(log(min(a,b))) for optimized
- **Worst Case:** O(a*b) for brute force, O(log(min(a,b))) for optimized

Q15. Print all prime numbers between 1 and N

Question: Generate all prime numbers from 2 up to N.

Solution 1 – Brute Force Approach:

```python
def primes_bf(N):
    """Generate primes by checking each number individually."""
    primes = []
    for num in range(2, N + 1):
        is_prime = True
        for i in range(2, num):
            if num % i == 0:
                is_prime = False
                break
        if is_prime:
            primes.append(num)
    return primes
```

Logic: For each number from 2 to N, check divisibility by all smaller numbers.

Inefficiencies: $O(N^2)$ time complexity. Rechecks divisors unnecessarily.

Solution 2 – Optimized Approach:

```python
def primes_opt(N):
    """Generate primes using Sieve of Eratosthenes."""
    if N < 2:
        return []

    sieve = [True] * (N + 1)
    sieve[0] = sieve[1] = False

    for i in range(2, int(N**0.5) + 1):
        if sieve[i]:
            for j in range(i*i, N + 1, i):
                sieve[j] = False

    return [i for i in range(2, N + 1) if sieve[i]]
```

Optimization: Sieve of Eratosthenes marks multiples of primes as non-prime.

Justification: Reduces from $O(N^2)$ to $O(N \log \log N)$, much more efficient for large N.

Time Complexity:

- **Brute Force:** $O(N^2)$
- **Optimized:** $O(N \log \log N)$

Space Complexity:

- **Brute Force:** $O(1)$ excluding output
- **Optimized:** $O(N)$ for sieve array

Best / Average / Worst Case Analysis:

- **Best Case:** $O(N)$ for optimized when N is small

- **Average Case:** O(N log log N) for sieve
- **Worst Case:** O(N^2) for brute force, O(N log log N) for sieve

Q16. Power of a number without ** operator

Question: Calculate x raised to power n without using ** operator.

Solution 1 – Brute Force Approach:

```python
def power_bf(x, n):
    """Power using repeated multiplication."""
    if n == 0:
        return 1
    if n < 0:
        return 1 / power_bf(x, -n)

    result = 1
    for _ in range(n):
        result *= x
    return result
```

Logic: Multiply x by itself n times. Handle negative exponents with recursion.

Inefficiencies: O(n) multiplications. Inefficient for large n.

Solution 2 – Optimized Approach:

```python
def power_opt(x, n):
    """Power using exponentiation by squaring."""
    if n == 0:
        return 1
    if n < 0:
        return 1 / power_opt(x, -n)

    result = 1
    while n > 0:
        if n % 2 == 1:  # If n is odd
            result *= x
        x *= x
        n //= 2
    return result
```

Optimization: Exponentiation by squaring: $x = (x^2)^{n/2} if n even, else x(x)^{(n-1)/2}$

Justification: Reduces from O(n) to O(log n) multiplications.

Time Complexity:

- **Brute Force:** O(n)
- **Optimized:** O(log n)

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) when n=0
- **Average Case:** O(log n) for optimized
- **Worst Case:** O(n) for brute force, O(log n) for optimized

Q17. Decimal to binary conversion

Question: Convert a decimal integer to its binary representation.

Solution 1 – Brute Force Approach:

```python
def dec_to_bin_bf(n):
    """Decimal to binary using string building."""
    if n == 0:
        return "0"

    is_negative = n < 0
    n = abs(n)
    binary = ""

    while n > 0:
        binary = str(n % 2) + binary
        n //= 2

    return "-" + binary if is_negative else binary
```

Logic: Repeatedly divide by 2, prepend remainder to string.

Inefficiencies: String concatenation is $O(k^2)$ for k bits since strings are immutable.

Solution 2 – Optimized Approach:

```python
def dec_to_bin_opt(n):
    """Decimal to binary using list and join."""
    if n == 0:
        return "0"

    is_negative = n < 0
    n = abs(n)
    bits = []

    while n > 0:
        bits.append(str(n % 2))
        n //= 2

    bits.reverse()
    result = "".join(bits)
    return "-" + result if is_negative else result
```

Optimization: Use list to store bits, then join once. Avoids repeated string concatenation.

Justification: String join is $O(k)$ vs $O(k^2)$ for repeated concatenation.

Time Complexity:

- **Brute Force:** $O(k^2)$ where k is number of bits
- **Optimized:** $O(k)$ where k is number of bits

Space Complexity:

- **Brute Force:** $O(k)$ for string

- **Optimized:** O(k) for list

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) when n=0
- **Average Case:** O(k) for optimized
- **Worst Case:** O(k^2) for brute force, O(k) for optimized

Q18. Binary to decimal conversion

Question: Convert a binary string to its decimal integer equivalent.

Solution 1 – Brute Force Approach:

```python
def bin_to_dec_bf(binary_str):
    """Binary to decimal using string traversal."""
    decimal = 0
    power = len(binary_str) - 1

    for char in binary_str:
        if char == '1':
            decimal += 2 ** power
        power -= 1

    return decimal
```

Logic: Traverse string, add $2^{position for each '1' bit}$.

Inefficiencies: Computes $2^{power for each bit using exponentiation}$.

Solution 2 – Optimized Approach:

```python
def bin_to_dec_opt(binary_str):
    """Binary to decimal using bit shifting."""
    decimal = 0
    for char in binary_str:
        decimal = decimal * 2 + (1 if char == '1' else 0)
    return decimal
```

Optimization: Multiply current result by 2 and add current bit. Equivalent to left shift in binary.

Justification: Avoids exponentiation operations, uses only multiplication and addition.

Time Complexity:

- **Brute Force:** O(k) where k is number of bits
- **Optimized:** O(k) where k is number of bits

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) for single-bit strings
- **Average Case:** O(k)
- **Worst Case:** O(k) for k-bit strings

Q19. Count even and odd digits in a number

Question: Count how many even and odd digits are in a given integer.

Solution 1 – Brute Force Approach:

```python
def count_even_odd_bf(n):
    """Count using string conversion."""
    n = abs(n)
    if n == 0:
        return 1, 0  # 0 is even

    even = odd = 0
    for char in str(n):
        digit = int(char)
        if digit % 2 == 0:
            even += 1
        else:
            odd += 1

    return even, odd
```

Logic: Convert to string, check each digit's parity using modulo.

Inefficiencies: String conversion and int conversion for each digit.

Solution 2 – Optimized Approach:

```python
def count_even_odd_opt(n):
    """Count using mathematical operations."""
    n = abs(n)
    if n == 0:
        return 1, 0

    even = odd = 0
    while n > 0:
        digit = n % 10
        if digit % 2 == 0:
            even += 1
        else:
            odd += 1
        n //= 10

    return even, odd
```

Optimization: Extract digits using modulo, check parity directly.

Justification: Avoids string and int conversions, works directly with integers.

Time Complexity:

- **Brute Force:** O(d) where d is number of digits
- **Optimized:** O(d) where d is number of digits

Space Complexity:

- **Brute Force:** O(d) for string

- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) for single-digit numbers
- **Average Case:** O(d)
- **Worst Case:** O(d) for large numbers

Q20. Check if a number is perfect number

Question: Check if a number equals the sum of its proper divisors (excluding itself).

Solution 1 – Brute Force Approach:

```
def is_perfect_bf(n):
    """Check perfect number by summing all divisors."""
    if n <= 1:
        return False

    divisor_sum = 0
    for i in range(1, n):
        if n % i == 0:
            divisor_sum += i

    return divisor_sum == n
```

Logic: Sum all divisors from 1 to n-1, compare with n.

Inefficiencies: Checks all numbers up to n-1. O(n) time.

Solution 2 – Optimized Approach:

```
def is_perfect_opt(n):
    """Check perfect number up to sqrt(n)."""
    if n <= 1:
        return False

    divisor_sum = 1  # 1 is always a divisor
    i = 2
    while i * i <= n:
        if n % i == 0:
            divisor_sum += i
            if i != n // i:  # Avoid adding square root twice
                divisor_sum += n // i
        i += 1

    return divisor_sum == n
```

Optimization: Only check divisors up to sqrt(n). If i divides n, then n/i also divides n.

Justification: Reduces from O(n) to O(n) divisor checks.

Time Complexity:

- **Brute Force:** O(n)
- **Optimized:** O(n)

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) when n1
- **Average Case:** O(n) for optimized
- **Worst Case:** O(n) for brute force, O(n) for optimized when n is perfect

SECTION B: ARRAYS / LISTS (30 Questions)

Q21. Print all elements of a list

Question: Print each element of a given list on a separate line.

Solution 1 – Brute Force Approach:

```python
def print_list_bf(lst):
    """Print each element using for loop."""
    for element in lst:
        print(element)
```

Logic: Iterate through the list using a for loop, print each element.

Inefficiencies: Makes separate print calls for each element, which can be inefficient for large lists due to I/O overhead.

Solution 2 – Optimized Approach:

```python
def print_list_opt(lst):
    """Print using join for strings or single print."""
    # If elements are strings
    if all(isinstance(x, str) for x in lst):
        print('\n'.join(lst))
    else:
        # Convert all to strings first
        print('\n'.join(map(str, lst)))
```

Optimization: Use join to create a single string with newlines, then print once. Reduces I/O calls.

Justification: Single print call is faster than N print calls, especially for large lists.

Time Complexity:

- **Brute Force:** O(n) where n is list length
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(1) excluding output
- **Optimized:** O(n) for the joined string

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) - must process all elements
- **Average Case:** O(n)
- **Worst Case:** O(n)

Q22. Sum of all elements in a list

Question: Calculate the sum of all elements in a list of numbers.

Solution 1 – Brute Force Approach:

```python
def sum_list_bf(lst):
    """Sum using explicit loop."""
    total = 0
    for num in lst:
        total += num
    return total
```

Logic: Initialize total to 0, iterate through list, add each element to total.

Inefficiencies: Explicit loop in Python is slower than built-in functions.

Solution 2 – Optimized Approach:

```python
def sum_list_opt(lst):
    """Sum using built-in sum()."""
    return sum(lst)
```

Optimization: Use Python's built-in sum() function which is implemented in C.

Justification: Built-in functions are optimized and faster than Python loops.

Time Complexity:

- **Brute Force:** O(n) where n is list length
- **Optimized:** O(n) where n is list length (but faster constant factor)

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) for empty list
- **Average Case:** O(n)
- **Worst Case:** O(n)

Q23. Maximum and minimum elements in a list

Question: Find both maximum and minimum values in a list.

Solution 1 – Brute Force Approach:

```python
def find_min_max_bf(lst):
    """Find min and max using separate passes."""
    if not lst:
        return None, None

    min_val = lst[0]
    max_val = lst[0]

    for num in lst:
        if num < min_val:
            min_val = num
        if num > max_val:
            max_val = num

    return min_val, max_val
```

Logic: Initialize with first element, compare each element with current min and max, update if needed.

Inefficiencies: Performs 2 comparisons per element. Can be optimized to 1.5 comparisons per element.

Solution 2 – Optimized Approach:

```python
def find_min_max_opt(lst):
    """Find min and max in pairs to reduce comparisons."""
    if not lst:
        return None, None

    n = len(lst)
    if n % 2 == 0:
        if lst[0] < lst[1]:
            min_val, max_val = lst[0], lst[1]
        else:
            min_val, max_val = lst[1], lst[0]
        start = 2
    else:
        min_val = max_val = lst[0]
        start = 1

    for i in range(start, n, 2):
        if lst[i] < lst[i + 1]:
            small, large = lst[i], lst[i + 1]
        else:
            small, large = lst[i + 1], lst[i]

        if small < min_val:
            min_val = small
        if large > max_val:
            max_val = large
```

```
27
28      return min_val, max_val
```

Optimization: Process elements in pairs. Compare pair elements first, then compare with min/max. Reduces comparisons to 1.5n vs 2n.

Justification: Tournament method reduces total comparisons by 25%.

Time Complexity:

- **Brute Force:** O(n) with 2n comparisons
- **Optimized:** O(n) with 1.5n comparisons

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for both
- **Average Case:** O(n) for both
- **Worst Case:** O(n) for both

Q24. Reverse a list

Question: Reverse the order of elements in a list.

Solution 1 – Brute Force Approach:

```
def reverse_list_bf(lst):
    """Reverse using new list."""
    reversed_lst = []
    for i in range(len(lst) - 1, -1, -1):
        reversed_lst.append(lst[i])
    return reversed_lst
```

Logic: Create new list, iterate original list backwards, append elements.

Inefficiencies: Creates new list (O(n) extra space). Multiple append operations.

Solution 2 – Optimized Approach:

```
def reverse_list_opt(lst):
    """Reverse in-place using two pointers."""
    left, right = 0, len(lst) - 1
    while left < right:
        lst[left], lst[right] = lst[right], lst[left]
        left += 1
        right -= 1
    return lst
```

Optimization: Reverse in-place using two-pointer technique. Swap elements from ends moving inward.

Justification: O(1) extra space vs O(n). In-place modification.

Time Complexity:

- **Brute Force:** O(n) where n is list length
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(n) for new list
- **Optimized:** O(1) in-place

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) for empty or single-element list
- **Average Case:** O(n)
- **Worst Case:** O(n)

Q25. Check if list is sorted in ascending order

Question: Determine if a list is sorted in non-decreasing order.

Solution 1 – Brute Force Approach:

```
def is_sorted_bf(lst):
    """Check sorted by comparing all adjacent pairs."""
    for i in range(len(lst) - 1):
        if lst[i] > lst[i + 1]:
            return False
    return True
```

Logic: Compare each element with its next neighbor. If any element is greater than next, list is not sorted.

Inefficiencies: Checks all pairs even if unsorted early. No significant optimization needed.

Solution 2 – Optimized Approach:

```
def is_sorted_opt(lst):
    """Check sorted using built-in function."""
    return all(lst[i] <= lst[i + 1] for i in range(len(lst) - 1))
```

Optimization: Use Python's all() with generator expression for early termination.

Justification: Cleaner code, same time complexity, early exit when unsorted found.

Time Complexity:

- **Brute Force:** O(n) where n is list length
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1) for generator

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if first two elements unsorted
- **Average Case:** O(n)
- **Worst Case:** O(n) if sorted or last pair unsorted

Q26. Second largest element in a list

Question: Find the second largest distinct element in a list.

Solution 1 – Brute Force Approach:

```
def second_largest_bf(lst):
    """Find second largest by sorting."""
    if len(lst) < 2:
        return None

    unique = list(set(lst))
    if len(unique) < 2:
        return None

    unique.sort()
    return unique[-2]
```

Logic: Remove duplicates with set, sort, return second last element.

Inefficiencies: O(n log n) due to sorting. Creates multiple copies.

Solution 2 – Optimized Approach:

```
def second_largest_opt(lst):
    """Find second largest in single pass."""
    if len(lst) < 2:
        return None

    first = second = float('-inf')

    for num in lst:
        if num > first:
            second = first
            first = num
        elif num > second and num != first:
            second = num

    return second if second != float('-inf') else None
```

Optimization: Track largest and second largest in single pass. Update when finding new max or new second max.

Justification: O(n) time vs O(n log n), O(1) space vs O(n).

Time Complexity:

- **Brute Force:** O(n log n) due to sorting
- **Optimized:** O(n) single pass

Space Complexity:

- **Brute Force:** O(n) for set and sorted list
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized

- **Average Case:** O(n log n) for brute force, O(n) for optimized
- **Worst Case:** O(n log n) for brute force, O(n) for optimized

Q27. Remove duplicates from a list

Question: Remove duplicate elements from a list, order doesn't matter.
Solution 1 – Brute Force Approach:

```python
def remove_duplicates_bf(lst):
    """Remove duplicates using nested loop."""
    result = []
    for i in range(len(lst)):
        if lst[i] not in result:
            result.append(lst[i])
    return result
```

Logic: For each element, check if already in result list, add if not present.
Inefficiencies: O(n^2) time - 'in' check is O(k) for result list of size k.
Solution 2 – Optimized Approach:

```python
def remove_duplicates_opt(lst):
    """Remove duplicates using set."""
    return list(set(lst))
```

Optimization: Convert to set (removes duplicates), convert back to list.
Justification: O(n) time vs O(n^2), uses hash set for O(1) lookups.
Time Complexity:

- **Brute Force:** O(n^2) where n is list length
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(n) for result list
- **Optimized:** O(n) for set

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** O(n^2) for brute force, O(n) for optimized
- **Worst Case:** O(n^2) for brute force, O(n) for optimized

Q28. Remove duplicates while preserving order

Question: Remove duplicate elements while maintaining original order of first occurrences.

Solution 1 – Brute Force Approach:

```
def remove_duplicates_ordered_bf(lst):
    """Remove duplicates preserving order using list."""
    seen = []
    result = []
    for item in lst:
        if item not in seen:
            seen.append(item)
            result.append(item)
    return result
```

Logic: Maintain 'seen' list to track elements, add to result when first seen.

Inefficiencies: O(n^2) - 'in' check on 'seen' list is O(k) for k seen elements.

Solution 2 – Optimized Approach:

```
def remove_duplicates_ordered_opt(lst):
    """Remove duplicates preserving order using set."""
    seen = set()
    result = []
    for item in lst:
        if item not in seen:
            seen.add(item)
            result.append(item)
    return result
```

Optimization: Use set for O(1) membership tests instead of list with O(k) tests.

Justification: O(n) time vs O(n^2), maintains order while using efficient lookups.

Time Complexity:

- **Brute Force:** O(n^2) where n is list length
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(n) for both seen and result lists
- **Optimized:** O(n) for set and result list

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** O(n^2) for brute force, O(n) for optimized
- **Worst Case:** O(n^2) for brute force, O(n) for optimized

Q29. Count frequency of each element

Question: Count how many times each distinct element appears in a list.

Solution 1 – Brute Force Approach:

```python
def count_frequency_bf(lst):
    """Count frequency using nested loops."""
    freq = {}
    for item in lst:
        if item not in freq:
            # Count occurrences
            count = 0
            for other in lst:
                if other == item:
                    count += 1
            freq[item] = count
    return freq
```

Logic: For each new element, count its occurrences by scanning entire list.

Inefficiencies: $O(n^2)$ time - for each unique element, scan entire list.

Solution 2 – Optimized Approach:

```python
def count_frequency_opt(lst):
    """Count frequency using dictionary."""
    freq = {}
    for item in lst:
        freq[item] = freq.get(item, 0) + 1
    return freq
```

Optimization: Single pass with dictionary. Update count for each element using hash map.

Justification: O(n) time vs $O(n^2)$, one pass instead of nested loops.

Time Complexity:

- **Brute Force:** $O(n^2)$ where n is list length
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(k) where k is number of unique elements
- **Optimized:** O(k) where k is number of unique elements

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** $O(n^2)$ for brute force, O(n) for optimized
- **Worst Case:** $O(n^2)$ for brute force, O(n) for optimized

Q30. Move zeros to end preserving order

Question: Move all zeros to the end while maintaining relative order of non-zero elements.

Solution 1 – Brute Force Approach:

```python
def move_zeros_bf(lst):
    """Move zeros by creating new lists."""
    non_zeros = [x for x in lst if x != 0]
    zeros = [0] * (len(lst) - len(non_zeros))
    return non_zeros + zeros
```

Logic: Create two lists - non-zeros and zeros, concatenate them.

Inefficiencies: Creates new lists, O(n) extra space. Multiple passes.

Solution 2 – Optimized Approach:

```python
def move_zeros_opt(lst):
    """Move zeros in-place using two-pointer."""
    non_zero_idx = 0

    # Move non-zeros to front
    for i in range(len(lst)):
        if lst[i] != 0:
            lst[non_zero_idx] = lst[i]
            non_zero_idx += 1

    # Fill remaining with zeros
    for i in range(non_zero_idx, len(lst)):
        lst[i] = 0

    return lst
```

Optimization: Two-pass in-place. First pass: move non-zeros to front. Second pass: fill end with zeros.

Justification: O(1) extra space vs O(n), in-place modification.

Time Complexity:

- **Brute Force:** O(n) where n is list length
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(n) for new lists
- **Optimized:** O(1) in-place

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for both
- **Average Case:** O(n) for both
- **Worst Case:** O(n) for both

Q31. Rotate list by K positions to right

Question: Rotate list elements right by K positions (circular shift).

Solution 1 – Brute Force Approach:

```python
def rotate_list_bf(lst, k):
    """Rotate by performing k shifts."""
    if not lst:
        return lst

    k = k % len(lst)  # Handle k > len(lst)
    for _ in range(k):
        last = lst.pop()
        lst.insert(0, last)
    return lst
```

Logic: Perform k shifts: pop last element, insert at front.

Inefficiencies: O(kn) time - each pop/insert is O(n) due to shifting elements.

Solution 2 – Optimized Approach:

```python
def rotate_list_opt(lst, k):
    """Rotate using slicing or reversal algorithm."""
    if not lst:
        return lst

    n = len(lst)
    k = k % n

    # Method 1: Slicing (Pythonic)
    return lst[-k:] + lst[:-k]

    # Method 2: Reversal algorithm (in-place)
    # Reverse entire list
    # Reverse first k elements
    # Reverse remaining n-k elements
```

Optimization: Use slicing for O(n) time, or reversal algorithm for O(1) extra space.

Justification: O(n) time vs O(kn), much faster for large k.

Time Complexity:

- **Brute Force:** O(kn) where n is list length
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(1) in-place
- **Optimized:** O(n) for slicing, O(1) for reversal algorithm

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** O(kn) for brute force, O(n) for optimized
- **Worst Case:** $O(n^2)$ for brute force (k=n), O(n) for optimized

Q32. Missing number in 1 to N list

Question: Find missing number in list containing 1 to N with one number missing.
Solution 1 – Brute Force Approach:

```python
def missing_number_bf(nums):
    """Find missing by checking each number."""
    n = len(nums) + 1
    for i in range(1, n + 1):
        if i not in nums:
            return i
```

Logic: Check each number 1 to N if present in list.
Inefficiencies: O(n^2) - 'in' check is O(n) for list, done n times.
Solution 2 – Optimized Approach:

```python
def missing_number_opt(nums):
    """Find missing using sum formula or XOR."""
    n = len(nums) + 1
    # Method 1: Sum formula
    total_sum = n * (n + 1) // 2
    actual_sum = sum(nums)
    return total_sum - actual_sum

    # Method 2: XOR (avoids overflow)
    # xor1 = XOR of 1 to N
    # xor2 = XOR of all nums
    # missing = xor1 ^ xor2
```

Optimization: Use mathematical formula: missing = total sum (1 to N) - actual sum.
Justification: O(n) time vs O(n^2), no nested loops.
Time Complexity:

- **Brute Force:** O(n^2) where n is list length
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if first number missing
- **Average Case:** O(n^2) for brute force, O(n) for optimized
- **Worst Case:** O(n^2) for brute force, O(n) for optimized

Q33. Find all duplicate elements

Question: Find all elements that appear more than once in a list.
Solution 1 – Brute Force Approach:

```python
def find_duplicates_bf(lst):
    """Find duplicates using nested loops."""
    duplicates = []
    n = len(lst)
    for i in range(n):
        for j in range(i + 1, n):
            if lst[i] == lst[j] and lst[i] not in duplicates:
                duplicates.append(lst[i])
    return duplicates
```

Logic: Compare each element with all later elements, add to duplicates if match found.
Inefficiencies: O(n^2) time, checks all pairs.
Solution 2 – Optimized Approach:

```python
def find_duplicates_opt(lst):
    """Find duplicates using frequency count."""
    freq = {}
    duplicates = []

    for item in lst:
        freq[item] = freq.get(item, 0) + 1

    for item, count in freq.items():
        if count > 1:
            duplicates.append(item)

    return duplicates
```

Optimization: Count frequencies with hash map in O(n), then find counts ¿ 1.
Justification: O(n) time vs O(n^2), linear time with hash map.
Time Complexity:

- **Brute Force:** O(n^2) where n is list length
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(k) where k is number of duplicates
- **Optimized:** O(u) where u is number of unique elements

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** O(n^2) for brute force, O(n) for optimized
- **Worst Case:** O(n^2) for brute force, O(n) for optimized

Q34. Intersection of two lists

Question: Find common elements between two lists.

Solution 1 – Brute Force Approach:

```
def intersection_bf(lst1, lst2):
    """Intersection using nested loops."""
    result = []
    for item1 in lst1:
        if item1 in lst2 and item1 not in result:
            result.append(item1)
    return result
```

Logic: For each element in list1, check if in list2, add to result if not already added.

Inefficiencies: O(n*m) where n,m are list lengths. 'in' on list2 is O(m).

Solution 2 – Optimized Approach:

```
def intersection_opt(lst1, lst2):
    """Intersection using sets."""
    set1 = set(lst1)
    set2 = set(lst2)
    return list(set1 & set2)
```

Optimization: Convert to sets, use set intersection which is O(min(n,m)).

Justification: O(n+m) time vs O(n*m), uses hash sets for efficient lookups.

Time Complexity:

- **Brute Force:** O(n*m) where n,m are list lengths
- **Optimized:** O(n+m) for conversion + O(min(n,m)) for intersection

Space Complexity:

- **Brute Force:** O(k) where k is intersection size
- **Optimized:** O(n+m) for sets

Best / Average / Worst Case Analysis:

- **Best Case:** O(n+m) for optimized
- **Average Case:** O(n*m) for brute force, O(n+m) for optimized
- **Worst Case:** O(n*m) for brute force, O(n+m) for optimized

Q35. Union of two lists

Question: Find all distinct elements from both lists.

Solution 1 – Brute Force Approach:

```
def union_bf(lst1, lst2):
    """Union by checking and adding."""
    union = []
    for item in lst1 + lst2:
        if item not in union:
            union.append(item)
    return union
```

Logic: Combine lists, add each element to result if not already present.

Inefficiencies: $O((n+m)^2)$ - 'in' check on union list is O(k) for k elements.

Solution 2 – Optimized Approach:

```
def union_opt(lst1, lst2):
    """Union using sets."""
    set1 = set(lst1)
    set2 = set(lst2)
    return list(set1 | set2)
```

Optimization: Convert to sets, use set union operation.

Justification: O(n+m) time vs $O((n+m)^2)$, set operations are efficient.

Time Complexity:

- **Brute Force:** $O((n+m)^2)$ where n,m are list lengths
- **Optimized:** O(n+m) for conversion + O(n+m) for union

Space Complexity:

- **Brute Force:** O(n+m) for result
- **Optimized:** O(n+m) for sets and result

Best / Average / Worst Case Analysis:

- **Best Case:** O(n+m) for optimized
- **Average Case:** $O((n+m)^2)$ for brute force, O(n+m) for optimized
- **Worst Case:** $O((n+m)^2)$ for brute force, O(n+m) for optimized

Q36. Find pairs with sum K

Question: Find all pairs (i,j) where i+j = K in a list.

Solution 1 – Brute Force Approach:

```
def find_pairs_bf(lst, K):
    """Find pairs using nested loops."""
    pairs = []
    n = len(lst)
    for i in range(n):
        for j in range(i + 1, n):
            if lst[i] + lst[j] == K:
                pairs.append((lst[i], lst[j]))
    return pairs
```

Logic: Check all possible pairs, add to result if sum equals K.

Inefficiencies: $O(n^2)$ time, checks all n(n-1)/2 pairs.

Solution 2 – Optimized Approach:

```
def find_pairs_opt(lst, K):
    """Find pairs using hash set."""
    seen = set()
    pairs = []

    for num in lst:
        complement = K - num
        if complement in seen:
            pairs.append((min(num, complement), max(num,
               complement)))
        seen.add(num)

    return pairs
```

Optimization: Use hash set to store seen numbers. For each number, check if complement (K-num) was seen.

Justification: O(n) time vs $O(n^2)$, one pass with O(1) lookups.

Time Complexity:

- **Brute Force:** $O(n^2)$ where n is list length
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(1) excluding output
- **Optimized:** O(n) for hash set

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** $O(n^2)$ for brute force, O(n) for optimized
- **Worst Case:** $O(n^2)$ for brute force, O(n) for optimized

Q37. Majority element

Question: Find element appearing more than n/2 times (if exists).

Solution 1 – Brute Force Approach:

```python
def majority_element_bf(lst):
    """Find majority by counting each element."""
    n = len(lst)
    for num in lst:
        count = 0
        for other in lst:
            if other == num:
                count += 1
        if count > n // 2:
            return num
    return None
```

Logic: For each element, count its occurrences by scanning entire list.

Inefficiencies: $O(n^2)$ time, counts each element separately.

Solution 2 – Optimized Approach:

```python
def majority_element_opt(lst):
    """Find majority using Boyer-Moore Voting Algorithm."""
    candidate = None
    count = 0

    # Phase 1: Find candidate
    for num in lst:
        if count == 0:
            candidate = num
        count += (1 if num == candidate else -1)

    # Phase 2: Verify candidate
    if lst.count(candidate) > len(lst) // 2:
        return candidate
    return None
```

Optimization: Boyer-Moore algorithm: cancel out pairs of different elements, remaining candidate is potential majority.

Justification: O(n) time vs $O(n^2)$, O(1) space, two passes.

Time Complexity:

- **Brute Force:** $O(n^2)$ where n is list length
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized

- **Average Case:** $O(n^2)$ for brute force, $O(n)$ for optimized
- **Worst Case:** $O(n^2)$ for brute force, $O(n)$ for optimized

Q38. Leader elements in array

Question: Find elements greater than all elements to their right.

Solution 1 – Brute Force Approach:

```python
def find_leaders_bf(lst):
    """Find leaders using nested loops."""
    leaders = []
    n = len(lst)
    for i in range(n):
        is_leader = True
        for j in range(i + 1, n):
            if lst[i] < lst[j]:
                is_leader = False
                break
        if is_leader:
            leaders.append(lst[i])
    return leaders
```

Logic: For each element, check all elements to its right to see if any is greater.

Inefficiencies: $O(n^2)$ time, checks all right elements for each position.

Solution 2 – Optimized Approach:

```python
def find_leaders_opt(lst):
    """Find leaders by scanning from right."""
    leaders = []
    max_from_right = float('-inf')

    for i in range(len(lst) - 1, -1, -1):
        if lst[i] > max_from_right:
            leaders.append(lst[i])
            max_from_right = lst[i]

    return leaders[::-1]  # Reverse to get left-to-right order
```

Optimization: Scan from right to left, track maximum seen so far. Element is leader if greater than max from right.

Justification: $O(n)$ time vs $O(n^2)$, single pass from right.

Time Complexity:

- **Brute Force:** $O(n^2)$ where n is list length
- **Optimized:** $O(n)$ where n is list length

Space Complexity:

- **Brute Force:** $O(1)$ excluding output
- **Optimized:** $O(1)$ excluding output

Best / Average / Worst Case Analysis:

- **Best Case:** $O(n)$ for optimized
- **Average Case:** $O(n^2)$ for brute force, $O(n)$ for optimized
- **Worst Case:** $O(n^2)$ for brute force, $O(n)$ for optimized

Q39. Positive before negative numbers

Question: Rearrange array so positive numbers come before negative numbers.

Solution 1 – Brute Force Approach:

```python
def rearrange_bf(lst):
    """Rearrange by creating new lists."""
    positives = [x for x in lst if x >= 0]
    negatives = [x for x in lst if x < 0]
    return positives + negatives
```

Logic: Create two separate lists, concatenate them.

Inefficiencies: O(n) extra space, two passes through list.

Solution 2 – Optimized Approach:

```python
def rearrange_opt(lst):
    """Rearrange in-place using two-pointer."""
    left, right = 0, len(lst) - 1

    while left < right:
        # Find negative from left
        while left < right and lst[left] >= 0:
            left += 1
        # Find positive from right
        while left < right and lst[right] < 0:
            right -= 1

        if left < right:
            lst[left], lst[right] = lst[right], lst[left]
            left += 1
            right -= 1

    return lst
```

Optimization: Two-pointer technique from both ends. Swap when left finds negative and right finds positive.

Justification: O(1) extra space vs O(n), in-place rearrangement.

Time Complexity:

- **Brute Force:** O(n) where n is list length
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(n) for new lists
- **Optimized:** O(1) in-place

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for both
- **Average Case:** O(n) for both
- **Worst Case:** O(n) for both

Q40. Maximum subarray sum (Kadane's)

Question: Find maximum sum of contiguous subarray.

Solution 1 – Brute Force Approach:

```
def max_subarray_sum_bf(lst):
    """Maximum subarray sum using all subarrays."""
    max_sum = float('-inf')
    n = len(lst)

    for i in range(n):
        for j in range(i, n):
            current_sum = sum(lst[i:j+1])
            if current_sum > max_sum:
                max_sum = current_sum

    return max_sum
```

Logic: Generate all subarrays, compute sum for each, track maximum.

Inefficiencies: $O(n^3)$ time - n^2 subarrays, each sum calculation is O(k).

Solution 2 – Optimized Approach:

```
def max_subarray_sum_opt(lst):
    """Maximum subarray sum using Kadane's algorithm."""
    max_so_far = float('-inf')
    max_ending_here = 0

    for num in lst:
        max_ending_here = max(num, max_ending_here + num)
        max_so_far = max(max_so_far, max_ending_here)

    return max_so_far
```

Optimization: Kadane's algorithm: track maximum sum ending at current position. Either start new subarray or extend previous.

Justification: O(n) time vs $O(n^3)$, single pass dynamic programming.

Time Complexity:

- **Brute Force:** $O(n^3)$ where n is list length
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** $O(n^3)$ for brute force, O(n) for optimized
- **Worst Case:** $O(n^3)$ for brute force, O(n) for optimized

Q41. Equilibrium index of array

Question: Find index where sum of left elements equals sum of right elements.

Solution 1 – Brute Force Approach:

```python
def equilibrium_index_bf(lst):
    """Equilibrium index by computing left/right sums for each
       index."""
    n = len(lst)
    for i in range(n):
        left_sum = sum(lst[:i])
        right_sum = sum(lst[i+1:])
        if left_sum == right_sum:
            return i
    return -1
```

Logic: For each index, compute sum of left and right subarrays, compare.

Inefficiencies: O(n^2) time - each sum computation is O(k), done n times.

Solution 2 – Optimized Approach:

```python
def equilibrium_index_opt(lst):
    """Equilibrium index using prefix sums."""
    total_sum = sum(lst)
    left_sum = 0

    for i, num in enumerate(lst):
        # right_sum = total_sum - left_sum - num
        right_sum = total_sum - left_sum - num
        if left_sum == right_sum:
            return i
        left_sum += num

    return -1
```

Optimization: Track left sum incrementally, compute right sum as total - left - current.

Justification: O(n) time vs O(n^2), single pass with prefix sum tracking.

Time Complexity:

- **Brute Force:** O(n^2) where n is list length
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if first index is equilibrium
- **Average Case:** O(n^2) for brute force, O(n) for optimized
- **Worst Case:** O(n^2) for brute force, O(n) for optimized

Q42. Product of array except self

Question: For each element, compute product of all other elements.

Solution 1 – Brute Force Approach:

```python
def product_except_self_bf(lst):
    """Product except self using nested loops."""
    n = len(lst)
    result = [1] * n

    for i in range(n):
        for j in range(n):
            if i != j:
                result[i] *= lst[j]

    return result
```

Logic: For each position, multiply all other elements.

Inefficiencies: $O(n^2)$ time, recomputes products for each element.

Solution 2 – Optimized Approach:

```python
def product_except_self_opt(lst):
    """Product except self using prefix and suffix products."""
    n = len(lst)
    result = [1] * n

    # Left products
    left_product = 1
    for i in range(n):
        result[i] = left_product
        left_product *= lst[i]

    # Right products
    right_product = 1
    for i in range(n - 1, -1, -1):
        result[i] *= right_product
        right_product *= lst[i]

    return result
```

Optimization: Two-pass approach: compute left products in first pass, multiply with right products in second pass.

Justification: $O(n)$ time vs $O(n^2)$, avoids division (which handles zeros better).

Time Complexity:

- **Brute Force:** $O(n^2)$ where n is list length
- **Optimized:** $O(n)$ where n is list length

Space Complexity:

- **Brute Force:** $O(1)$ excluding output
- **Optimized:** $O(1)$ excluding output (output array doesn't count as extra)

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** $O(n^2)$ for brute force, O(n) for optimized
- **Worst Case:** $O(n^2)$ for brute force, O(n) for optimized

Q43. Sort 0s, 1s, and 2s (Dutch Flag)

Question: Sort array containing only 0, 1, 2 in single pass.

Solution 1 – Brute Force Approach:

```python
def sort_colors_bf(lst):
    """Sort by counting and rebuilding."""
    count_0 = count_1 = count_2 = 0

    for num in lst:
        if num == 0:
            count_0 += 1
        elif num == 1:
            count_1 += 1
        else:
            count_2 += 1

    return [0] * count_0 + [1] * count_1 + [2] * count_2
```

Logic: Count occurrences of each, create new array with correct counts.

Inefficiencies: Two passes (count + build), creates new array.

Solution 2 – Optimized Approach:

```python
def sort_colors_opt(lst):
    """Sort in-place using Dutch National Flag algorithm."""
    low, mid, high = 0, 0, len(lst) - 1

    while mid <= high:
        if lst[mid] == 0:
            lst[low], lst[mid] = lst[mid], lst[low]
            low += 1
            mid += 1
        elif lst[mid] == 1:
            mid += 1
        else:  # lst[mid] == 2
            lst[mid], lst[high] = lst[high], lst[mid]
            high -= 1

    return lst
```

Optimization: Three-pointer Dutch flag algorithm: maintain regions for 0s, 1s, 2s.

Justification: O(n) single pass, O(1) space, in-place sorting.

Time Complexity:

- **Brute Force:** O(n) where n is list length
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(n) for new array or O(1) if counts only
- **Optimized:** O(1) in-place

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for both
- **Average Case:** O(n) for both
- **Worst Case:** O(n) for both

Q44. Merge two sorted arrays

Question: Merge two sorted arrays into one sorted array.

Solution 1 – Brute Force Approach:

```python
def merge_sorted_bf(arr1, arr2):
    """Merge by concatenating and sorting."""
    merged = arr1 + arr2
    merged.sort()
    return merged
```

Logic: Concatenate arrays, sort the combined array.

Inefficiencies: O((n+m)log(n+m)) time due to sorting, ignores that inputs are sorted.

Solution 2 – Optimized Approach:

```python
def merge_sorted_opt(arr1, arr2):
    """Merge using two-pointer technique."""
    i, j = 0, 0
    merged = []

    while i < len(arr1) and j < len(arr2):
        if arr1[i] <= arr2[j]:
            merged.append(arr1[i])
            i += 1
        else:
            merged.append(arr2[j])
            j += 1

    # Add remaining elements
    merged.extend(arr1[i:])
    merged.extend(arr2[j:])

    return merged
```

Optimization: Two-pointer merge: compare front elements, add smaller to result.

Justification: O(n+m) time vs O((n+m)log(n+m)), linear merge using sorted property.

Time Complexity:

- **Brute Force:** O((n+m)log(n+m)) where n,m are array lengths
- **Optimized:** O(n+m) where n,m are array lengths

Space Complexity:

- **Brute Force:** O(n+m) for new array
- **Optimized:** O(n+m) for result

Best / Average / Worst Case Analysis:

- **Best Case:** O(n+m) for optimized
- **Average Case:** O((n+m)log(n+m)) for brute force, O(n+m) for optimized
- **Worst Case:** O((n+m)log(n+m)) for brute force, O(n+m) for optimized

Q45. Kth largest element

Question: Find Kth largest element in unsorted array.

Solution 1 – Brute Force Approach:

```python
def kth_largest_bf(lst, k):
    """Find Kth largest by sorting."""
    if k > len(lst):
        return None
    lst.sort()
    return lst[-k]
```

Logic: Sort array, return element at position len(lst)-k.

Inefficiencies: O(n log n) time for full sort when we only need Kth element.

Solution 2 – Optimized Approach:

```python
import heapq

def kth_largest_opt(lst, k):
    """Find Kth largest using min-heap."""
    if k > len(lst):
        return None

    # Create min-heap of first k elements
    min_heap = lst[:k]
    heapq.heapify(min_heap)

    # Process remaining elements
    for num in lst[k:]:
        if num > min_heap[0]:  # Larger than smallest in heap
            heapq.heapreplace(min_heap, num)

    return min_heap[0]  # Root is Kth largest
```

Optimization: Use min-heap of size K. Maintain K largest elements, root is Kth largest.

Justification: O(n log k) time vs O(n log n), better when k ¡¡ n.

Time Complexity:

- **Brute Force:** O(n log n) where n is list length
- **Optimized:** O(n log k) where n is list length, k is parameter

Space Complexity:

- **Brute Force:** O(1) if sort in-place, O(n) if not
- **Optimized:** O(k) for heap

Best / Average / Worst Case Analysis:

- **Best Case:** O(n log n) for brute force, O(n) for optimized when k=1
- **Average Case:** O(n log n) for brute force, O(n log k) for optimized
- **Worst Case:** O(n log n) for brute force, O(n log k) for optimized

Q46. Peak element in array

Question: Find a peak element (greater than neighbors) in array.

Solution 1 – Brute Force Approach:

```python
def find_peak_bf(lst):
    """Find peak by checking each element."""
    n = len(lst)
    if n == 0:
        return None
    if n == 1:
        return 0

    for i in range(n):
        left = lst[i-1] if i > 0 else float('-inf')
        right = lst[i+1] if i < n-1 else float('-inf')
        if lst[i] >= left and lst[i] >= right:
            return i

    return None
```

Logic: Check each element against its neighbors, return first peak found.

Inefficiencies: O(n) time worst case, checks all elements.

Solution 2 – Optimized Approach:

```python
def find_peak_opt(lst):
    """Find peak using binary search."""
    n = len(lst)
    if n == 0:
        return None

    left, right = 0, n - 1

    while left < right:
        mid = (left + right) // 2

        if lst[mid] > lst[mid + 1]:
            # Peak in left half (including mid)
            right = mid
        else:
            # Peak in right half
            left = mid + 1

    return left
```

Optimization: Binary search approach. Compare mid with mid+1, move toward larger neighbor.

Justification: O(log n) time vs O(n), uses the property that there's always a peak.

Time Complexity:

- **Brute Force:** O(n) where n is list length
- **Optimized:** O(log n) where n is list length

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if first element is peak
- **Average Case:** O(n) for brute force, O(log n) for optimized
- **Worst Case:** O(n) for brute force, O(log n) for optimized

Q47. Check if array is palindrome

Question: Check if array reads same forwards and backwards.

Solution 1 – Brute Force Approach:

```python
def is_array_palindrome_bf(lst):
    """Check palindrome by creating reverse."""
    reversed_lst = []
    for i in range(len(lst) - 1, -1, -1):
        reversed_lst.append(lst[i])
    return lst == reversed_lst
```

Logic: Create reversed copy, compare with original.

Inefficiencies: O(n) extra space for reversed copy, two passes.

Solution 2 – Optimized Approach:

```python
def is_array_palindrome_opt(lst):
    """Check palindrome using two pointers."""
    left, right = 0, len(lst) - 1

    while left < right:
        if lst[left] != lst[right]:
            return False
        left += 1
        right -= 1

    return True
```

Optimization: Two-pointer technique: compare from both ends moving inward.

Justification: O(1) extra space vs O(n), in-place comparison.

Time Complexity:

- **Brute Force:** O(n) where n is list length
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(n) for reversed copy
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if first and last elements differ
- **Average Case:** O(n) for both
- **Worst Case:** O(n) for both (when palindrome)

Q48. Split array with equal sum

Question: Check if array can be split into two subarrays with equal sum.

Solution 1 – Brute Force Approach:

```python
def can_split_equal_sum_bf(lst):
    """Check all possible split points."""
    total = sum(lst)

    for i in range(len(lst)):
        left_sum = sum(lst[:i])
        right_sum = total - left_sum
        if left_sum == right_sum:
            return True

    return False
```

Logic: Try every split point, compute left and right sums, compare.

Inefficiencies: O(n^2) time - sum computation for each split is O(k).

Solution 2 – Optimized Approach:

```python
def can_split_equal_sum_opt(lst):
    """Check using prefix sum tracking."""
    total = sum(lst)
    left_sum = 0

    for i in range(len(lst) - 1):  # Don't include last element
        left_sum += lst[i]
        right_sum = total - left_sum
        if left_sum == right_sum:
            return True

    return False
```

Optimization: Track left sum incrementally, compute right sum as total - left.

Justification: O(n) time vs O(n^2), single pass with cumulative sum.

Time Complexity:

- **Brute Force:** O(n^2) where n is list length
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if split at first position works
- **Average Case:** O(n^2) for brute force, O(n) for optimized
- **Worst Case:** O(n^2) for brute force, O(n) for optimized

Q49. Minimum jumps to reach end

Question: Find minimum jumps needed to reach end where each element shows max jump from that position.

Solution 1 – Brute Force Approach:

```
def min_jumps_bf(jumps):
    """Find min jumps using recursion/backtracking."""
    def dfs(position):
        if position >= len(jumps) - 1:
            return 0

        min_jump = float('inf')
        max_step = jumps[position]

        for step in range(1, max_step + 1):
            if position + step < len(jumps):
                min_jump = min(min_jump, 1 + dfs(position +
                    step))

        return min_jump

    return dfs(0)
```

Logic: Recursively try all possible jumps from each position, find minimum.

Inefficiencies: Exponential time $O(2^n)$ *due to exploring all paths. Overlapping subproblems.*

Solution 2 – Optimized Approach:

```
def min_jumps_opt(jumps):
    """Find min jumps using greedy BFS."""
    if len(jumps) <= 1:
        return 0

    jumps_needed = 0
    current_end = 0
    farthest = 0

    for i in range(len(jumps) - 1):
        farthest = max(farthest, i + jumps[i])

        if i == current_end:
            jumps_needed += 1
            current_end = farthest

            if current_end >= len(jumps) - 1:
                break

    return jumps_needed if current_end >= len(jumps) - 1 else -1
```

Optimization: Greedy BFS: track farthest reachable, increment jump when current range exhausted.

Justification: O(n) time vs exponential, greedy approach finds optimal solution for this problem.

Time Complexity:

- **Brute Force:** O(2^n)$wherenislistlength$
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(n) recursion stack
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** O(2^n)$forbruteforce, O(n)foroptimized$
- **Worst Case:** O(2^n)$forbruteforce, O(n)foroptimized$

Q50. Longest increasing subsequence length

Question: Find length of longest increasing subsequence (not necessarily contiguous).

Solution 1 – Brute Force Approach:

```python
def lis_length_bf(lst):
    """LIS length using recursion exploring all subsequences."""
    def dfs(prev_idx, current_idx):
        if current_idx == len(lst):
            return 0

        taken = 0
        if prev_idx == -1 or lst[current_idx] > lst[prev_idx]:
            taken = 1 + dfs(current_idx, current_idx + 1)

        not_taken = dfs(prev_idx, current_idx + 1)

        return max(taken, not_taken)

    return dfs(-1, 0)
```

Logic: For each element, either include it (if increasing) or skip it, explore both paths.

Inefficiencies: $O(2^n) time, exponential due to exploring all subsequences.$

Solution 2 – Optimized Approach:

```python
def lis_length_opt(lst):
    """LIS length using DP with binary search (patience
       sorting)."""
    if not lst:
        return 0

    tails = [0] * len(lst)
    size = 0

    for num in lst:
        # Binary search for position to replace/append
        left, right = 0, size
        while left < right:
            mid = (left + right) // 2
            if tails[mid] < num:
                left = mid + 1
            else:
                right = mid

        tails[left] = num
        if left == size:
            size += 1

    return size
```

Optimization: Patience sorting algorithm: maintain array of smallest possible tail values for increasing sequences of each length, use binary search.

Justification: O(n log n) time vs $O(2^n), optimal solution using DP with binary search.$

Time Complexity:

- **Brute Force:** $O(2^n) where n is list length$
- **Optimized:** O(n log n) where n is list length

Space Complexity:

- **Brute Force:** O(n) recursion stack
- **Optimized:** O(n) for tails array

Best / Average / Worst Case Analysis:

- **Best Case:** O(n log n) for optimized
- **Average Case:** $O(2^n) for brute force, O(nlogn) for optimized$
- **Worst Case:** $O(2^n) for brute force, O(nlogn) for optimized$

SECTION C: STRINGS (30 Questions)

Q51. Reverse a string

Question: Reverse the order of characters in a string.

Solution 1 – Brute Force Approach:

```python
def reverse_string_bf(s):
    """Reverse using string concatenation."""
    reversed_str = ""
    for i in range(len(s) - 1, -1, -1):
        reversed_str += s[i]
    return reversed_str
```

Logic: Iterate string backwards, append each character to new string.

Inefficiencies: $O(n^2)$ time due to string concatenation (strings immutable, each += creates new string).

Solution 2 – Optimized Approach:

```python
def reverse_string_opt(s):
    """Reverse using slicing or list join."""
    # Method 1: Slicing (Pythonic)
    return s[::-1]

    # Method 2: List join
    # return ''.join(reversed(s))

    # Method 3: Two-pointer in-place (if mutable)
    # chars = list(s)
    # left, right = 0, len(chars)-1
    # while left < right:
    #     chars[left], chars[right] = chars[right], chars[left]
    #     left += 1; right -= 1
    # return ''.join(chars)
```

Optimization: Use slicing (O(n) time) or convert to list and join.
Justification: O(n) time vs $O(n^2)$, Python slicing is highly optimized in C.
Time Complexity:

- **Brute Force:** $O(n^2)$ where n is string length
- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(n) for new string
- **Optimized:** O(n) for new string

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** $O(n^2)$ for brute force, O(n) for optimized
- **Worst Case:** $O(n^2)$ for brute force, O(n) for optimized

Q52. Check if string is palindrome

Question: Check if string reads same forwards and backwards ignoring case/punctuation.

Solution 1 – Brute Force Approach:

```python
def is_palindrome_bf(s):
    """Check palindrome by creating reversed string."""
    # Clean string: lowercase, remove non-alphanumeric
    cleaned = ''.join(c.lower() for c in s if c.isalnum())
    return cleaned == cleaned[::-1]
```

Logic: Clean string, compare with its reverse.

Inefficiencies: Creates reversed copy (O(n) extra space). Two passes for cleaning.

Solution 2 – Optimized Approach:

```python
def is_palindrome_opt(s):
    """Check palindrome using two pointers."""
    left, right = 0, len(s) - 1

    while left < right:
        # Skip non-alphanumeric from left
        while left < right and not s[left].isalnum():
            left += 1
        # Skip non-alphanumeric from right
        while left < right and not s[right].isalnum():
            right -= 1

        if s[left].lower() != s[right].lower():
            return False

        left += 1
        right -= 1

    return True
```

Optimization: Two-pointer in-place comparison. Skip non-alphanumeric, compare case-insensitively.

Justification: O(1) extra space vs O(n), single pass in-place.

Time Complexity:

- **Brute Force:** O(n) where n is string length
- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(n) for cleaned and reversed strings
- **Optimized:** O(1) in-place

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if first and last characters differ
- **Average Case:** O(n) for both
- **Worst Case:** O(n) for both (when palindrome)

Q53. Count vowels in a string

Question: Count number of vowels (a, e, i, o, u) in a string, case-insensitive.

Solution 1 – Brute Force Approach:

```
def count_vowels_bf(s):
    """Count vowels using multiple checks."""
    count = 0
    for char in s:
        if (char == 'a' or char == 'e' or char == 'i' or
            char == 'o' or char == 'u' or
            char == 'A' or char == 'E' or char == 'I' or
            char == 'O' or char == 'U'):
            count += 1
    return count
```

Logic: Check each character against all vowel possibilities.

Inefficiencies: Multiple OR conditions per character. Case-sensitive checking.

Solution 2 – Optimized Approach:

```
def count_vowels_opt(s):
    """Count vowels using set membership."""
    vowels = set('aeiouAEIOU')
    count = 0
    for char in s:
        if char in vowels:
            count += 1
    return count

    # Or using comprehension:
    # return sum(1 for char in s if char in vowels)
```

Optimization: Use set for O(1) membership test. Convert to lowercase once.

Justification: O(1) per character check vs multiple comparisons. Cleaner code.

Time Complexity:

- **Brute Force:** O(n) where n is string length
- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1) for set (constant size 10)

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for both
- **Average Case:** O(n) for both
- **Worst Case:** O(n) for both

Q54. Frequency of each character

Question: Count frequency of each character in a string.

Solution 1 – Brute Force Approach:

```python
def char_frequency_bf(s):
    """Count frequency using nested loops."""
    freq = {}
    for i in range(len(s)):
        if s[i] not in freq:
            count = 0
            for j in range(len(s)):
                if s[i] == s[j]:
                    count += 1
            freq[s[i]] = count
    return freq
```

Logic: For each character, count its occurrences by scanning entire string.

Inefficiencies: O(n^2) time, counts each character separately.

Solution 2 – Optimized Approach:

```python
def char_frequency_opt(s):
    """Count frequency using dictionary."""
    freq = {}
    for char in s:
        freq[char] = freq.get(char, 0) + 1
    return freq

    # Or using collections.Counter:
    # from collections import Counter
    # return dict(Counter(s))
```

Optimization: Single pass with hash map. Update count for each character.

Justification: O(n) time vs O(n^2), one pass instead of nested loops.

Time Complexity:

- **Brute Force:** O(n^2) where n is string length
- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(k) where k is unique characters
- **Optimized:** O(k) where k is unique characters

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** O(n^2) for brute force, O(n) for optimized
- **Worst Case:** O(n^2) for brute force, O(n) for optimized

Q55. Remove duplicate characters

Question: Remove duplicate characters from string while preserving order of first occurrence.

Solution 1 – Brute Force Approach:

```
def remove_duplicates_bf(s):
    """Remove duplicates using list check."""
    result = []
    for char in s:
        if char not in result:
            result.append(char)
    return ''.join(result)
```

Logic: Maintain result list, add character if not already present.

Inefficiencies: O(n^2) time - 'in' check on list is O(k) for k unique characters.

Solution 2 – Optimized Approach:

```
def remove_duplicates_opt(s):
    """Remove duplicates using set."""
    seen = set()
    result = []
    for char in s:
        if char not in seen:
            seen.add(char)
            result.append(char)
    return ''.join(result)
```

Optimization: Use set for O(1) membership tests instead of list with O(k) tests.

Justification: O(n) time vs O(n^2), maintains order with efficient lookups.

Time Complexity:

- **Brute Force:** O(n^2) where n is string length
- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(n) for result list
- **Optimized:** O(k) for set and result, where k is unique characters

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** O(n^2) for brute force, O(n) for optimized
- **Worst Case:** O(n^2) for brute force, O(n) for optimized

Q56. Check if two strings are anagrams

Question: Check if two strings contain same characters in same frequencies (anagrams).

Solution 1 – Brute Force Approach:

```python
def are_anagrams_bf(s1, s2):
    """Check anagrams by checking character counts."""
    if len(s1) != len(s2):
        return False

    # For each char in s1, count occurrences in both strings
    for char in s1:
        if s1.count(char) != s2.count(char):
            return False

    return True
```

Logic: For each character, compare its count in both strings using count().

Inefficiencies: O(n^2) time - count() is O(n) for each character.

Solution 2 – Optimized Approach:

```python
def are_anagrams_opt(s1, s2):
    """Check anagrams using sorted strings or frequency count."""
    if len(s1) != len(s2):
        return False

    # Method 1: Sort and compare
    # return sorted(s1) == sorted(s2)  # O(n log n)

    # Method 2: Frequency count
    freq = [0] * 256  # Assuming ASCII

    for char in s1:
        freq[ord(char)] += 1

    for char in s2:
        freq[ord(char)] -= 1
        if freq[ord(char)] < 0:
            return False

    return True
```

Optimization: Use frequency array (O(n)) or sorted comparison (O(n log n)).

Justification: O(n) time vs O(n^2), linear time with constant space for ASCII.

Time Complexity:

- **Brute Force:** O(n^2) where n is string length
- **Optimized:** O(n) where n is string length (frequency array)

Space Complexity:

- **Brute Force:** O(1)

- **Optimized:** O(1) for frequency array (constant 256)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if lengths differ
- **Average Case:** $O(n^2)$ for brute force, O(n) for optimized
- **Worst Case:** $O(n^2)$ for brute force, O(n) for optimized

Q57. First non-repeating character

Question: Find first character that appears only once in string.

Solution 1 – Brute Force Approach:

```python
def first_non_repeating_bf(s):
    """Find first non-repeating using nested loops."""
    for i in range(len(s)):
        repeating = False
        for j in range(len(s)):
            if i != j and s[i] == s[j]:
                repeating = True
                break
        if not repeating:
            return s[i]
    return None
```

Logic: For each character, check all other positions for duplicates.

Inefficiencies: O(n^2) time, checks all pairs.

Solution 2 – Optimized Approach:

```python
def first_non_repeating_opt(s):
    """Find first non-repeating using frequency count."""
    freq = {}

    # First pass: count frequencies
    for char in s:
        freq[char] = freq.get(char, 0) + 1

    # Second pass: find first with count 1
    for char in s:
        if freq[char] == 1:
            return char

    return None
```

Optimization: Two-pass with frequency dictionary. First pass counts, second pass finds first count=1.

Justification: O(n) time vs O(n^2), two linear passes.

Time Complexity:

- **Brute Force:** O(n^2) where n is string length
- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(k) where k is unique characters

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if first character is non-repeating

- **Average Case:** $O(n^2)$ for brute force, $O(n)$ for optimized
- **Worst Case:** $O(n^2)$ for brute force, $O(n)$ for optimized

Q58. Last non-repeating character

Question: Find last character that appears only once in string.

Solution 1 – Brute Force Approach:

```python
def last_non_repeating_bf(s):
    """Find last non-repeating using nested loops."""
    for i in range(len(s) - 1, -1, -1):
        repeating = False
        for j in range(len(s)):
            if i != j and s[i] == s[j]:
                repeating = True
                break
        if not repeating:
            return s[i]
    return None
```

Logic: Similar to Q57 but iterate backwards, check for duplicates.

Inefficiencies: O(n^2) time, same inefficiency as Q57.

Solution 2 – Optimized Approach:

```python
def last_non_repeating_opt(s):
    """Find last non-repeating using frequency count."""
    freq = {}

    # First pass: count frequencies
    for char in s:
        freq[char] = freq.get(char, 0) + 1

    # Second pass: find last with count 1 (iterate backwards)
    for i in range(len(s) - 1, -1, -1):
        if freq[s[i]] == 1:
            return s[i]

    return None
```

Optimization: Two-pass with frequency dictionary. Second pass iterates backwards to find last count=1.

Justification: O(n) time vs O(n^2), similar to Q57 with backward iteration.

Time Complexity:

- **Brute Force:** O(n^2) where n is string length
- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(k) where k is unique characters

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if last character is non-repeating

- **Average Case:** $O(n^2)$ for brute force, $O(n)$ for optimized
- **Worst Case:** $O(n^2)$ for brute force, $O(n)$ for optimized

Q59. Reverse words in a sentence

Question: Reverse order of words in a sentence while preserving word order.

Solution 1 – Brute Force Approach:

```python
def reverse_words_bf(s):
    """Reverse words using string manipulation."""
    words = []
    current_word = ""

    # Split words manually
    for char in s:
        if char == ' ':
            if current_word:
                words.append(current_word)
                current_word = ""
        else:
            current_word += char

    if current_word:
        words.append(current_word)

    # Reverse and join
    reversed_words = []
    for i in range(len(words) - 1, -1, -1):
        reversed_words.append(words[i])

    return ' '.join(reversed_words)
```

Logic: Manually split into words, store in list, reverse list, join with spaces.

Inefficiencies: Manual splitting, multiple passes, string concatenation.

Solution 2 – Optimized Approach:

```python
def reverse_words_opt(s):
    """Reverse words using built-in functions."""
    # Split on whitespace (handles multiple spaces)
    words = s.split()
    # Reverse and join
    return ' '.join(reversed(words))

    # One-liner: return ' '.join(reversed(s.split()))
```

Optimization: Use built-in split() and join() functions which are optimized in C.

Justification: Cleaner, more efficient, handles edge cases (multiple spaces).

Time Complexity:

- **Brute Force:** O(n) where n is string length
- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(n) for words lists

- **Optimized:** O(n) for words list

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for both
- **Average Case:** O(n) for both
- **Worst Case:** O(n) for both

Q60. Count number of words

Question: Count number of words in a string (words separated by whitespace).
Solution 1 – Brute Force Approach:

```python
def count_words_bf(s):
    """Count words by tracking state."""
    word_count = 0
    in_word = False

    for char in s:
        if char == ' ':
            if in_word:
                word_count += 1
                in_word = False
        else:
            in_word = True

    # Count last word if string doesn't end with space
    if in_word:
        word_count += 1

    return word_count
```

Logic: Track whether currently inside a word, increment count when word ends.
Inefficiencies: Manual state tracking, handles edge cases manually.
Solution 2 – Optimized Approach:

```python
def count_words_opt(s):
    """Count words using split()."""
    # split() without arguments handles multiple whitespace
    return len(s.split())
```

Optimization: Use split() which handles all whitespace and edge cases.
Justification: Simpler, more robust, built-in function handles complexities.
Time Complexity:

- **Brute Force:** O(n) where n is string length
- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(n) for split list

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for both
- **Average Case:** O(n) for both
- **Worst Case:** O(n) for both

Q61. Longest word in sentence

Question: Find longest word in a sentence (by character count).

Solution 1 – Brute Force Approach:

```
def longest_word_bf(s):
    """Find longest word by manual tracking."""
    longest = ""
    current_word = ""

    for char in s:
        if char == ' ':
            if len(current_word) > len(longest):
                longest = current_word
            current_word = ""
        else:
            current_word += char

    # Check last word
    if len(current_word) > len(longest):
        longest = current_word

    return longest
```

Logic: Build words character by character, track longest seen.

Inefficiencies: String concatenation creates new strings. Manual word building.

Solution 2 – Optimized Approach:

```
def longest_word_opt(s):
    """Find longest word using split and max."""
    words = s.split()
    if not words:
        return ""

    # Using max with key function
    return max(words, key=len)

    # Alternative: manual loop with split
    # longest = words[0]
    # for word in words[1:]:
    #     if len(word) > len(longest):
    #         longest = word
    # return longest
```

Optimization: Use split() for word extraction, max() with key=len for finding longest.

Justification: Cleaner, uses optimized built-ins, handles punctuation better.

Time Complexity:

- **Brute Force:** O(n) where n is string length
- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(m) for longest word
- **Optimized:** O(n) for words list

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for both
- **Average Case:** O(n) for both
- **Worst Case:** O(n) for both

Q62. Check if string contains only digits

Question: Check if string contains only numeric digits (0-9).

Solution 1 – Brute Force Approach:

```
def is_digits_only_bf(s):
    """Check digits by comparing with digit characters."""
    for char in s:
        if not (char >= '0' and char <= '9'):
            return False
    return True
```

Logic: Check each character's ASCII value is between '0' and '9'.

Inefficiencies: Manual ASCII comparison, doesn't use built-in functions.

Solution 2 – Optimized Approach:

```
def is_digits_only_opt(s):
    """Check digits using isdigit() or all()."""
    # Method 1: isdigit() handles unicode digits too
    return s.isdigit()

    # Method 2: Using all()
    # return all(char.isdigit() for char in s)
```

Optimization: Use str.isdigit() which handles Unicode and is optimized.

Justification: More robust (handles Unicode), cleaner, built-in is optimized.

Time Complexity:

- **Brute Force:** O(n) where n is string length
- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if first character non-digit
- **Average Case:** O(n) for both
- **Worst Case:** O(n) for both (when all digits)

Q63. String to integer without int()

Question: Convert numeric string to integer without using int() function.

Solution 1 – Brute Force Approach:

```
def str_to_int_bf(s):
    """Convert string to int using digit mapping."""
    if not s:
        return 0

    # Handle negative
    is_negative = False
    if s[0] == '-':
        is_negative = True
        s = s[1:]
    elif s[0] == '+':
        s = s[1:]

    # Digit mapping
    digit_map = {'0':0, '1':1, '2':2, '3':3, '4':4,
                 '5':5, '6':6, '7':7, '8':8, '9':9}

    result = 0
    for char in s:
        if char not in digit_map:
            raise ValueError(f"Invalid character: {char}")
        result = result * 10 + digit_map[char]

    return -result if is_negative else result
```

Logic: Handle sign, map each character to digit, build number digit by digit.

Inefficiencies: Dictionary lookup per character. Manual validation.

Solution 2 – Optimized Approach:

```
def str_to_int_opt(s):
    """Convert string to int using ord()."""
    if not s:
        return 0

    # Handle sign
    is_negative = False
    if s[0] == '-':
        is_negative = True
        s = s[1:]
    elif s[0] == '+':
        s = s[1:]

    result = 0
    for char in s:
        # Convert char to digit using ASCII
        if '0' <= char <= '9':
            digit = ord(char) - ord('0')
            result = result * 10 + digit
```

```
        else:
            raise ValueError(f"Invalid character: {char}")

    return -result if is_negative else result
```

Optimization: Use ord() for direct digit conversion instead of dictionary.
Justification: Faster digit conversion, less memory than dictionary.
Time Complexity:

- **Brute Force:** O(n) where n is string length
- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(1) for digit map (constant 10)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) for empty string
- **Average Case:** O(n) for both
- **Worst Case:** O(n) for both

Q64. Generate all substrings

Question: Generate all possible contiguous substrings of a string.

Solution 1 – Brute Force Approach:

```python
def all_substrings_bf(s):
    """Generate all substrings using nested loops."""
    n = len(s)
    substrings = []

    for i in range(n):
        for j in range(i, n):
            substrings.append(s[i:j+1])

    return substrings
```

Logic: For each start index i, for each end index ji, take substring s[i:j+1].

Inefficiencies: $O(n^3)$ total characters across all substrings. Creates n(n+1)/2 substrings.

Solution 2 – Optimized Approach:

```python
def all_substrings_opt(s):
    """Generate all substrings using list comprehension."""
    n = len(s)
    return [s[i:j] for i in range(n) for j in range(i+1, n+1)]

    # This is already optimal for generating all substrings
    # The O( n  ) character count is inherent to the problem
```

Optimization: Use list comprehension for cleaner code, same algorithm.

Justification: Problem inherently requires $O(n^3)$ total characters for output. List comprehension is Pythonic.

Time Complexity:

- **Brute Force:** $O(n^3)$ total characters generated
- **Optimized:** $O(n^3)$ total characters generated

Space Complexity:

- **Brute Force:** $O(n^3)$ for storing all substrings
- **Optimized:** $O(n^3)$ for storing all substrings

Best / Average / Worst Case Analysis:

- **Best Case:** $O(n^3)$ for both (inherent to problem)
- **Average Case:** $O(n^3)$ for both
- **Worst Case:** $O(n^3)$ for both

Q65. Check if string is rotation

Question: Check if one string is rotation of another (e.g., "abcd" and "cdab").

Solution 1 – Brute Force Approach:

```python
def is_rotation_bf(s1, s2):
    """Check rotation by trying all rotations."""
    if len(s1) != len(s2):
        return False

    n = len(s1)
    # Try all rotation points
    for i in range(n):
        rotated = s1[i:] + s1[:i]
        if rotated == s2:
            return True

    return False
```

Logic: Generate all possible rotations of s1, compare with s2.

Inefficiencies: $O(n^2)$ time - creates n strings of length n.

Solution 2 – Optimized Approach:

```python
def is_rotation_opt(s1, s2):
    """Check rotation using concatenation trick."""
    if len(s1) != len(s2):
        return False

    # Key insight: s2 is rotation of s1 iff s2 is substring of
       s1+s1
    concatenated = s1 + s1
    return s2 in concatenated

    # Note: in operator in Python uses efficient string search
```

Optimization: Concatenate s1 with itself, check if s2 is substring. Rotation preserves relative order.

Justification: O(n) time (substring search) vs $O(n^2)$, clever mathematical insight.

Time Complexity:

- **Brute Force:** $O(n^2)$ where n is string length
- **Optimized:** O(n) where n is string length (Python's in uses efficient algorithm)

Space Complexity:

- **Brute Force:** O(n) for rotated strings
- **Optimized:** O(n) for concatenated string

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if lengths differ
- **Average Case:** $O(n^2)$ for brute force, O(n) for optimized
- **Worst Case:** $O(n^2)$ for brute force, O(n) for optimized

Q66. Longest common prefix

Question: Find longest common prefix among array of strings.

Solution 1 – Brute Force Approach:

```python
def longest_common_prefix_bf(strs):
    """Find LCP by comparing character by character."""
    if not strs:
        return ""

    # Take first string as reference
    for i in range(len(strs[0])):
        char = strs[0][i]
        # Check this character in all other strings
        for j in range(1, len(strs)):
            if i >= len(strs[j]) or strs[j][i] != char:
                return strs[0][:i]

    return strs[0]
```

Logic: Compare characters position by position across all strings.

Inefficiencies: Compares all strings at each position. Could be optimized with early exit.

Solution 2 – Optimized Approach:

```python
def longest_common_prefix_opt(strs):
    """Find LCP using horizontal scanning or divide & conquer."""
    if not strs:
        return ""

    # Horizontal scanning
    prefix = strs[0]

    for s in strs[1:]:
        # Find common prefix between current prefix and next
            string
        while not s.startswith(prefix):
            prefix = prefix[:-1]
            if not prefix:
                return ""

    return prefix

    # Alternative: Vertical scanning (similar to brute force but
        cleaner)
    # Alternative: Divide and conquer O(m log n)
```

Optimization: Horizontal scanning: iteratively reduce prefix by comparing with each string.

Justification: More elegant, uses built-in startswith(), early exit when prefix becomes empty.

Time Complexity:

- **Brute Force:** O(S) where S is sum of all characters
- **Optimized:** O(S) where S is sum of all characters

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if first character differs
- **Average Case:** O(S) for both
- **Worst Case:** O(S) for both (when all strings equal)

Q67. Longest substring without repeating characters

Question: Find length of longest substring with all distinct characters.

Solution 1 – Brute Force Approach:

```python
def longest_unique_substring_bf(s):
    """Find longest unique substring by checking all
       substrings."""
    n = len(s)
    max_len = 0

    for i in range(n):
        for j in range(i, n):
            substring = s[i:j+1]
            # Check if all characters are unique
            if len(set(substring)) == len(substring):
                max_len = max(max_len, len(substring))

    return max_len
```

Logic: Generate all substrings, check if all characters unique using set.

Inefficiencies: $O(n^3)$ time - n^2 substrings, each set creation is $O(k)$.

Solution 2 – Optimized Approach:

```python
def longest_unique_substring_opt(s):
    """Find longest unique substring using sliding window."""
    char_index = {}
    max_len = 0
    left = 0

    for right in range(len(s)):
        # If character seen and within current window
        if s[right] in char_index and char_index[s[right]] >=
           left:
            # Move left pointer past previous occurrence
            left = char_index[s[right]] + 1

        # Update character's latest index
        char_index[s[right]] = right
        # Update max length
        max_len = max(max_len, right - left + 1)

    return max_len
```

Optimization: Sliding window with hash map tracking last seen indices. Expand right, contract left when duplicate found.

Justification: $O(n)$ time vs $O(n^3)$, single pass with $O(1)$ amortized operations.

Time Complexity:

- **Brute Force:** $O(n^3)$ where n is string length
- **Optimized:** $O(n)$ where n is string length

Space Complexity:

- **Brute Force:** O(n) for sets
- **Optimized:** O(min(n, 256)) for character map

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** $O(n^3)$ for brute force, O(n) for optimized
- **Worst Case:** $O(n^3)$ for brute force, O(n) for optimized

Q68. Longest palindromic substring

Question: Find longest palindrome substring in a string.

Solution 1 – Brute Force Approach:

```python
def longest_palindromic_bf(s):
    """Find longest palindrome by checking all substrings."""
    n = len(s)
    if n == 0:
        return ""

    longest = ""

    for i in range(n):
        for j in range(i, n):
            substring = s[i:j+1]
            if substring == substring[::-1]:
                if len(substring) > len(longest):
                    longest = substring

    return longest
```

Logic: Generate all substrings, check if palindrome by comparing with reverse.

Inefficiencies: $O(n^3)$ time - n^2 substrings, each reverse is O(k).

Solution 2 – Optimized Approach:

```python
def longest_palindromic_opt(s):
    """Find longest palindrome using expansion around center."""
    if not s:
        return ""

    start, end = 0, 0

    def expand_around_center(left, right):
        """Expand from center and return palindrome bounds."""
        while left >= 0 and right < len(s) and s[left] ==
           s[right]:
            left -= 1
            right += 1
        return left + 1, right - 1

    for i in range(len(s)):
        # Odd length palindromes (single center)
        l1, r1 = expand_around_center(i, i)
        # Even length palindromes (double center)
        l2, r2 = expand_around_center(i, i + 1)

        # Update longest
        if r1 - l1 > end - start:
            start, end = l1, r1
        if r2 - l2 > end - start:
            start, end = l2, r2

```

```
27      return s[start:end+1]
```

Optimization: Expand around center for each position. Check both odd and even length palindromes.

Justification: $O(n^2)$ time vs $O(n^3)$, dynamic programming can achieve $O(n^2)$ with $O(n^2)$ space.

Time Complexity:

- **Brute Force:** $O(n^3)$ where n is string length
- **Optimized:** $O(n^2)$ where n is string length

Space Complexity:

- **Brute Force:** $O(n)$ for substrings
- **Optimized:** $O(1)$

Best / Average / Worst Case Analysis:

- **Best Case:** $O(n^2)$ for optimized
- **Average Case:** $O(n^3)$ for brute force, $O(n^2)$ for optimized
- **Worst Case:** $O(n^3)$ for brute force, $O(n^2)$ for optimized

Q69. Remove all spaces from string

Question: Remove all whitespace characters from a string.

Solution 1 – Brute Force Approach:

```
def remove_spaces_bf(s):
    """Remove spaces by building new string."""
    result = ""
    for char in s:
        if char != ' ':
            result += char
    return result
```

Logic: Iterate through string, append non-space characters to result.

Inefficiencies: O(n^2) time due to string concatenation creating new strings.

Solution 2 – Optimized Approach:

```
def remove_spaces_opt(s):
    """Remove spaces using replace or list join."""
    # Method 1: replace (removes only spaces)
    return s.replace(' ', '')

    # Method 2: join with filter (removes all whitespace)
    # return ''.join(s.split())

    # Method 3: list comprehension
    # return ''.join([c for c in s if c != ' '])
```

Optimization: Use str.replace() which is implemented in C and highly optimized.

Justification: O(n) time vs O(n^2), single operation instead of character-by-character.

Time Complexity:

- **Brute Force:** O(n^2) where n is string length
- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(n) for new string
- **Optimized:** O(n) for new string

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** O(n^2) for brute force, O(n) for optimized
- **Worst Case:** O(n^2) for brute force, O(n) for optimized

Q70. Replace specific characters

Question: Replace all occurrences of specific characters in string.

Solution 1 – Brute Force Approach:

```
def replace_chars_bf(s, old_chars, new_char):
    """Replace characters by building new string."""
    result = ""
    for char in s:
        if char in old_chars:
            result += new_char
        else:
            result += char
    return result
```

Logic: Check each character against old$_c hars$, $appendreplacementororiginal$.

Inefficiencies: O(n^2) time due to string concatenation. 'in' check on string is O(k).

Solution 2 – Optimized Approach:

```
def replace_chars_opt(s, old_chars, new_char):
    """Replace characters using translate() or comprehension."""
    # Method 1: str.maketrans and translate (fastest)
    trans_table = str.maketrans(''.join(old_chars), new_char *
       len(old_chars))
    return s.translate(trans_table)

    # Method 2: List comprehension
    # return ''.join([new_char if c in old_chars else c for c in
       s])
```

Optimization: Use str.maketrans() and translate() which are C-optimized for character translation.

Justification: O(n) time vs O(n^2), translation table approach is highly optimized.

Time Complexity:

- **Brute Force:** O(n*k) where n is string length, k is old$_c harslength$
- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(n) for new string
- **Optimized:** O(1) for translation table + O(n) for new string

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** O(n*k) for brute force, O(n) for optimized
- **Worst Case:** O(n*k) for brute force, O(n) for optimized

Q71. Check valid parentheses

Question: Check if string of parentheses '()[]' is properly balanced.

Solution 1 – Brute Force Approach:

```
def valid_parentheses_bf(s):
    """Check parentheses by repeatedly removing pairs."""
    while '()' in s or '[]' in s or '{}' in s:
        s = s.replace('()', '').replace('[]', '').replace('{}',
            '')
    return len(s) == 0
```

Logic: Repeatedly remove valid pairs until string empty or unchanged.

Inefficiencies: O(n^2) time - each replace scans entire string, done multiple times.

Solution 2 – Optimized Approach:

```
def valid_parentheses_opt(s):
    """Check parentheses using stack."""
    stack = []
    mapping = {')': '(', ']': '[', '}': '{'}

    for char in s:
        if char in mapping.values():  # Opening bracket
            stack.append(char)
        elif char in mapping:  # Closing bracket
            if not stack or stack[-1] != mapping[char]:
                return False
            stack.pop()
        # Ignore other characters

    return len(stack) == 0
```

Optimization: Use stack to track opening brackets. When closing bracket found, check if matches top.

Justification: O(n) time vs O(n^2), single pass with stack.

Time Complexity:

- **Brute Force:** O(n^2) where n is string length
- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(n) for modified strings
- **Optimized:** O(n) for stack in worst case

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if first character is invalid closing
- **Average Case:** O(n^2) for brute force, O(n) for optimized
- **Worst Case:** O(n^2) for brute force, O(n) for optimized

Q72. Count occurrences of substring

Question: Count how many times substring appears in string (non-overlapping).

Solution 1 – Brute Force Approach:

```
def count_substring_bf(s, sub):
    """Count substring occurrences using manual search."""
    count = 0
    n, m = len(s), len(sub)

    for i in range(n - m + 1):
        match = True
        for j in range(m):
            if s[i + j] != sub[j]:
                match = False
                break
        if match:
            count += 1

    return count
```

Logic: Slide substring over string, compare character by character.

Inefficiencies: O(n*m) time in worst case where n,m are string lengths.

Solution 2 – Optimized Approach:

```
def count_substring_opt(s, sub):
    """Count substring using built-in count() or KMP."""
    # Method 1: Built-in count (non-overlapping)
    return s.count(sub)

    # Method 2: KMP algorithm for overlapping count
    # def kmp_count(s, sub):
    #     # KMP implementation for overlapping matches
    #     ...
```

Optimization: Use str.count() which is C-optimized. For overlapping matches, use KMP algorithm.

Justification: Built-in count() uses optimized algorithm. KMP provides O(n+m) for overlapping.

Time Complexity:

- **Brute Force:** O(n*m) where n,m are string and substring lengths
- **Optimized:** O(n+m) for KMP, O(n) for count() (implementation dependent)

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(m) for KMP prefix table

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for both

- **Average Case:** O(n*m) for brute force, O(n+m) for KMP
- **Worst Case:** O(n*m) for brute force (e.g., "aaaa", "aa"), O(n+m) for KMP

Q73. Capitalize first letter of each word

Question: Capitalize first letter of each word in sentence.

Solution 1 – Brute Force Approach:

```
def capitalize_words_bf(s):
    """Capitalize words by manual word detection."""
    result = ""
    capitalize_next = True

    for char in s:
        if char == ' ':
            capitalize_next = True
            result += char
        else:
            if capitalize_next:
                result += char.upper()
                capitalize_next = False
            else:
                result += char.lower()

    return result
```

Logic: Track when to capitalize, capitalize first character after space.

Inefficiencies: O(n^2) due to string concatenation. Manual case handling.

Solution 2 – Optimized Approach:

```
def capitalize_words_opt(s):
    """Capitalize words using title() or split/join."""
    # Method 1: title() (but be careful with apostrophes)
    # return s.title()  # "don't" becomes "Don'T"

    # Method 2: split and capitalize each word
    words = s.split()
    capitalized = [word[0].upper() + word[1:].lower() for word
       in words]
    return ' '.join(capitalized)
```

Optimization: Use split/join with list comprehension. Handle each word separately.

Justification: Cleaner, handles edge cases better than title(). O(n) time.

Time Complexity:

- **Brute Force:** O(n^2) where n is string length
- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(n) for new string
- **Optimized:** O(n) for words list and result

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized

- **Average Case:** $O(n^2)$ for brute force, $O(n)$ for optimized
- **Worst Case:** $O(n^2)$ for brute force, $O(n)$ for optimized

Q74. Strings differ by exactly one character

Question: Check if two strings of same length differ by exactly one character.

Solution 1 – Brute Force Approach:

```python
def differ_by_one_bf(s1, s2):
    """Check difference by comparing all characters."""
    if len(s1) != len(s2):
        return False

    diff_count = 0
    for i in range(len(s1)):
        if s1[i] != s2[i]:
            diff_count += 1
            if diff_count > 1:
                return False

    return diff_count == 1
```

Logic: Compare character by character, count differences, return True if exactly one.

Inefficiencies: Always compares all characters even if multiple differences early.

Solution 2 – Optimized Approach:

```python
def differ_by_one_opt(s1, s2):
    """Check difference with early exit."""
    if len(s1) != len(s2):
        return False

    diff_found = False
    for i in range(len(s1)):
        if s1[i] != s2[i]:
            if diff_found:  # Second difference found
                return False
            diff_found = True

    return diff_found  # Must have exactly one difference
```

Optimization: Early exit when second difference found. Use boolean flag instead of counter.

Justification: Early exit improves best case. Same worst case but cleaner.

Time Complexity:

- **Brute Force:** O(n) where n is string length
- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if first character differs in both strings

- **Average Case:** O(n) for both
- **Worst Case:** O(n) for both (when identical or last character differs)

Q75. Run-length encoding compression

Question: Compress string using run-length encoding (e.g., "aaabbc" → "a3b2c1").

Solution 1 – Brute Force Approach:

```python
def rle_compress_bf(s):
    """Compress using run-length encoding."""
    if not s:
        return ""

    compressed = ""
    count = 1

    for i in range(1, len(s)):
        if s[i] == s[i-1]:
            count += 1
        else:
            compressed += s[i-1] + str(count)
            count = 1

    # Add last character run
    compressed += s[-1] + str(count)

    return compressed
```

Logic: Track current character run, append character and count when run ends.

Inefficiencies: $O(n^2)$ due to string concatenation. Creates string for each run.

Solution 2 – Optimized Approach:

```python
def rle_compress_opt(s):
    """Compress using list join."""
    if not s:
        return ""

    compressed_chars = []
    count = 1

    for i in range(1, len(s)):
        if s[i] == s[i-1]:
            count += 1
        else:
            compressed_chars.append(s[i-1] + str(count))
            count = 1

    # Add last character run
    compressed_chars.append(s[-1] + str(count))

    return ''.join(compressed_chars)
```

Optimization: Use list to accumulate compressed parts, join once at end.

Justification: $O(n)$ time vs $O(n^2)$, avoids repeated string concatenation.

Time Complexity:

- **Brute Force:** $O(n^2)$ where n is string length

- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(n) for compressed string
- **Optimized:** O(n) for list and result

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** $O(n^2)$ for brute force, O(n) for optimized
- **Worst Case:** $O(n^2)$ for brute force, O(n) for optimized

Q76. Run-length encoding decompression

Question: Decompress run-length encoded string (e.g., "a3b2c1" → "aaabbc").

Solution 1 – Brute Force Approach:

```
def rle_decompress_bf(compressed):
    """Decompress run-length encoded string."""
    if not compressed:
        return ""

    result = ""
    i = 0
    n = len(compressed)

    while i < n:
        char = compressed[i]
        i += 1

        # Extract number (may have multiple digits)
        count_str = ""
        while i < n and compressed[i].isdigit():
            count_str += compressed[i]
            i += 1

        count = int(count_str) if count_str else 1
        result += char * count

    return result
```

Logic: Parse character followed by digits, repeat character count times.

Inefficiencies: $O(n^2)$ due to string concatenation. Digit parsing could be optimized.

Solution 2 – Optimized Approach:

```
def rle_decompress_opt(compressed):
    """Decompress using list and join."""
    if not compressed:
        return ""

    result_chars = []
    i = 0
    n = len(compressed)

    while i < n:
        char = compressed[i]
        i += 1

        # Extract number using while loop
        count = 0
        while i < n and compressed[i].isdigit():
            count = count * 10 + int(compressed[i])
            i += 1

        # Default count to 1 if no digits
```

```
        count = max(count, 1)
        result_chars.append(char * count)

    return ''.join(result_chars)
```

Optimization: Use list accumulation and join. Parse digits more efficiently.
Justification: O(n) time vs $O(n^2)$, efficient digit parsing and list join.
Time Complexity:

- **Brute Force:** $O(n^2)$ where n is compressed length
- **Optimized:** O(m) where m is decompressed length

Space Complexity:

- **Brute Force:** O(m) for result
- **Optimized:** O(m) for list and result

Best / Average / Worst Case Analysis:

- **Best Case:** O(m) for optimized
- **Average Case:** $O(n^2)$ for brute force, O(m) for optimized
- **Worst Case:** $O(n^2)$ for brute force, O(m) for optimized

Q77. Lexicographically smallest rotation

Question: Find lexicographically smallest rotation of string (e.g., "cba" → "acb").

Solution 1 – Brute Force Approach:

```python
def smallest_rotation_bf(s):
    """Find smallest rotation by trying all."""
    if not s:
        return ""

    n = len(s)
    smallest = s

    for i in range(n):
        rotated = s[i:] + s[:i]
        if rotated < smallest:
            smallest = rotated

    return smallest
```

Logic: Generate all rotations, compare lexicographically, track smallest.

Inefficiencies: $O(n^2)$ time - creates n strings of length n, string comparison is O(n).

Solution 2 – Optimized Approach:

```python
def smallest_rotation_opt(s):
    """Find smallest rotation using Booth's algorithm."""
    if not s:
        return ""

    n = len(s)
    s = s + s  # Duplicate string for easier rotation comparison
    failure = [0] * (2 * n)

    k = 0
    for j in range(1, 2 * n):
        i = failure[j - k - 1]
        while i != 0 and s[j] != s[k + i]:
            if s[j] < s[k + i]:
                k = j - i
            i = failure[i - 1]

        if s[j] != s[k + i]:
            if s[j] < s[k]:
                k = j
            failure[j - k] = 0
        else:
            failure[j - k] = i + 1

    return s[k:k + n]
```

Optimization: Booth's algorithm O(n) using modified KMP failure function.

Justification: O(n) time vs $O(n^2)$, specialized algorithm for this problem.

Time Complexity:

- **Brute Force:** $O(n^2)$ where n is string length
- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(n) for rotated strings
- **Optimized:** O(n) for failure array and duplicated string

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** $O(n^2)$ for brute force, O(n) for optimized
- **Worst Case:** $O(n^2)$ for brute force, O(n) for optimized

Q78. Check balanced brackets (multiple types)

Question: Check if string with brackets ()[]¡¿ is properly balanced.

Solution 1 – Brute Force Approach:

```python
def balanced_brackets_bf(s):
    """Check balanced brackets by removing pairs."""
    brackets = ['()', '[]', '{}', '<>']

    changed = True
    while changed:
        changed = False
        for pair in brackets:
            if pair in s:
                s = s.replace(pair, '')
                changed = True

    return len(s) == 0
```

Logic: Repeatedly remove valid bracket pairs until string empty or unchanged.

Inefficiencies: O(n^2) time - each iteration scans entire string multiple times.

Solution 2 – Optimized Approach:

```python
def balanced_brackets_opt(s):
    """Check balanced brackets using stack."""
    stack = []
    mapping = {')': '(', ']': '[', '}': '{', '>': '<'}
    opening = set(mapping.values())

    for char in s:
        if char in opening:
            stack.append(char)
        elif char in mapping:
            if not stack or stack[-1] != mapping[char]:
                return False
            stack.pop()
        # Ignore other characters

    return len(stack) == 0
```

Optimization: Stack-based approach similar to Q71 but extended for more bracket types.

Justification: O(n) time vs O(n^2), single pass with stack.

Time Complexity:

- **Brute Force:** O(n^2) where n is string length
- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(n) for modified strings
- **Optimized:** O(n) for stack in worst case

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if first character is invalid closing
- **Average Case:** $O(n^2)$ for brute force, O(n) for optimized
- **Worst Case:** $O(n^2)$ for brute force, O(n) for optimized

Q79. Minimum window substring

Question: Find minimum substring containing all characters of pattern.

Solution 1 – Brute Force Approach:

```python
def min_window_bf(s, t):
    """Find minimum window by checking all substrings."""
    if not s or not t or len(s) < len(t):
        return ""

    min_window = ""
    min_len = float('inf')

    # Check all substrings
    for i in range(len(s)):
        for j in range(i + len(t) - 1, len(s)):
            substring = s[i:j+1]

            # Check if substring contains all characters of t
            contains_all = True
            for char in t:
                if substring.count(char) < t.count(char):
                    contains_all = False
                    break

            if contains_all and len(substring) < min_len:
                min_window = substring
                min_len = len(substring)

    return min_window
```

Logic: Generate all substrings of sufficient length, check if contains all pattern chars.

Inefficiencies: O(n³*m) where n is s length, m is t length. Extremely inefficient.

Solution 2 – Optimized Approach:

```python
def min_window_opt(s, t):
    """Find minimum window using sliding window."""
    if not s or not t or len(s) < len(t):
        return ""

    from collections import Counter

    t_count = Counter(t)
    required = len(t_count)
    formed = 0

    window_counts = {}
    left = right = 0
    min_len = float('inf')
    min_window = ""

    while right < len(s):
        char = s[right]
```

```
        window_counts[char] = window_counts.get(char, 0) + 1

        if char in t_count and window_counts[char] ==
           t_count[char]:
            formed += 1

        # Try to contract window while it's valid
        while left <= right and formed == required:
            char = s[left]

            # Update minimum window
            if right - left + 1 < min_len:
                min_len = right - left + 1
                min_window = s[left:right+1]

            # Remove left character from window
            window_counts[char] -= 1
            if char in t_count and window_counts[char] <
               t_count[char]:
                formed -= 1

            left += 1

        right += 1

    return min_window
```

Optimization: Sliding window with two pointers and hash maps. Expand right until window valid, then contract left.

Justification: O(n) time vs O(n^3*m), optimal solution for this problem.

Time Complexity:

- **Brute Force:** O(n^3*m) where n,m are string lengths
- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(n) for substrings
- **Optimized:** O(k) where k is unique characters in pattern

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** O(n^3*m) for brute force, O(n) for optimized
- **Worst Case:** O(n^3*m) for brute force, O(n) for optimized

Q80. Remove adjacent duplicates

Question: Remove adjacent duplicate characters from string (e.g., "aabbcc" → "abc").

Solution 1 – Brute Force Approach:

```
def remove_adjacent_duplicates_bf(s):
    """Remove adjacent duplicates by checking neighbors."""
    result = ""
    i = 0
    n = len(s)

    while i < n:
        result += s[i]
        # Skip duplicates
        j = i + 1
        while j < n and s[j] == s[i]:
            j += 1
        i = j

    return result
```

Logic: For each character, add to result, skip subsequent duplicates.

Inefficiencies: $O(n^2)$ due to string concatenation. Nested while loops.

Solution 2 – Optimized Approach:

```
def remove_adjacent_duplicates_opt(s):
    """Remove adjacent duplicates using stack/list."""
    if not s:
        return ""

    result = []

    for char in s:
        if result and result[-1] == char:
            continue  # Skip duplicate
        result.append(char)

    return ''.join(result)

    # Alternative: Two-pointer in-place if string was mutable
    # chars = list(s)
    # write_idx = 0
    # for read_idx in range(len(chars)):
    #     if write_idx == 0 or chars[read_idx] !=
       chars[write_idx-1]:
    #         chars[write_idx] = chars[read_idx]
    #         write_idx += 1
    # return ''.join(chars[:write_idx])
```

Optimization: Use list as stack, skip when last element equals current character.

Justification: O(n) time vs $O(n^2)$, single pass with O(1) amortized operations.

Time Complexity:

- **Brute Force:** $O(n^2)$ where n is string length

- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(n) for result
- **Optimized:** O(n) for result list

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** O(n^2) for brute force, O(n) for optimized
- **Worst Case:** O(n^2) for brute force, O(n) for optimized

SECTION D: RECURSION & BACKTRACKING (20 Questions)

Q81. Recursive factorial

Question: Compute factorial of n using recursion: n! = n × (n-1)!

Solution 1 – Brute Force Recursive Approach:

```
def factorial_recursive_bf(n):
    """Recursive factorial without memoization."""
    if n < 0:
        return "Undefined"
    if n == 0 or n == 1:
        return 1
    return n * factorial_recursive_bf(n - 1)
```

Logic: Base cases: factorial(0)=1, factorial(1)=1. Recursive case: n! = n × (n-1)!.

Inefficiencies: No memoization leads to repeated calculations. Stack depth O(n) could cause recursion limit error for large n.

Solution 2 – Optimized Approach:

```
def factorial_recursive_opt(n, memo={}):
    """Recursive factorial with memoization."""
    if n < 0:
        return "Undefined"
    if n in memo:
        return memo[n]
    if n == 0 or n == 1:
        result = 1
    else:
        result = n * factorial_recursive_opt(n - 1, memo)
    memo[n] = result
    return result

# Alternative: iterative approach
def factorial_iterative(n):
```

```
    """Iterative factorial - better for large n."""
    if n < 0:
        return "Undefined"
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result
```

Optimization: Add memoization to avoid repeated calculations. Or use iterative approach to avoid recursion limits.

Justification: Memoization reduces repeated work. Iterative approach uses O(1) stack space vs O(n).

Time Complexity:

- **Brute Force:** O(n) where n is input
- **Optimized:** O(n) where n is input

Space Complexity:

- **Brute Force:** O(n) recursion stack
- **Optimized:** O(n) for memo dict or O(1) for iterative

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) when n1
- **Average Case:** O(n) for both
- **Worst Case:** O(n) for both

Q82. Recursive Fibonacci sequence

Question: Generate Fibonacci sequence recursively: F(n) = F(n-1) + F(n-2), with F(0)=0, F(1)=1.

Solution 1 – Brute Force Recursive Approach:

```
def fibonacci_recursive_bf(n):
    """Recursive Fibonacci without memoization."""
    if n <= 0:
        return 0
    if n == 1:
        return 1
    return fibonacci_recursive_bf(n-1) +
       fibonacci_recursive_bf(n-2)
```

Logic: Base cases: F(0)=0, F(1)=1. Recursive case: F(n) = F(n-1) + F(n-2).

Inefficiencies: Exponential time $O(2^n) due to repeated calculations of same subproblems.$

Solution 2 – Optimized Approach:

```
def fibonacci_recursive_opt(n, memo={}):
    """Recursive Fibonacci with memoization (top-down DP)."""
    if n in memo:
        return memo[n]
    if n <= 0:
        result = 0
    elif n == 1:
        result = 1
    else:
        result = (fibonacci_recursive_opt(n-1, memo) +
                  fibonacci_recursive_opt(n-2, memo))
    memo[n] = result
    return result

# Alternative: bottom-up DP
def fibonacci_iterative(n):
    """Iterative Fibonacci (bottom-up DP)."""
    if n <= 0:
        return 0
    if n == 1:
        return 1

    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b
```

Optimization: Add memoization (top-down DP) or use iterative approach (bottom-up DP).

Justification: Memoization reduces from $O(2^n) to O(n). Iterative uses O(1) space vs O(n) recursion st$

Time Complexity:

- **Brute Force:** $O(2^n) exponential$
- **Optimized:** O(n) linear with memoization

Space Complexity:

- **Brute Force:** O(n) recursion stack
- **Optimized:** O(n) for memo dict or O(1) for iterative

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) when n1
- **Average Case:** $O(2^n) for brute force, O(n) for optimized$
- **Worst Case:** $O(2^n) for brute force, O(n) for optimized$

Q83. Recursive sum of digits

Question: Compute sum of digits of a number using recursion.

Solution 1 – Brute Force Recursive Approach:

```python
def sum_digits_recursive_bf(n):
    """Recursive sum of digits without helper."""
    if n < 0:
        return sum_digits_recursive_bf(-n)
    if n < 10:
        return n
    return (n % 10) + sum_digits_recursive_bf(n // 10)
```

Logic: Base case: single digit returns itself. Recursive case: last digit + sum of remaining digits.

Inefficiencies: No significant inefficiencies for this simple recursion.

Solution 2 – Optimized Approach:

```python
def sum_digits_recursive_opt(n):
    """Recursive sum of digits with tail recursion
       optimization."""
    def helper(num, acc):
        if num == 0:
            return acc
        return helper(num // 10, acc + (num % 10))

    n = abs(n)
    return helper(n, 0)

# Python doesn't optimize tail recursion, but this is the pattern
```

Optimization: Use tail recursion pattern (though Python doesn't optimize it). Makes intention clearer.

Justification: Tail recursion can be optimized by compilers (in other languages). Cleaner separation of concerns.

Time Complexity:

- **Brute Force:** O(d) where d is number of digits
- **Optimized:** O(d) where d is number of digits

Space Complexity:

- **Brute Force:** O(d) recursion stack
- **Optimized:** O(d) recursion stack (Python doesn't optimize tail calls)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) for single-digit numbers
- **Average Case:** O(d) where d is digit count
- **Worst Case:** O(d) where d is digit count

Q84. Recursive string reversal

Question: Reverse a string using recursion.

Solution 1 – Brute Force Recursive Approach:

```python
def reverse_string_recursive_bf(s):
    """Recursive string reversal using slicing."""
    if len(s) <= 1:
        return s
    return reverse_string_recursive_bf(s[1:]) + s[0]
```

Logic: Base case: empty or single character string returns itself. Recursive case: reverse of substring + first character.

Inefficiencies: Creates new strings with each recursion $O(n^2)$ total string operations.

Solution 2 – Optimized Approach:

```python
def reverse_string_recursive_opt(s):
    """Recursive reversal using indices."""
    def helper(left, right):
        if left >= right:
            return
        # Swap characters
        chars[left], chars[right] = chars[right], chars[left]
        helper(left + 1, right - 1)

    chars = list(s)
    helper(0, len(chars) - 1)
    return ''.join(chars)

# Alternative: iterative is simpler for this problem
def reverse_string_iterative(s):
    return s[::-1]
```

Optimization: Use indices to swap in-place, avoid string slicing. Convert to list, swap, convert back.

Justification: O(n) time vs $O(n^2)$ from string concatenation. More efficient memory usage.

Time Complexity:

- **Brute Force:** $O(n^2)$ due to string concatenation
- **Optimized:** O(n) linear time

Space Complexity:

- **Brute Force:** $O(n^2)$ total string memory
- **Optimized:** O(n) for character list + O(n) recursion stack

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) for empty or single-character string
- **Average Case:** $O(n^2)$ for brute force, O(n) for optimized
- **Worst Case:** $O(n^2)$ for brute force, O(n) for optimized

Q85. Recursive palindrome check

Question: Check if string is palindrome using recursion.

Solution 1 – Brute Force Recursive Approach:

```python
def is_palindrome_recursive_bf(s):
    """Recursive palindrome check with slicing."""
    s = ''.join(c.lower() for c in s if c.isalnum())
    if len(s) <= 1:
        return True
    if s[0] != s[-1]:
        return False
    return is_palindrome_recursive_bf(s[1:-1])
```

Logic: Base case: empty or single character is palindrome. Recursive case: check first and last characters, recurse on middle.

Inefficiencies: Creates new strings with slicing $O(n^2)$ operations. Cleans string each call.

Solution 2 – Optimized Approach:

```python
def is_palindrome_recursive_opt(s):
    """Recursive palindrome check with indices."""
    def helper(left, right):
        # Skip non-alphanumeric
        while left < right and not s[left].isalnum():
            left += 1
        while left < right and not s[right].isalnum():
            right -= 1

        if left >= right:
            return True
        if s[left].lower() != s[right].lower():
            return False
        return helper(left + 1, right - 1)

    return helper(0, len(s) - 1)
```

Optimization: Use indices to avoid string slicing. Skip non-alphanumeric in helper function.

Justification: $O(n)$ time vs $O(n^2)$, works in-place without creating new strings.

Time Complexity:

- **Brute Force:** $O(n^2)$ due to string slicing
- **Optimized:** $O(n)$ linear time

Space Complexity:

- **Brute Force:** $O(n^2)$ total string memory
- **Optimized:** $O(n)$ recursion stack

Best / Average / Worst Case Analysis:

- **Best Case:** $O(1)$ if first and last characters differ

- **Average Case:** $O(n^2)$ for brute force, O(n) for optimized
- **Worst Case:** $O(n^2)$ for brute force, O(n) for optimized (when palindrome)

Q86. Recursive power computation

Question: Compute x raised to power n using recursion.

Solution 1 – Brute Force Recursive Approach:

```python
def power_recursive_bf(x, n):
    """Recursive power using simple recursion."""
    if n == 0:
        return 1
    if n < 0:
        return 1 / power_recursive_bf(x, -n)
    return x * power_recursive_bf(x, n - 1)
```

Logic: Base case: x = 1. Recursive case: $x = x \times x^{1}$. Handle negative exponents.

Inefficiencies: O(n) recursive calls for positive n. Repeated calculations.

Solution 2 – Optimized Approach:

```python
def power_recursive_opt(x, n):
    """Recursive power using exponentiation by squaring."""
    if n == 0:
        return 1
    if n < 0:
        return 1 / power_recursive_opt(x, -n)

    # Exponentiation by squaring
    half = power_recursive_opt(x, n // 2)
    if n % 2 == 0:
        return half * half
    else:
        return x * half * half
```

Optimization: Exponentiation by squaring: $x = (x^{2})^{n/2} if even, x(x)^{(n-1)/2} if odd$.

Justification: O(log n) time vs O(n), reduces recursive calls significantly.

Time Complexity:

- **Brute Force:** O(n) where n is exponent
- **Optimized:** O(log n) where n is exponent

Space Complexity:

- **Brute Force:** O(n) recursion stack
- **Optimized:** O(log n) recursion stack

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) when n=0
- **Average Case:** O(n) for brute force, O(log n) for optimized
- **Worst Case:** O(n) for brute force, O(log n) for optimized

Q87. Generate all subsets of a list

Question: Generate all possible subsets (power set) of a list.

Solution 1 – Brute Force Approach:

```python
def subsets_bf(nums):
    """Generate subsets using recursion exploring all
       combinations."""
    result = []

    def backtrack(start, current):
        # Add current subset
        result.append(current.copy())

        # Try adding each remaining element
        for i in range(start, len(nums)):
            current.append(nums[i])
            backtrack(i + 1, current)
            current.pop()  # backtrack

    backtrack(0, [])
    return result
```

Logic: For each element, either include or exclude it. Backtrack to explore all combinations.

Inefficiencies: Generates 2 subsets which is optimal. No significant inefficiency.

Solution 2 – Optimized Approach:

```python
def subsets_opt(nums):
    """Generate subsets using iterative approach or bitmask."""
    # Method 1: Iterative approach
    result = [[]]
    for num in nums:
        result += [curr + [num] for curr in result]
    return result

    # Method 2: Bitmask (for small n <= 64)
    # n = len(nums)
    # result = []
    # for mask in range(1 << n):
    #     subset = []
    #     for i in range(n):
    #         if mask & (1 << i):
    #             subset.append(nums[i])
    #     result.append(subset)
    # return result
```

Optimization: Iterative approach builds subsets incrementally. Bitmask approach for small n.

Justification: Iterative approach avoids recursion overhead. Bitmask is elegant for small sets.

Time Complexity:

- **Brute Force:** O(n × 2) where n is list length
- **Optimized:** O(n × 2) where n is list length (inherent to problem)

Space Complexity:

- **Brute Force:** O(n) recursion stack + O(n × 2) output
- **Optimized:** O(n × 2) output

Best / Average / Worst Case Analysis:

- **Best Case:** O(2) for both (inherent to problem)
- **Average Case:** O(n × 2) for both
- **Worst Case:** O(n × 2) for both

Q88. Generate all permutations of a string

Question: Generate all permutations of a string (all arrangements of characters).

Solution 1 – Brute Force Approach:

```
def permutations_bf(s):
    """Generate permutations using recursion."""
    result = []

    def backtrack(chars, current):
        if not chars:
            result.append(''.join(current))
            return

        for i in range(len(chars)):
            # Choose character
            current.append(chars[i])
            # Recurse with remaining characters
            backtrack(chars[:i] + chars[i+1:], current)
            # Backtrack
            current.pop()

    backtrack(list(s), [])
    return result
```

Logic: For each position, try each remaining character. Recurse with reduced character set.

Inefficiencies: Creates new string slices O(n^2) operations per recursive call.

Solution 2 – Optimized Approach:

```
def permutations_opt(s):
    """Generate permutations using swapping (in-place)."""
    result = []
    chars = list(s)

    def backtrack(start):
        if start == len(chars):
            result.append(''.join(chars))
            return

        for i in range(start, len(chars)):
            # Swap to put chars[i] at position start
            chars[start], chars[i] = chars[i], chars[start]
            # Recurse on remaining positions
            backtrack(start + 1)
            # Backtrack: restore original order
            chars[start], chars[i] = chars[i], chars[start]

    backtrack(0)
    return result
```

Optimization: In-place swapping avoids creating new lists. Swap characters to generate permutations.

Justification: $O(n! \times n)$ time vs $O(n! \times n^2)$. Reduces string copying overhead.
Time Complexity:

- **Brute Force:** $O(n! \times n^2)$ where n is string length
- **Optimized:** $O(n! \times n)$ where n is string length

Space Complexity:

- **Brute Force:** $O(n)$ recursion stack + $O(n! \times n)$ output
- **Optimized:** $O(n)$ recursion stack + $O(n! \times n)$ output

Best / Average / Worst Case Analysis:

- **Best Case:** $O(n! \times n)$ for optimized
- **Average Case:** $O(n! \times n^2)$ for brute force, $O(n! \times n)$ for optimized
- **Worst Case:** $O(n! \times n^2)$ for brute force, $O(n! \times n)$ for optimized

Q89. Generate combinations of size K from N numbers

Question: Generate all combinations of size K from numbers 1 to N.

Solution 1 – Brute Force Approach:

```python
def combinations_bf(n, k):
    """Generate combinations using recursion exploring all."""
    result = []

    def backtrack(start, current):
        if len(current) == k:
            result.append(current.copy())
            return

        for i in range(start, n + 1):
            current.append(i)
            backtrack(i + 1, current)
            current.pop()

    backtrack(1, [])
    return result
```

Logic: Recursively build combinations of size K. At each step, choose next number from remaining.

Inefficiencies: Explores all possibilities but pruning occurs naturally. No significant inefficiency.

Solution 2 – Optimized Approach:

```python
def combinations_opt(n, k):
    """Generate combinations using iterative approach."""
    from itertools import combinations
    # Using itertools (C implementation)
    return list(combinations(range(1, n + 1), k))

    # Manual implementation with same algorithm
    # result = []
    # combo = list(range(1, k + 1))
    # result.append(combo.copy())
    # while True:
    #     # Find next combination
    #     i = k - 1
    #     while i >= 0 and combo[i] == n - k + i + 1:
    #         i -= 1
    #     if i < 0:
    #         break
    #     combo[i] += 1
    #     for j in range(i + 1, k):
    #         combo[j] = combo[j - 1] + 1
    #     result.append(combo.copy())
    # return result
```

Optimization: Use itertools.combinations (C-optimized) or iterative next-combination algorithm.

Justification: Built-in itertools is fastest. Iterative approach avoids recursion overhead.

Time Complexity:

- **Brute Force:** O(C(n,k) × k) where C(n,k) = n!/(k!(n-k)!)
- **Optimized:** O(C(n,k) × k) where C(n,k) = n!/(k!(n-k)!)

Space Complexity:

- **Brute Force:** O(k) recursion stack + O(C(n,k) × k) output
- **Optimized:** O(C(n,k) × k) output

Best / Average / Worst Case Analysis:

- **Best Case:** O(C(n,k) × k) for both
- **Average Case:** O(C(n,k) × k) for both
- **Worst Case:** O(C(n,k) × k) for both

Q90. Ways to climb N stairs

Question: Count ways to climb N stairs taking 1 or 2 steps at a time.

Solution 1 – Brute Force Recursive Approach:

```python
def climb_stairs_bf(n):
    """Recursive solution without memoization."""
    if n <= 1:
        return 1
    return climb_stairs_bf(n-1) + climb_stairs_bf(n-2)
```

Logic: Base cases: 0 or 1 stairs = 1 way. Recurrence: ways(n) = ways(n-1) + ways(n-2) (Fibonacci).

Inefficiencies: Exponential O(2) due to repeated calculations of same subproblems.

Solution 2 – Optimized Approach:

```python
def climb_stairs_opt(n):
    """Dynamic programming solution."""
    if n <= 1:
        return 1

    # Bottom-up DP
    dp = [0] * (n + 1)
    dp[0] = dp[1] = 1

    for i in range(2, n + 1):
        dp[i] = dp[i-1] + dp[i-2]

    return dp[n]

    # Space-optimized version
    # if n <= 1:
    #     return 1
    # a, b = 1, 1
    # for _ in range(2, n + 1):
    #     a, b = b, a + b
    # return b
```

Optimization: Dynamic programming (bottom-up) to avoid repeated calculations. Can optimize space to O(1).

Justification: O(n) time vs O(2), linear time with constant space possible.

Time Complexity:

- **Brute Force:** O(2) exponential
- **Optimized:** O(n) linear

Space Complexity:

- **Brute Force:** O(n) recursion stack
- **Optimized:** O(1) with space optimization

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) when n1
- **Average Case:** O(2) for brute force, O(n) for optimized
- **Worst Case:** O(2) for brute force, O(n) for optimized

Q91. Tower of Hanoi

Question: Solve Tower of Hanoi problem for N disks, moving from source to destination using auxiliary peg.

Solution 1 – Brute Force Recursive Approach:

```python
def tower_of_hanoi_bf(n, source, destination, auxiliary):
    """Recursive Tower of Hanoi solution."""
    if n == 1:
        print(f"Move disk 1 from {source} to {destination}")
        return

    # Move n-1 disks from source to auxiliary
    tower_of_hanoi_bf(n-1, source, auxiliary, destination)

    # Move nth disk from source to destination
    print(f"Move disk {n} from {source} to {destination}")

    # Move n-1 disks from auxiliary to destination
    tower_of_hanoi_bf(n-1, auxiliary, destination, source)
```

Logic: Recursive solution: Move n-1 disks to auxiliary, move largest disk to destination, move n-1 disks to destination.

Inefficiencies: Requires 2 - 1 moves which is optimal. No better algorithm exists.

Solution 2 – Optimized Approach:

```python
def tower_of_hanoi_opt(n, source, destination, auxiliary):
    """Tower of Hanoi with move counting and iterative option."""
    total_moves = 0

    def hanoi(n, src, dest, aux):
        nonlocal total_moves
        if n == 0:
            return

        # Move n-1 disks from source to auxiliary
        hanoi(n-1, src, aux, dest)

        # Move nth disk from source to destination
        print(f"Move disk {n} from {src} to {dest}")
        total_moves += 1

        # Move n-1 disks from auxiliary to destination
        hanoi(n-1, aux, dest, src)

    hanoi(n, source, destination, auxiliary)
    print(f"Total moves: {total_moves}")
    return total_moves

# Note: No asymptotically better solution exists than 2^n - 1
    moves
```

Optimization: Same algorithm but with move counting. No asymptotically better solution exists.

Justification: 2 - 1 moves is mathematically proven minimum. Algorithm is optimal.

Time Complexity:

- **Brute Force:** O(2) moves and operations
- **Optimized:** O(2) moves and operations (optimal)

Space Complexity:

- **Brute Force:** O(n) recursion stack
- **Optimized:** O(n) recursion stack

Best / Average / Worst Case Analysis:

- **Best Case:** O(2) for both (inherent to problem)
- **Average Case:** O(2) for both
- **Worst Case:** O(2) for both

Q92. Rat in a Maze

Question: Find path for rat from top-left to bottom-right in maze with obstacles.

Solution 1 – Brute Force Backtracking Approach:

```
def rat_in_maze_bf(maze):
    """Find path using backtracking exploring all directions."""
    n = len(maze)
    if maze[0][0] == 0 or maze[n-1][n-1] == 0:
        return []

    path = []
    visited = [[False] * n for _ in range(n)]

    def backtrack(i, j):
        # If reached destination
        if i == n-1 and j == n-1:
            path.append((i, j))
            return True

        # Mark current cell as visited
        visited[i][j] = True

        # Try all possible directions: Down, Right, Up, Left
        directions = [(1, 0), (0, 1), (-1, 0), (0, -1)]

        for dx, dy in directions:
            ni, nj = i + dx, j + dy
            if (0 <= ni < n and 0 <= nj < n and
                not visited[ni][nj] and maze[ni][nj] == 1):
                if backtrack(ni, nj):
                    path.append((i, j))
                    return True

        # Backtrack
        visited[i][j] = False
        return False

    if backtrack(0, 0):
        return path[::-1]  # Reverse to get start to end
    return []
```

Logic: DFS with backtracking. Try all directions from current cell, backtrack if dead end.

Inefficiencies: May explore many dead ends. Order of directions affects performance.

Solution 2 – Optimized Approach:

```
def rat_in_maze_opt(maze):
    """Optimized with BFS for shortest path or heuristic
       ordering."""
    from collections import deque

    n = len(maze)
```

```
    if maze[0][0] == 0 or maze[n-1][n-1] == 0:
        return []

    # BFS for shortest path
    queue = deque()
    queue.append((0, 0, [(0, 0)]))  # (i, j, path)
    visited = [[False] * n for _ in range(n)]
    visited[0][0] = True

    # Directions in optimal order: Right, Down, Left, Up
    directions = [(0, 1), (1, 0), (0, -1), (-1, 0)]

    while queue:
        i, j, path = queue.popleft()

        if i == n-1 and j == n-1:
            return path

        for dx, dy in directions:
            ni, nj = i + dx, j + dy
            if (0 <= ni < n and 0 <= nj < n and
                not visited[ni][nj] and maze[ni][nj] == 1):
                visited[ni][nj] = True
                queue.append((ni, nj, path + [(ni, nj)]))

    return []
```

Optimization: Use BFS for shortest path. Order directions optimally (right/down first for typical mazes).

Justification: BFS finds shortest path. Direction ordering reduces exploration.

Time Complexity:

- **Brute Force:** $O(4^n) in worst case (explore all paths)$
- **Optimized:** $O(n^2)$ with BFS (visits each cell once)

Space Complexity:

- **Brute Force:** $O(n^2)$ for visited matrix + $O(n^2)$ recursion stack
- **Optimized:** $O(n^2)$ for visited matrix + $O(n^2)$ queue

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized with straight path
- **Average Case:** $O(4^n) for brute force, O(n) for optimized$
- **Worst Case:** $O(4^n) for brute force, O(n) for optimized$

Q93. N-Queens problem

Question: Place N queens on N×N chessboard so no two queens threaten each other.

Solution 1 – Brute Force Backtracking Approach:

```python
def n_queens_bf(n):
    """N-Queens using backtracking checking all positions."""
    board = [['.' for _ in range(n)] for _ in range(n)]
    solutions = []

    def is_safe(row, col):
        # Check column
        for i in range(row):
            if board[i][col] == 'Q':
                return False

        # Check diagonal \
        i, j = row-1, col-1
        while i >= 0 and j >= 0:
            if board[i][j] == 'Q':
                return False
            i -= 1; j -= 1

        # Check diagonal /
        i, j = row-1, col+1
        while i >= 0 and j < n:
            if board[i][j] == 'Q':
                return False
            i -= 1; j += 1

        return True

    def backtrack(row):
        if row == n:
            solutions.append([''.join(row) for row in board])
            return

        for col in range(n):
            if is_safe(row, col):
                board[row][col] = 'Q'
                backtrack(row + 1)
                board[row][col] = '.'

    backtrack(0)
    return solutions
```

Logic: Place queen row by row. For each row, try each column, check if safe, recurse.

Inefficiencies: is_safe() checks O(n) positions each time. Could use bitmask for faster checking.

Solution 2 – Optimized Approach:

```python
def n_queens_opt(n):
    """N-Queens using bitmasking for faster checking."""
```

```
    solutions = []

    def backtrack(row, cols, diag1, diag2, board):
        if row == n:
            solutions.append([''.join(row) for row in board])
            return

        # Get available positions using bitmask
        available = (~(cols | diag1 | diag2)) & ((1 << n) - 1)

        while available:
            # Get least significant set bit
            col_bit = available & -available
            col = (col_bit.bit_length() - 1)

            # Place queen
            board[row][col] = 'Q'

            # Recurse with updated masks
            backtrack(row + 1,
                      cols | col_bit,
                      (diag1 | col_bit) << 1,
                      (diag2 | col_bit) >> 1,
                      board)

            # Remove queen
            board[row][col] = '.'

            # Remove this column from available
            available &= available - 1

    board = [['.' for _ in range(n)] for _ in range(n)]
    backtrack(0, 0, 0, 0, board)
    return solutions
```

Optimization: Use bitmasking to track attacked columns/diagonals. O(1) safety check instead of O(n).

Justification: Significantly faster for larger n. Bit operations are very efficient.

Time Complexity:

- **Brute Force:** O(n!) worst case (but pruned)
- **Optimized:** O(n!) worst case but with much smaller constant factor

Space Complexity:

- **Brute Force:** $O(n^2)$ for board + O(n) recursion stack
- **Optimized:** $O(n^2)$ for board + O(n) recursion stack

Best / Average / Worst Case Analysis:

- **Best Case:** O(n!) for both (inherent to problem)

- **Average Case:** O(n!) for both
- **Worst Case:** O(n!) for both

Q94. Sudoku solver

Question: Solve 9×9 Sudoku puzzle using backtracking.

Solution 1 – Brute Force Backtracking Approach:

```
def sudoku_solver_bf(board):
    """Sudoku solver trying all numbers in empty cells."""
    n = 9

    def find_empty():
        for i in range(n):
            for j in range(n):
                if board[i][j] == 0:
                    return i, j
        return None

    def is_valid(num, row, col):
        # Check row
        for j in range(n):
            if board[row][j] == num:
                return False

        # Check column
        for i in range(n):
            if board[i][col] == num:
                return False

        # Check 3x3 box
        box_row, box_col = 3 * (row // 3), 3 * (col // 3)
        for i in range(box_row, box_row + 3):
            for j in range(box_col, box_col + 3):
                if board[i][j] == num:
                    return False

        return True

    def solve():
        empty = find_empty()
        if not empty:
            return True

        row, col = empty

        for num in range(1, 10):
            if is_valid(num, row, col):
                board[row][col] = num
                if solve():
                    return True
                board[row][col] = 0  # backtrack

        return False

```

```
    return solve()
```

Logic: Find empty cell, try numbers 1-9, check validity, recurse, backtrack if dead end.

Inefficiencies: Always tries numbers 1-9. No cell ordering or number ordering heuristics.

Solution 2 – Optimized Approach:

```
def sudoku_solver_opt(board):
    """Sudoku solver with MRV heuristic and forward checking."""
    n = 9

    # Track possible values for each cell
    rows = [set(range(1, 10)) for _ in range(n)]
    cols = [set(range(1, 10)) for _ in range(n)]
    boxes = [set(range(1, 10)) for _ in range(n)]

    # Initialize sets with existing numbers
    empty_cells = []
    for i in range(n):
        for j in range(n):
            if board[i][j] != 0:
                num = board[i][j]
                rows[i].remove(num)
                cols[j].remove(num)
                boxes[(i//3)*3 + j//3].remove(num)
            else:
                empty_cells.append((i, j))

    # Get possible values for a cell
    def get_possible(i, j):
        box_idx = (i//3)*3 + j//3
        return rows[i] & cols[j] & boxes[box_idx]

    # MRV: choose cell with fewest possibilities
    empty_cells.sort(key=lambda cell: len(get_possible(*cell)))

    def backtrack(idx):
        if idx == len(empty_cells):
            return True

        i, j = empty_cells[idx]
        possible = get_possible(i, j)

        for num in possible:
            # Place number
            board[i][j] = num
            rows[i].remove(num)
            cols[j].remove(num)
            boxes[(i//3)*3 + j//3].remove(num)

            if backtrack(idx + 1):
                return True
```

```
46
47              # Backtrack
48              board[i][j] = 0
49              rows[i].add(num)
50              cols[j].add(num)
51              boxes[(i//3)*3 + j//3].add(num)
52
53          return False
54
55      return backtrack(0)
```

Optimization: Use MRV (Minimum Remaining Values) heuristic. Maintain possible values sets for forward checking.

Justification: MRV reduces branching factor. Forward checking detects failures early.

Time Complexity:

- **Brute Force:** $O(9^{empty}) where empty is number of empty cells$
- **Optimized:** Still exponential but much faster in practice

Space Complexity:

- **Brute Force:** O(1) additional space
- **Optimized:** $O(n^2)$ for tracking possible values

Best / Average / Worst Case Analysis:

- **Best Case:** Exponential for both (NP-complete problem)
- **Average Case:** Exponential but optimized is much faster
- **Worst Case:** Exponential for both

Q95. Word Search in 2D grid

Question: Search for word in 2D character grid (letters can be used only once).

Solution 1 – Brute Force Backtracking Approach:

```python
def word_search_bf(board, word):
    """Word search trying all starting positions and
       directions."""
    rows, cols = len(board), len(board[0])
    n = len(word)

    def backtrack(i, j, k, visited):
        if k == n:
            return True

        if (i < 0 or i >= rows or j < 0 or j >= cols or
            visited[i][j] or board[i][j] != word[k]):
            return False

        visited[i][j] = True

        # Try all 4 directions
        directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
        for dx, dy in directions:
            if backtrack(i + dx, j + dy, k + 1, visited):
                return True

        visited[i][j] = False
        return False

    visited = [[False] * cols for _ in range(rows)]

    # Try all starting positions
    for i in range(rows):
        for j in range(cols):
            if backtrack(i, j, 0, visited):
                return True

    return False
```

Logic: For each starting position, DFS in all 4 directions, backtrack if mismatch or visited.

Inefficiencies: Creates new visited matrix for each starting position. No pruning.

Solution 2 – Optimized Approach:

```python
def word_search_opt(board, word):
    """Word search with in-place modification and early
       pruning."""
    rows, cols = len(board), len(board[0])
    n = len(word)

    # Pruning: check if board has enough characters
    from collections import Counter
```

```
    board_counts = Counter(ch for row in board for ch in row)
    word_counts = Counter(word)
    for ch, count in word_counts.items():
        if board_counts[ch] < count:
            return False

    def backtrack(i, j, k):
        if k == n:
            return True

        if (i < 0 or i >= rows or j < 0 or j >= cols or
            board[i][j] != word[k]):
            return False

        # Mark cell as visited by modifying in-place
        temp = board[i][j]
        board[i][j] = '#'

        # Try all 4 directions
        found = (backtrack(i+1, j, k+1) or
                 backtrack(i-1, j, k+1) or
                 backtrack(i, j+1, k+1) or
                 backtrack(i, j-1, k+1))

        # Restore cell
        board[i][j] = temp
        return found

    # Try all starting positions
    for i in range(rows):
        for j in range(cols):
            if backtrack(i, j, 0):
                return True

    return False
```

Optimization: In-place modification instead of visited matrix. Character count pruning.

Justification: O(1) extra space vs O(rows×cols). Pruning avoids unnecessary searches.

Time Complexity:

- **Brute Force:** O(rows×cols×$4^{len(word)}$)
- **Optimized:** O(rows×cols×$4^{len(word)}$)$butwithpruning$

Space Complexity:

- **Brute Force:** O(rows×cols) for visited + O(len(word)) recursion stack
- **Optimized:** O(len(word)) recursion stack (in-place)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if word not in character counts
- **Average Case:** Exponential but optimized prunes early
- **Worst Case:** O(rows×cols×$4^{len(word)}$)$forboth$

Q96. All subsequences of a string

Question: Generate all subsequences (not necessarily contiguous) of a string.

Solution 1 – Brute Force Approach:

```
def subsequences_bf(s):
    """Generate subsequences using recursion exploring all."""
    result = []

    def backtrack(idx, current):
        if idx == len(s):
            result.append(''.join(current))
            return

        # Exclude current character
        backtrack(idx + 1, current)

        # Include current character
        current.append(s[idx])
        backtrack(idx + 1, current)
        current.pop()

    backtrack(0, [])
    return result
```

Logic: For each character, either include or exclude it. Explore both possibilities recursively.

Inefficiencies: Generates 2 subsequences which is optimal. No significant inefficiency.

Solution 2 – Optimized Approach:

```
def subsequences_opt(s):
    """Generate subsequences using bitmask or iterative."""
    n = len(s)
    result = []

    # Bitmask approach
    for mask in range(1 << n):
        subsequence = []
        for i in range(n):
            if mask & (1 << i):
                subsequence.append(s[i])
        result.append(''.join(subsequence))

    return result

    # Alternative: iterative approach
    # result = [""]
    # for char in s:
    #     result += [seq + char for seq in result]
    # return result
```

Optimization: Bitmask approach or iterative building. Avoids recursion overhead.

Justification: Bitmask is elegant for small n (64). Iterative approach clear and efficient.

Time Complexity:

- **Brute Force:** O(n × 2) where n is string length
- **Optimized:** O(n × 2) where n is string length (inherent to problem)

Space Complexity:

- **Brute Force:** O(n) recursion stack + O(n × 2) output
- **Optimized:** O(n × 2) output

Best / Average / Worst Case Analysis:

- **Best Case:** O(2) for both (inherent to problem)
- **Average Case:** O(n × 2) for both
- **Worst Case:** O(n × 2) for both

Q97. Subset sum problem

Question: Check if there exists subset with sum equal to K.

Solution 1 – Brute Force Recursive Approach:

```python
def subset_sum_bf(nums, k):
    """Check subset sum using recursion exploring all subsets."""
    def backtrack(idx, current_sum):
        if current_sum == k:
            return True
        if idx == len(nums) or current_sum > k:
            return False

        # Include current number
        if backtrack(idx + 1, current_sum + nums[idx]):
            return True

        # Exclude current number
        if backtrack(idx + 1, current_sum):
            return True

        return False

    return backtrack(0, 0)
```

Logic: For each number, either include or exclude it. Track current sum, return True if equals K.

Inefficiencies: Exponential O(2) time. No pruning for negative numbers or large K.

Solution 2 – Optimized Approach:

```python
def subset_sum_opt(nums, k):
    """Subset sum using dynamic programming."""
    n = len(nums)

    # DP table: dp[i][j] = can we make sum j using first i
       elements
    dp = [[False] * (k + 1) for _ in range(n + 1)]

    # Base case: sum 0 is always possible
    for i in range(n + 1):
        dp[i][0] = True

    # Fill DP table
    for i in range(1, n + 1):
        for j in range(1, k + 1):
            if nums[i-1] <= j:
                dp[i][j] = dp[i-1][j] or dp[i-1][j - nums[i-1]]
            else:
                dp[i][j] = dp[i-1][j]

    return dp[n][k]

    # Space optimized version
```

```
    # dp = [False] * (k + 1)
    # dp[0] = True
    # for num in nums:
    #     for j in range(k, num - 1, -1):
    #         dp[j] = dp[j] or dp[j - num]
    # return dp[k]
```

Optimization: Dynamic programming (0/1 knapsack). O(n×k) time vs O(2).

Justification: Pseudo-polynomial time. Much faster when k is not too large compared to n.

Time Complexity:

- **Brute Force:** O(2) exponential
- **Optimized:** O(n×k) pseudo-polynomial

Space Complexity:

- **Brute Force:** O(n) recursion stack
- **Optimized:** O(n×k) for DP table or O(k) space-optimized

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if k=0
- **Average Case:** O(2) for brute force, O(n×k) for optimized
- **Worst Case:** O(2) for brute force, O(n×k) for optimized

Q98. Generate all binary strings of length N

Question: Generate all possible binary strings of length N (strings of 0s and 1s).

Solution 1 – Brute Force Recursive Approach:

```python
def binary_strings_bf(n):
    """Generate binary strings using recursion."""
    result = []

    def backtrack(current):
        if len(current) == n:
            result.append(''.join(current))
            return

        # Try '0'
        current.append('0')
        backtrack(current)
        current.pop()

        # Try '1'
        current.append('1')
        backtrack(current)
        current.pop()

    backtrack([])
    return result
```

Logic: Build string character by character. For each position, try '0' then '1'.

Inefficiencies: Generates 2 strings which is optimal. No significant inefficiency.

Solution 2 – Optimized Approach:

```python
def binary_strings_opt(n):
    """Generate binary strings using iterative approach."""
    result = []

    # Generate all numbers from 0 to 2^n - 1
    for i in range(1 << n):
        # Convert to binary string with leading zeros
        binary = bin(i)[2:].zfill(n)
        result.append(binary)

    return result

    # Alternative: using product from itertools
    # from itertools import product
    # return [''.join(p) for p in product('01', repeat=n)]
```

Optimization: Generate numbers 0 to 2-1, convert to binary. Or use itertools.product.

Justification: Simpler, avoids recursion overhead. itertools.product is Pythonic.

Time Complexity:

- **Brute Force:** O(n × 2) where n is string length
- **Optimized:** O(n × 2) where n is string length (inherent to problem)

Space Complexity:

- **Brute Force:** O(n) recursion stack + O(n × 2) output
- **Optimized:** O(n × 2) output

Best / Average / Worst Case Analysis:

- **Best Case:** O(2) for both (inherent to problem)
- **Average Case:** O(n × 2) for both
- **Worst Case:** O(n × 2) for both

Q99. Generate valid parentheses combinations

Question: Generate all valid combinations of N pairs of parentheses.

Solution 1 – Brute Force Approach:

```python
def generate_parentheses_bf(n):
    """Generate all strings of 2n parentheses, filter valid
       ones."""
    result = []

    def generate_all(current):
        if len(current) == 2 * n:
            # Check if valid
            balance = 0
            for ch in current:
                if ch == '(':
                    balance += 1
                else:
                    balance -= 1
                if balance < 0:
                    return
            if balance == 0:
                result.append(''.join(current))
            return

        # Try '('
        current.append('(')
        generate_all(current)
        current.pop()

        # Try ')'
        current.append(')')
        generate_all(current)
        current.pop()

    generate_all([])
    return result
```

Logic: Generate all strings of length 2n with '(' and ')', filter valid ones using balance check.

Inefficiencies: Generates $2^{2n} strings, most invalid. Late validation.$

Solution 2 – Optimized Approach:

```python
def generate_parentheses_opt(n):
    """Generate valid parentheses using backtracking with
       constraints."""
    result = []

    def backtrack(current, open_count, close_count):
        if len(current) == 2 * n:
            result.append(''.join(current))
            return

```

```
        # Add '(' if we haven't used all opening parentheses
        if open_count < n:
            current.append('(')
            backtrack(current, open_count + 1, close_count)
            current.pop()

        # Add ')' if we have more opening than closing
        if close_count < open_count:
            current.append(')')
            backtrack(current, open_count, close_count + 1)
            current.pop()

    backtrack([], 0, 0)
    return result
```

Optimization: Track counts of open and close parentheses. Only add ')' when close_count ¡ open_count.

Justification: Only generates valid strings. Reduces from $O(2^{2n})toO(Catalan(n)) = O(4/n)$.

Time Complexity:

- **Brute Force:** $O(2^{2n}2n) = O(n4)$
- **Optimized:** O(Catalan(n)) = O(4/n)

Space Complexity:

- **Brute Force:** O(n) recursion stack + O(Catalan(n) × 2n) output
- **Optimized:** O(n) recursion stack + O(Catalan(n) × 2n) output

Best / Average / Worst Case Analysis:

- **Best Case:** O(4/n) for optimized
- **Average Case:** O(n × 4) for brute force, O(4/n) for optimized
- **Worst Case:** O(n × 4) for brute force, O(4/n) for optimized

Q100. Generate unique permutations with duplicates

Question: Generate all unique permutations of string that may have duplicate characters.

Solution 1 – Brute Force Approach:

```
def unique_permutations_bf(s):
    """Generate all permutations, filter duplicates."""
    result = []
    chars = list(s)

    def backtrack(start):
        if start == len(chars):
            perm = ''.join(chars)
            if perm not in result:  # Check for duplicates
                result.append(perm)
            return

        for i in range(start, len(chars)):
            chars[start], chars[i] = chars[i], chars[start]
            backtrack(start + 1)
            chars[start], chars[i] = chars[i], chars[start]

    backtrack(0)
    return result
```

Logic: Generate all permutations using swapping, store in list, filter duplicates.

Inefficiencies: Generates duplicate permutations, filters at end. O(n!) permutations even with duplicates.

Solution 2 – Optimized Approach:

```
def unique_permutations_opt(s):
    """Generate unique permutations using frequency count."""
    from collections import Counter
    result = []

    def backtrack(current, counter):
        if len(current) == len(s):
            result.append(''.join(current))
            return

        for char in list(counter.keys()):  # Use list to avoid
           modification during iteration
            if counter[char] > 0:
                # Choose this character
                current.append(char)
                counter[char] -= 1

                # Recurse
                backtrack(current, counter)

                # Backtrack
                current.pop()
```

```
                counter[char] += 1

    backtrack([], Counter(s))
    return result
```

Optimization: Use frequency counter. Only choose characters with remaining count ¿ 0.

Justification: Generates only unique permutations. Avoids duplicate generation and filtering.

Time Complexity:

- **Brute Force:** O(n! × n) where n is string length (even with duplicates)
- **Optimized:** O(k) where k is number of unique permutations

Space Complexity:

- **Brute Force:** O(n) recursion stack + O(n! × n) output
- **Optimized:** O(n) recursion stack + O(k × n) output

Best / Average / Worst Case Analysis:

- **Best Case:** O(k) for optimized (k = unique permutations)
- **Average Case:** O(n! × n) for brute force, O(k) for optimized
- **Worst Case:** O(n! × n) for brute force (all characters unique), O(n! × n) for optimized (all unique)

SECTION E: SEARCHING, SORTING & ADVANCED PYTHON (50 Questions)

Q101. Linear search implementation

Question: Search for target element in array by checking each element sequentially.

Solution 1 – Brute Force Approach:

```
def linear_search_bf(arr, target):
    """Linear search checking each element."""
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1
```

Logic: Iterate through array from start to end, compare each element with target.

Inefficiencies: O(n) time always, even if element at beginning. No early optimization possible.

Solution 2 – Optimized Approach:

```python
def linear_search_opt(arr, target):
    """Linear search with sentinel or early exit optimization."""
    # Add sentinel to avoid bounds checking
    n = len(arr)
    if n == 0:
        return -1

    # Store last element
    last = arr[n-1]
    # Replace last element with target
    arr[n-1] = target

    i = 0
    while arr[i] != target:
        i += 1

    # Restore last element
    arr[n-1] = last

    if i < n-1 or last == target:
        return i
    return -1
```

Optimization: Sentinel technique: place target at end to eliminate bounds checking in loop.

Justification: Reduces comparisons by eliminating i ¡ n check in loop. Marginally faster.

Time Complexity:

- **Brute Force:** O(n) where n is array length
- **Optimized:** O(n) where n is array length

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if target at index 0
- **Average Case:** O(n) for both
- **Worst Case:** O(n) for both (target not present or at end)

Q102. Iterative binary search

Question: Search for target in sorted array using iterative binary search.

Solution 1 – Brute Force Approach:

```python
def binary_search_iterative_bf(arr, target):
    """Binary search with while loop."""
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1
```

Logic: Maintain left and right pointers. Calculate mid, compare with target, adjust pointers accordingly.

Inefficiencies: Standard binary search is already optimal. Integer overflow possible in mid calculation.

Solution 2 – Optimized Approach:

```python
def binary_search_iterative_opt(arr, target):
    """Binary search with overflow-safe mid calculation."""
    left, right = 0, len(arr) - 1

    while left <= right:
        # Avoid overflow: left + (right - left) // 2
        mid = left + (right - left) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1
```

Optimization: Use left + (right - left) // 2 to avoid integer overflow for large arrays.

Justification: Same time complexity but safer for large indices.

Time Complexity:

- **Brute Force:** O(log n) where n is array length
- **Optimized:** O(log n) where n is array length

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if target at middle
- **Average Case:** O(log n) for both
- **Worst Case:** O(log n) for both

Q103. Recursive binary search

Question: Search for target in sorted array using recursive binary search.

Solution 1 – Brute Force Approach:

```python
def binary_search_recursive_bf(arr, target):
    """Recursive binary search."""
    def search(left, right):
        if left > right:
            return -1

        mid = (left + right) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            return search(mid + 1, right)
        else:
            return search(left, mid - 1)

    return search(0, len(arr) - 1)
```

Logic: Recursively divide search space in half based on comparison with middle element.

Inefficiencies: Recursion overhead. Stack depth O(log n). Integer overflow in mid calculation.

Solution 2 – Optimized Approach:

```python
def binary_search_recursive_opt(arr, target):
    """Recursive binary search with tail recursion."""
    def search(left, right):
        if left > right:
            return -1

        # Overflow-safe mid calculation
        mid = left + (right - left) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            return search(mid + 1, right)  # Tail recursion
        else:
            return search(left, mid - 1)    # Tail recursion

    return search(0, len(arr) - 1)
```

Optimization: Use overflow-safe mid calculation. Tail recursion pattern (though Python doesn't optimize).

Justification: Same time complexity but safer and cleaner.

Time Complexity:

- **Brute Force:** O(log n) where n is array length
- **Optimized:** O(log n) where n is array length

Space Complexity:

- **Brute Force:** O(log n) recursion stack
- **Optimized:** O(log n) recursion stack

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if target at middle
- **Average Case:** O(log n) for both
- **Worst Case:** O(log n) for both

Q104. Bubble sort implementation

Question: Sort array using bubble sort (repeatedly swap adjacent elements if wrong order).

Solution 1 – Brute Force Approach:

```python
def bubble_sort_bf(arr):
    """Bubble sort with nested loops."""
    n = len(arr)

    for i in range(n):
        for j in range(n - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr
```

Logic: Pass through array n times, each time comparing adjacent elements and swapping if out of order.

Inefficiencies: Always does n passes even if array sorted early. Compares all pairs each pass.

Solution 2 – Optimized Approach:

```python
def bubble_sort_opt(arr):
    """Bubble sort with early termination."""
    n = len(arr)

    for i in range(n):
        swapped = False
        # Last i elements already sorted
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        # If no swaps, array is sorted
        if not swapped:
            break

    return arr
```

Optimization: Track if any swaps occurred. Reduce inner loop range as elements bubble to end.

Justification: $O(n^2)$ worst case but O(n) best case (already sorted). Early termination.

Time Complexity:

- **Brute Force:** $O(n^2)$ always
- **Optimized:** $O(n^2)$ worst, O(n) best

Space Complexity:

- **Brute Force:** O(1) in-place

- **Optimized:** O(1) in-place

Best / Average / Worst Case Analysis:

- **Best Case:** O(n^2) for brute force, O(n) for optimized
- **Average Case:** O(n^2) for both
- **Worst Case:** O(n^2) for both

Q105. Selection sort implementation

Question: Sort array using selection sort (repeatedly find minimum and swap with current position).

Solution 1 – Brute Force Approach:

```python
def selection_sort_bf(arr):
    """Selection sort finding minimum each iteration."""
    n = len(arr)

    for i in range(n):
        # Find minimum in unsorted portion
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j

        # Swap with current position
        arr[i], arr[min_idx] = arr[min_idx], arr[i]

    return arr
```

Logic: For each position i, find minimum element in arr[i:], swap with arr[i].

Inefficiencies: Always does n passes. Finds minimum by scanning entire unsorted portion.

Solution 2 – Optimized Approach:

```python
def selection_sort_opt(arr):
    """Selection sort with early exit if already sorted."""
    n = len(arr)

    for i in range(n):
        min_idx = i
        is_sorted = True

        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
            if arr[j] < arr[j - 1]:
                is_sorted = False

        # If already sorted, break
        if is_sorted:
            break

        # Swap if needed
        if min_idx != i:
            arr[i], arr[min_idx] = arr[min_idx], arr[i]

    return arr
```

Optimization: Check if already sorted during min search. Early exit if sorted.

Justification: Still $O(n^2)$ but can exit early if array becomes sorted.

Time Complexity:

- **Brute Force:** $O(n^2)$ always
- **Optimized:** $O(n^2)$ worst, $O(n)$ best

Space Complexity:

- **Brute Force:** $O(1)$ in-place
- **Optimized:** $O(1)$ in-place

Best / Average / Worst Case Analysis:

- **Best Case:** $O(n^2)$ for brute force, $O(n)$ for optimized
- **Average Case:** $O(n^2)$ for both
- **Worst Case:** $O(n^2)$ for both

Q106. Insertion sort implementation

Question: Sort array using insertion sort (build sorted array one element at a time).

Solution 1 – Brute Force Approach:

```python
def insertion_sort_bf(arr):
    """Insertion sort with nested loops."""
    n = len(arr)

    for i in range(1, n):
        key = arr[i]
        j = i - 1

        # Shift elements greater than key right
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key

    return arr
```

Logic: For each element, insert into correct position in sorted portion to its left.

Inefficiencies: Standard insertion sort is already efficient for small arrays or nearly sorted arrays.

Solution 2 – Optimized Approach:

```python
def insertion_sort_opt(arr):
    """Insertion sort with binary search for insertion point."""
    n = len(arr)

    for i in range(1, n):
        key = arr[i]

        # Find insertion position using binary search
        left, right = 0, i - 1
        while left <= right:
            mid = (left + right) // 2
            if arr[mid] < key:
                left = mid + 1
            else:
                right = mid - 1

        # Insertion position is 'left'
        # Shift elements
        for j in range(i - 1, left - 1, -1):
            arr[j + 1] = arr[j]

        arr[left] = key

    return arr
```

Optimization: Use binary search to find insertion point O(log n) instead of linear search O(n).

Justification: Reduces comparisons from $O(n^2)$ to O(n log n) but shifts remain $O(n^2)$.

Time Complexity:

- **Brute Force:** $O(n^2)$ worst, O(n) best (nearly sorted)
- **Optimized:** $O(n^2)$ worst (shifts), O(n log n) comparisons

Space Complexity:

- **Brute Force:** O(1) in-place
- **Optimized:** O(1) in-place

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for both (already sorted)
- **Average Case:** $O(n^2)$ for brute force, $O(n^2)$ for optimized (shifts dominate)
- **Worst Case:** $O(n^2)$ for both

Q107. Merge sort implementation

Question: Sort array using merge sort (divide and conquer, recursive merging).

Solution 1 – Brute Force Approach:

```python
def merge_sort_bf(arr):
    """Merge sort creating new arrays for each merge."""
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort_bf(arr[:mid])
    right = merge_sort_bf(arr[mid:])

    # Merge
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result
```

Logic: Divide array in half recursively until single elements, merge sorted halves.

Inefficiencies: Creates many temporary arrays. Not in-place.

Solution 2 – Optimized Approach:

```python
def merge_sort_opt(arr):
    """Merge sort with auxiliary array for merging."""
    def merge_sort_helper(arr, temp, left, right):
        if left < right:
            mid = (left + right) // 2

            # Sort halves
            merge_sort_helper(arr, temp, left, mid)
            merge_sort_helper(arr, temp, mid + 1, right)

            # Merge halves
            i, j, k = left, mid + 1, left

            while i <= mid and j <= right:
                if arr[i] <= arr[j]:
                    temp[k] = arr[i]
                    i += 1
                else:
```

```
                    temp[k] = arr[j]
                    j += 1
                k += 1

            # Copy remaining elements
            while i <= mid:
                temp[k] = arr[i]
                i += 1; k += 1
            while j <= right:
                temp[k] = arr[j]
                j += 1; k += 1

            # Copy back to original array
            for m in range(left, right + 1):
                arr[m] = temp[m]

    temp = [0] * len(arr)
    merge_sort_helper(arr, temp, 0, len(arr) - 1)
    return arr
```

Optimization: Use single auxiliary array for all merges. In-place merging with auxiliary storage.

Justification: Reduces memory allocations. More cache-friendly.

Time Complexity:

- **Brute Force:** O(n log n) where n is array length
- **Optimized:** O(n log n) where n is array length

Space Complexity:

- **Brute Force:** O(n log n) for recursive calls and arrays
- **Optimized:** O(n) for auxiliary array + O(log n) recursion stack

Best / Average / Worst Case Analysis:

- **Best Case:** O(n log n) for both
- **Average Case:** O(n log n) for both
- **Worst Case:** O(n log n) for both

Q108. Quick sort implementation

Question: Sort array using quick sort (partition around pivot, recursively sort partitions).

Solution 1 – Brute Force Approach:

```python
def quick_sort_bf(arr):
    """Quick sort creating new lists for partitions."""
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quick_sort_bf(left) + middle + quick_sort_bf(right)
```

Logic: Choose pivot, create three lists: less than, equal to, greater than pivot. Recursively sort.

Inefficiencies: Creates many new lists O(n log n) memory. Not in-place.

Solution 2 – Optimized Approach:

```python
def quick_sort_opt(arr):
    """In-place quick sort with Hoare or Lomuto partition."""
    def partition(arr, low, high):
        """Lomuto partition scheme."""
        pivot = arr[high]
        i = low - 1

        for j in range(low, high):
            if arr[j] <= pivot:
                i += 1
                arr[i], arr[j] = arr[j], arr[i]

        arr[i + 1], arr[high] = arr[high], arr[i + 1]
        return i + 1

    def quick_sort_helper(arr, low, high):
        if low < high:
            pi = partition(arr, low, high)
            quick_sort_helper(arr, low, pi - 1)
            quick_sort_helper(arr, pi + 1, high)

    quick_sort_helper(arr, 0, len(arr) - 1)
    return arr
```

Optimization: In-place partitioning (Lomuto or Hoare). Randomized pivot selection for average case O(n log n).

Justification: O(1) extra space (excluding recursion stack). Cache-friendly.

Time Complexity:

- **Brute Force:** O(n log n) average, O(n^2) worst (bad pivot)

- **Optimized:** O(n log n) average, O(n^2) worst (but rare with randomization)

Space Complexity:

- **Brute Force:** O(n log n) for all created lists
- **Optimized:** O(log n) recursion stack average, O(n) worst

Best / Average / Worst Case Analysis:

- **Best Case:** O(n log n) for both
- **Average Case:** O(n log n) for both
- **Worst Case:** O(n^2) for both (already sorted with bad pivot)

Q109. Search in rotated sorted array

Question: Search for target in array that was sorted then rotated at unknown pivot.

Solution 1 – Brute Force Approach:

```python
def search_rotated_bf(arr, target):
    """Linear search in rotated array."""
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1
```

Logic: Check each element sequentially since rotation point unknown.

Inefficiencies: O(n) time, doesn't use sorted property after rotation.

Solution 2 – Optimized Approach:

```python
def search_rotated_opt(arr, target):
    """Binary search in rotated sorted array."""
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == target:
            return mid

        # Check which half is sorted
        if arr[left] <= arr[mid]:  # Left half sorted
            if arr[left] <= target < arr[mid]:
                right = mid - 1
            else:
                left = mid + 1
        else:  # Right half sorted
            if arr[mid] < target <= arr[right]:
                left = mid + 1
            else:
                right = mid - 1

    return -1
```

Optimization: Modified binary search. Determine which half is sorted, check if target in that range.

Justification: O(log n) time vs O(n), uses sorted property of at least one half.

Time Complexity:

- **Brute Force:** O(n) where n is array length
- **Optimized:** O(log n) where n is array length

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if target at middle
- **Average Case:** O(n) for brute force, O(log n) for optimized
- **Worst Case:** O(n) for brute force, O(log n) for optimized

Q110. First and last occurrence in sorted array

Question: Find first and last index of target in sorted array (could have duplicates).

Solution 1 – Brute Force Approach:

```
def find_first_last_bf(arr, target):
    """Linear scan to find first and last occurrence."""
    first = last = -1

    for i in range(len(arr)):
        if arr[i] == target:
            if first == -1:
                first = i
            last = i

    return [first, last]
```

Logic: Scan array, track first and last time target seen.

Inefficiencies: O(n) time, doesn't use sorted property.

Solution 2 – Optimized Approach:

```
def find_first_last_opt(arr, target):
    """Binary search for first and last occurrence."""
    def find_first(arr, target):
        left, right = 0, len(arr) - 1
        first = -1

        while left <= right:
            mid = (left + right) // 2

            if arr[mid] == target:
                first = mid
                right = mid - 1  # Continue searching left
            elif arr[mid] < target:
                left = mid + 1
            else:
                right = mid - 1

        return first

    def find_last(arr, target):
        left, right = 0, len(arr) - 1
        last = -1

        while left <= right:
            mid = (left + right) // 2

            if arr[mid] == target:
                last = mid
                left = mid + 1  # Continue searching right
            elif arr[mid] < target:
                left = mid + 1
            else:
```

```
                right = mid - 1

        return last

    return [find_first(arr, target), find_last(arr, target)]
```

Optimization: Two binary searches: one for first occurrence, one for last.
Justification: O(log n) time vs O(n), uses sorted property efficiently.
Time Complexity:

- **Brute Force:** O(n) where n is array length
- **Optimized:** O(log n) where n is array length

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if target not present
- **Average Case:** O(n) for brute force, O(log n) for optimized
- **Worst Case:** O(n) for brute force, O(log n) for optimized

Q111. Most frequent element using Counter

Question: Find element that appears most frequently in list using collections.Counter.
Solution 1 – Brute Force Approach:

```python
def most_frequent_bf(arr):
    """Find most frequent by counting each element."""
    if not arr:
        return None

    max_count = 0
    most_freq = None

    for i in range(len(arr)):
        count = 0
        for j in range(len(arr)):
            if arr[i] == arr[j]:
                count += 1

        if count > max_count:
            max_count = count
            most_freq = arr[i]

    return most_freq
```

Logic: For each element, count its occurrences by scanning entire list.
Inefficiencies: $O(n^2)$ time, counts each element separately.
Solution 2 – Optimized Approach:

```python
from collections import Counter

def most_frequent_opt(arr):
    """Find most frequent using Counter."""
    if not arr:
        return None

    # Count frequencies
    counter = Counter(arr)

    # Find element with max frequency
    return max(counter.items(), key=lambda x: x[1])[0]

    # Or using most_common() method
    # return counter.most_common(1)[0][0]
```

Optimization: Use Counter for O(n) frequency counting. Use max with key function.
Justification: O(n) time vs $O(n^2)$, uses hash map for efficient counting.
Time Complexity:

- **Brute Force:** $O(n^2)$ where n is list length
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(k) where k is unique elements

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** O(n^2) for brute force, O(n) for optimized
- **Worst Case:** O(n^2) for brute force, O(n) for optimized

Q112. Top K frequent elements

Question: Find K most frequent elements in list.

Solution 1 – Brute Force Approach:

```python
def top_k_frequent_bf(arr, k):
    """Find top K frequent by sorting frequency list."""
    # Count frequencies
    freq = {}
    for num in arr:
        freq[num] = freq.get(num, 0) + 1

    # Sort by frequency
    sorted_items = sorted(freq.items(), key=lambda x: x[1],
       reverse=True)

    # Get top K
    return [item[0] for item in sorted_items[:k]]
```

Logic: Count frequencies with dictionary, sort items by frequency, take top K.

Inefficiencies: O(n log n) due to sorting when we only need top K.

Solution 2 – Optimized Approach:

```python
import heapq
from collections import Counter

def top_k_frequent_opt(arr, k):
    """Find top K frequent using heap."""
    # Count frequencies
    counter = Counter(arr)

    # Use min-heap of size k
    heap = []

    for num, count in counter.items():
        heapq.heappush(heap, (count, num))
        if len(heap) > k:
            heapq.heappop(heap)  # Remove smallest

    # Extract results
    return [num for count, num in heap]

    # Alternative: nlargest
    # return [num for num, _ in counter.most_common(k)]
```

Optimization: Use min-heap of size K to track top K elements. O(n log k) vs O(n log n).

Justification: Better when k ¡¡ n. Heap operations O(log k) vs sort O(n log n).

Time Complexity:

- **Brute Force:** O(n log n) where n is list length
- **Optimized:** O(n log k) where n is list length, k is parameter

Space Complexity:

- **Brute Force:** O(n) for frequency dict
- **Optimized:** O(n) for frequency + O(k) for heap

Best / Average / Worst Case Analysis:

- **Best Case:** O(n log k) for optimized
- **Average Case:** O(n log n) for brute force, O(n log k) for optimized
- **Worst Case:** O(n log n) for brute force, O(n log k) for optimized

Q113. Stack implementation using list

Question: Implement stack (LIFO) data structure using Python list.

Solution 1 – Brute Force Approach:

```python
class StackBF:
    """Stack implementation using list with basic operations."""
    def __init__(self):
        self.items = []

    def push(self, item):
        """Add item to top of stack."""
        self.items.append(item)

    def pop(self):
        """Remove and return top item."""
        if not self.is_empty():
            return self.items.pop()
        raise IndexError("pop from empty stack")

    def peek(self):
        """Return top item without removing."""
        if not self.is_empty():
            return self.items[-1]
        raise IndexError("peek from empty stack")

    def is_empty(self):
        """Check if stack is empty."""
        return len(self.items) == 0

    def size(self):
        """Return number of items in stack."""
        return len(self.items)
```

Logic: Use list with append() for push, pop() for pop, [-1] for peek.

Inefficiencies: Standard implementation is already optimal for stack operations.

Solution 2 – Optimized Approach:

```python
class StackOpt:
    """Optimized stack with capacity limit and array
       preallocation."""
    def __init__(self, capacity=None):
        self.capacity = capacity
        self.items = []

    def push(self, item):
        """Add item with capacity check."""
        if self.capacity and len(self.items) >= self.capacity:
            raise OverflowError("stack overflow")
        self.items.append(item)

    def pop(self):
        """Pop with underflow check."""
```

```
        if self.is_empty():
            raise IndexError("pop from empty stack")
        return self.items.pop()

    def peek(self):
        """Peek with empty check."""
        if self.is_empty():
            raise IndexError("peek from empty stack")
        return self.items[-1]

    def is_empty(self):
        return len(self.items) == 0

    def is_full(self):
        return self.capacity and len(self.items) >= self.capacity

    def size(self):
        return len(self.items)

    def clear(self):
        """Clear stack efficiently."""
        self.items.clear()
```

Optimization: Add capacity limit, efficient clear method. Error handling.

Justification: More robust with capacity checking. Clear method O(1) vs O(n) reassignment.

Time Complexity:

- **Brute Force:** O(1) for push/pop/peek (amortized)
- **Optimized:** O(1) for push/pop/peek (amortized)

Space Complexity:

- **Brute Force:** O(n) where n is stack size
- **Optimized:** O(n) where n is stack size

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) for all operations
- **Average Case:** O(1) for all operations
- **Worst Case:** O(n) for push when list needs resizing (amortized O(1))

Q114. Queue implementation using deque

Question: Implement queue (FIFO) data structure using collections.deque.

Solution 1 – Brute Force Approach:

```python
from collections import deque

class QueueBF:
    """Queue implementation using list (inefficient)."""
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        """Add item to rear."""
        self.items.append(item)

    def dequeue(self):
        """Remove and return front item."""
        if not self.is_empty():
            return self.items.pop(0)  # O(n) operation
        raise IndexError("dequeue from empty queue")

    def front(self):
        """Return front item without removing."""
        if not self.is_empty():
            return self.items[0]
        raise IndexError("front from empty queue")

    def is_empty(self):
        return len(self.items) == 0

    def size(self):
        return len(self.items)
```

Logic: Use list with append() for enqueue, pop(0) for dequeue (but pop(0) is O(n)).

Inefficiencies: pop(0) is O(n) due to shifting all elements.

Solution 2 – Optimized Approach:

```python
from collections import deque

class QueueOpt:
    """Queue implementation using deque (efficient)."""
    def __init__(self):
        self.items = deque()

    def enqueue(self, item):
        """Add item to rear - O(1)."""
        self.items.append(item)

    def dequeue(self):
        """Remove and return front item - O(1)."""
        if not self.is_empty():
            return self.items.popleft()
```

```
        raise IndexError("dequeue from empty queue")

    def front(self):
        """Return front item without removing - O(1)."""
        if not self.is_empty():
            return self.items[0]
        raise IndexError("front from empty queue")

    def rear(self):
        """Return rear item without removing - O(1)."""
        if not self.is_empty():
            return self.items[-1]
        raise IndexError("rear from empty queue")

    def is_empty(self):
        return len(self.items) == 0

    def size(self):
        return len(self.items)

    def clear(self):
        """Clear queue efficiently."""
        self.items.clear()
```

Optimization: Use deque which provides O(1) append and popleft operations.

Justification: O(1) enqueue/dequeue vs O(n) dequeue with list. Deque is doubly-linked list.

Time Complexity:

- **Brute Force:** O(1) enqueue, O(n) dequeue
- **Optimized:** O(1) enqueue and dequeue

Space Complexity:

- **Brute Force:** O(n) where n is queue size
- **Optimized:** O(n) where n is queue size

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) for optimized operations
- **Average Case:** O(n) for brute force dequeue, O(1) for optimized
- **Worst Case:** O(n) for brute force dequeue, O(1) for optimized

Q115. Valid parentheses using stack

Question: Check if string containing only '()[]' has valid parentheses pairing.

Solution 1 – Brute Force Approach:

```
def valid_parentheses_stack_bf(s):
    """Check by removing valid pairs repeatedly."""
    while '()' in s or '[]' in s or '{}' in s:
        s = s.replace('()', '').replace('[]', '').replace('{}',
            '')
    return len(s) == 0
```

Logic: Repeatedly remove valid pairs until string empty or unchanged.

Inefficiencies: O(n^2) time - each replace scans entire string, done multiple times.

Solution 2 – Optimized Approach:

```
def valid_parentheses_stack_opt(s):
    """Check using stack."""
    stack = []
    mapping = {')': '(', ']': '[', '}': '{'}

    for char in s:
        if char in mapping.values():  # Opening bracket
            stack.append(char)
        elif char in mapping:  # Closing bracket
            if not stack or stack[-1] != mapping[char]:
                return False
            stack.pop()
        # Ignore other characters

    return len(stack) == 0
```

Optimization: Use stack to track opening brackets. When closing bracket found, check if matches top.

Justification: O(n) time vs O(n^2), single pass with stack.

Time Complexity:

- **Brute Force:** O(n^2) where n is string length
- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(n) for modified strings
- **Optimized:** O(n) for stack in worst case

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if first character is invalid closing
- **Average Case:** O(n^2) for brute force, O(n) for optimized
- **Worst Case:** O(n^2) for brute force, O(n) for optimized

Q116. Next greater element

Question: For each element in array, find next greater element to its right.

Solution 1 – Brute Force Approach:

```python
def next_greater_bf(arr):
    """Find next greater using nested loops."""
    n = len(arr)
    result = [-1] * n

    for i in range(n):
        for j in range(i + 1, n):
            if arr[j] > arr[i]:
                result[i] = arr[j]
                break

    return result
```

Logic: For each element, scan right to find first greater element.

Inefficiencies: O(n^2) time, scans right portion for each element.

Solution 2 – Optimized Approach:

```python
def next_greater_opt(arr):
    """Find next greater using stack (monotonic decreasing)."""
    n = len(arr)
    result = [-1] * n
    stack = []  # Store indices

    for i in range(n):
        # While stack not empty and current > stack top
        while stack and arr[i] > arr[stack[-1]]:
            idx = stack.pop()
            result[idx] = arr[i]

        stack.append(i)

    return result
```

Optimization: Use monotonic decreasing stack. Process elements, pop from stack when current ¿ stack top.

Justification: O(n) time vs O(n^2), each element pushed/popped at most once.

Time Complexity:

- **Brute Force:** O(n^2) where n is array length
- **Optimized:** O(n) where n is array length

Space Complexity:

- **Brute Force:** O(1) excluding result
- **Optimized:** O(n) for stack

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** O(n^2) for brute force, O(n) for optimized
- **Worst Case:** O(n^2) for brute force, O(n) for optimized

Q117. Maximum in sliding window

Question: Find maximum element in each sliding window of size K.

Solution 1 – Brute Force Approach:

```python
def max_sliding_window_bf(arr, k):
    """Find max in each window using nested loops."""
    n = len(arr)
    if n == 0 or k == 0:
        return []

    result = []
    for i in range(n - k + 1):
        window_max = max(arr[i:i+k])
        result.append(window_max)

    return result
```

Logic: For each window starting position, compute max of window using max().

Inefficiencies: O(n×k) time, recomputes max for overlapping windows.

Solution 2 – Optimized Approach:

```python
from collections import deque

def max_sliding_window_opt(arr, k):
    """Find max in sliding window using deque."""
    n = len(arr)
    if n == 0 or k == 0:
        return []

    result = []
    dq = deque()  # Store indices

    for i in range(n):
        # Remove indices outside current window
        if dq and dq[0] <= i - k:
            dq.popleft()

        # Remove smaller elements from back
        while dq and arr[dq[-1]] < arr[i]:
            dq.pop()

        dq.append(i)

        # Add to result when window is complete
        if i >= k - 1:
            result.append(arr[dq[0]])

    return result
```

Optimization: Use deque to maintain candidates for maximum in decreasing order.

Justification: O(n) time vs O(n×k), each element processed at most twice.

Time Complexity:

- **Brute Force:** O(n×k) where n is array length, k is window size
- **Optimized:** O(n) where n is array length

Space Complexity:

- **Brute Force:** O(1) excluding result
- **Optimized:** O(k) for deque

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** O(n×k) for brute force, O(n) for optimized
- **Worst Case:** O(n×k) for brute force, O(n) for optimized

Q118. Longest substring with K distinct characters

Question: Find length of longest substring with exactly K distinct characters.

Solution 1 – Brute Force Approach:

```python
def longest_k_distinct_bf(s, k):
    """Check all substrings for K distinct characters."""
    n = len(s)
    max_len = 0

    for i in range(n):
        for j in range(i, n):
            substring = s[i:j+1]
            distinct = len(set(substring))
            if distinct == k:
                max_len = max(max_len, len(substring))

    return max_len
```

Logic: Generate all substrings, count distinct characters using set, track max length.

Inefficiencies: $O(n^3)$ time - n^2 substrings, each set creation $O(k)$.

Solution 2 – Optimized Approach:

```python
def longest_k_distinct_opt(s, k):
    """Find longest substring with K distinct using sliding
       window."""
    from collections import defaultdict

    n = len(s)
    if n * k == 0:
        return 0

    left = right = 0
    char_count = defaultdict(int)
    max_len = 1

    while right < n:
        # Expand window
        char_count[s[right]] += 1
        right += 1

        # Shrink window if distinct > k
        while len(char_count) > k:
            char_count[s[left]] -= 1
            if char_count[s[left]] == 0:
                del char_count[s[left]]
            left += 1

        # Update max length (now distinct <= k)
        max_len = max(max_len, right - left)

    return max_len if len(char_count) == k else 0
```

Optimization: Sliding window with hash map tracking character counts. Expand right, shrink left when distinct ¿ k.

Justification: O(n) time vs O(n^3), single pass with two pointers.

Time Complexity:

- **Brute Force:** O(n^3) where n is string length
- **Optimized:** O(n) where n is string length

Space Complexity:

- **Brute Force:** O(n) for sets
- **Optimized:** O(k) for character map

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** O(n^3) for brute force, O(n) for optimized
- **Worst Case:** O(n^3) for brute force, O(n) for optimized

Q119. Two sum using hashmap

Question: Find two numbers in array that sum to target value.

Solution 1 – Brute Force Approach:

```python
def two_sum_bf(arr, target):
    """Find two sum using nested loops."""
    n = len(arr)

    for i in range(n):
        for j in range(i + 1, n):
            if arr[i] + arr[j] == target:
                return [i, j]

    return []
```

Logic: Check all pairs of numbers, return indices if sum equals target.

Inefficiencies: O(n^2) time, checks all n(n-1)/2 pairs.

Solution 2 – Optimized Approach:

```python
def two_sum_opt(arr, target):
    """Find two sum using hash map."""
    num_map = {}

    for i, num in enumerate(arr):
        complement = target - num

        if complement in num_map:
            return [num_map[complement], i]

        num_map[num] = i

    return []
```

Optimization: Use hash map to store numbers and indices. For each number, check if complement exists.

Justification: O(n) time vs O(n^2), one pass with O(1) lookups.

Time Complexity:

- **Brute Force:** O(n^2) where n is array length
- **Optimized:** O(n) where n is array length

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(n) for hash map

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if first two numbers sum to target
- **Average Case:** O(n^2) for brute force, O(n) for optimized
- **Worst Case:** O(n^2) for brute force, O(n) for optimized

Q120. Count subarrays with sum K

Question: Count number of contiguous subarrays with sum equal to K.

Solution 1 – Brute Force Approach:

```python
def subarray_sum_k_bf(arr, k):
    """Count subarrays with sum K using all subarrays."""
    n = len(arr)
    count = 0

    for i in range(n):
        for j in range(i, n):
            subarray_sum = sum(arr[i:j+1])
            if subarray_sum == k:
                count += 1

    return count
```

Logic: Generate all subarrays, compute sum for each, count if equals K.

Inefficiencies: O(n^3) time - n^2 subarrays, each sum calculation O(k).

Solution 2 – Optimized Approach:

```python
def subarray_sum_k_opt(arr, k):
    """Count subarrays with sum K using prefix sum and hash
       map."""
    count = 0
    prefix_sum = 0
    sum_count = {0: 1}  # sum 0 occurs once initially

    for num in arr:
        prefix_sum += num

        # If (prefix_sum - k) exists in map, we found subarrays
        if (prefix_sum - k) in sum_count:
            count += sum_count[prefix_sum - k]

        # Update count of current prefix sum
        sum_count[prefix_sum] = sum_count.get(prefix_sum, 0) + 1

    return count
```

Optimization: Use prefix sum and hash map. Track how many times each prefix sum occurs.

Justification: O(n) time vs O(n^3), single pass with hash map.

Time Complexity:

- **Brute Force:** O(n^3) where n is array length
- **Optimized:** O(n) where n is array length

Space Complexity:

- **Brute Force:** O(1)

- **Optimized:** O(n) for hash map

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** O(n^3) for brute force, O(n) for optimized
- **Worst Case:** O(n^3) for brute force, O(n) for optimized

Q121. Kth smallest element using heap

Question: Find Kth smallest element in array using heap data structure.

Solution 1 – Brute Force Approach:

```python
def kth_smallest_bf(arr, k):
    """Find Kth smallest by sorting."""
    if k > len(arr):
        return None
    arr.sort()
    return arr[k-1]
```

Logic: Sort array, return element at position k-1.

Inefficiencies: O(n log n) time for full sort when we only need Kth smallest.

Solution 2 – Optimized Approach:

```python
import heapq

def kth_smallest_opt(arr, k):
    """Find Kth smallest using max-heap."""
    if k > len(arr) or k <= 0:
        return None

    # Create max-heap of first k elements (negate for max-heap)
    heap = [-x for x in arr[:k]]
    heapq.heapify(heap)

    # Process remaining elements
    for num in arr[k:]:
        if num < -heap[0]:  # Smaller than largest in heap
            heapq.heapreplace(heap, -num)

    # Root is Kth smallest (negate back)
    return -heap[0]

    # Alternative: using nsmallest
    # return heapq.nsmallest(k, arr)[-1]
```

Optimization: Use max-heap of size K. Maintain K smallest elements, root is Kth smallest.

Justification: O(n log k) time vs O(n log n), better when k ¡¡ n.

Time Complexity:

- **Brute Force:** O(n log n) where n is array length
- **Optimized:** O(n log k) where n is array length, k is parameter

Space Complexity:

- **Brute Force:** O(1) if sort in-place
- **Optimized:** O(k) for heap

Best / Average / Worst Case Analysis:

- **Best Case:** O(n log n) for brute force, O(n) for optimized when k=1
- **Average Case:** O(n log n) for brute force, O(n log k) for optimized
- **Worst Case:** O(n log n) for brute force, O(n log k) for optimized

Q122. Merge K sorted lists using heap

Question: Merge K sorted linked lists into one sorted list using heap.

Solution 1 – Brute Force Approach:

```
def merge_k_lists_bf(lists):
    """Merge by repeatedly merging 2 lists."""
    if not lists:
        return []

    result = lists[0]
    for lst in lists[1:]:
        result = merge_two_lists(result, lst)

    return result

def merge_two_lists(l1, l2):
    """Merge two sorted lists."""
    dummy = ListNode(0)
    current = dummy

    while l1 and l2:
        if l1.val <= l2.val:
            current.next = l1
            l1 = l1.next
        else:
            current.next = l2
            l2 = l2.next
        current = current.next

    current.next = l1 if l1 else l2
    return dummy.next
```

Logic: Repeatedly merge lists two at a time.

Inefficiencies: O(kN) time where k is number of lists, N is total elements.

Solution 2 – Optimized Approach:

```
import heapq

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def merge_k_lists_opt(lists):
    """Merge K sorted lists using min-heap."""
    # Create min-heap with first element of each list
    heap = []
    for i, lst in enumerate(lists):
        if lst:
            heapq.heappush(heap, (lst.val, i, lst))

    dummy = ListNode(0)
```

```
    current = dummy

    while heap:
        val, idx, node = heapq.heappop(heap)
        current.next = node
        current = current.next

        # Add next element from same list
        if node.next:
            heapq.heappush(heap, (node.next.val, idx, node.next))

    return dummy.next
```

Optimization: Use min-heap to always get smallest element among all lists. O(N log k) time.

Justification: O(N log k) vs O(kN), much better when k is large.

Time Complexity:

- **Brute Force:** O(kN) where k is number of lists, N total elements
- **Optimized:** O(N log k) where N total elements, k number of lists

Space Complexity:

- **Brute Force:** O(1) additional
- **Optimized:** O(k) for heap

Best / Average / Worst Case Analysis:

- **Best Case:** O(N log k) for optimized
- **Average Case:** O(kN) for brute force, O(N log k) for optimized
- **Worst Case:** O(kN) for brute force, O(N log k) for optimized

Q123. Insert in sorted list using bisect

Question: Insert element into sorted list while maintaining sorted order using bisect module.

Solution 1 – Brute Force Approach:

```
def insert_sorted_bf(lst, x):
    """Insert into sorted list by finding position manually."""
    if not lst:
        return [x]

    # Find insertion position
    i = 0
    while i < len(lst) and lst[i] < x:
        i += 1

    # Insert at position i
    lst.insert(i, x)
    return lst
```

Logic: Linear scan to find position where element should be inserted.

Inefficiencies: O(n) time for finding position + O(n) for insert shifting.

Solution 2 – Optimized Approach:

```
import bisect

def insert_sorted_opt(lst, x):
    """Insert into sorted list using bisect."""
    # Find insertion position using binary search
    pos = bisect.bisect_left(lst, x)

    # Insert at position
    lst.insert(pos, x)
    return lst

# Alternative: using bisect.insort
def insert_sorted_bisect(lst, x):
    """Insert using bisect.insort (one function call)."""
    bisect.insort(lst, x)
    return lst
```

Optimization: Use bisect.bisect_left for O(log n) position finding. Still O(n) for insert.

Justification: O(log n) for search vs O(n), though insert remains O(n).

Time Complexity:

- **Brute Force:** O(n) where n is list length
- **Optimized:** O(n) where n is list length (insert dominates)

Space Complexity:

- **Brute Force:** O(1) additional
- **Optimized:** O(1) additional

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if inserting at beginning or end
- **Average Case:** O(n) for both (insert shifting)
- **Worst Case:** O(n) for both (insert at beginning)

Q124. Count inversions in array

Question: Count number of inversions (i ¡ j but arr[i] ¿ arr[j]) in array.

Solution 1 – Brute Force Approach:

```
def count_inversions_bf(arr):
    """Count inversions using nested loops."""
    n = len(arr)
    count = 0

    for i in range(n):
        for j in range(i + 1, n):
            if arr[i] > arr[j]:
                count += 1

    return count
```

Logic: For each element, count how many later elements are smaller.

Inefficiencies: O(n^2) time, checks all n(n-1)/2 pairs.

Solution 2 – Optimized Approach:

```
def count_inversions_opt(arr):
    """Count inversions using merge sort modification."""
    def merge_sort_count(arr):
        if len(arr) <= 1:
            return arr, 0

        mid = len(arr) // 2
        left, inv_left = merge_sort_count(arr[:mid])
        right, inv_right = merge_sort_count(arr[mid:])

        merged, inv_merge = merge_count(left, right)

        return merged, inv_left + inv_right + inv_merge

    def merge_count(left, right):
        merged = []
        i = j = 0
        inv_count = 0

        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                merged.append(left[i])
                i += 1
            else:
                merged.append(right[j])
                inv_count += len(left) - i  # All remaining in
                   left are > right[j]
                j += 1

        merged.extend(left[i:])
        merged.extend(right[j:])

```

```
        return merged, inv_count

    _, count = merge_sort_count(arr)
    return count
```

Optimization: Modified merge sort. During merge, count inversions when element from right ¡ element from left.

Justification: O(n log n) time vs O(n^2), divide and conquer approach.

Time Complexity:

- **Brute Force:** O(n^2) where n is array length
- **Optimized:** O(n log n) where n is array length

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(n) for merge sort

Best / Average / Worst Case Analysis:

- **Best Case:** O(n log n) for optimized
- **Average Case:** O(n^2) for brute force, O(n log n) for optimized
- **Worst Case:** O(n^2) for brute force, O(n log n) for optimized

Q125. Median of two sorted arrays

Question: Find median of two sorted arrays of possibly different sizes.

Solution 1 – Brute Force Approach:

```python
def median_sorted_arrays_bf(nums1, nums2):
    """Find median by merging arrays."""
    merged = []
    i = j = 0

    # Merge two sorted arrays
    while i < len(nums1) and j < len(nums2):
        if nums1[i] <= nums2[j]:
            merged.append(nums1[i])
            i += 1
        else:
            merged.append(nums2[j])
            j += 1

    merged.extend(nums1[i:])
    merged.extend(nums2[j:])

    # Find median
    n = len(merged)
    if n % 2 == 0:
        return (merged[n//2 - 1] + merged[n//2]) / 2
    else:
        return merged[n//2]
```

Logic: Merge two arrays into one sorted array, find median of merged array.

Inefficiencies: O(m+n) time and space, when O(log(m+n)) possible.

Solution 2 – Optimized Approach:

```python
def median_sorted_arrays_opt(nums1, nums2):
    """Find median using binary search - O(log(min(m,n)))."""
    # Ensure nums1 is shorter
    if len(nums1) > len(nums2):
        nums1, nums2 = nums2, nums1

    m, n = len(nums1), len(nums2)
    low, high = 0, m

    while low <= high:
        # Partition nums1
        i = (low + high) // 2
        # Partition nums2 (j partitions total left half)
        j = (m + n + 1) // 2 - i

        # Edge cases
        nums1_left = nums1[i-1] if i > 0 else float('-inf')
        nums1_right = nums1[i] if i < m else float('inf')
        nums2_left = nums2[j-1] if j > 0 else float('-inf')
        nums2_right = nums2[j] if j < n else float('inf')
```

```
21
22          # Check if partition is correct
23          if nums1_left <= nums2_right and nums2_left <=
                nums1_right:
24              # Found correct partition
25              if (m + n) % 2 == 0:
26                  return (max(nums1_left, nums2_left) +
27                          min(nums1_right, nums2_right)) / 2
28              else:
29                  return max(nums1_left, nums2_left)
30          elif nums1_left > nums2_right:
31              # Move partition left in nums1
32              high = i - 1
33          else:
34              # Move partition right in nums1
35              low = i + 1
36
37      raise ValueError("Input arrays not sorted")
```

Optimization: Binary search on shorter array. Find partition where all left elements all right elements.

Justification: O(log(min(m,n))) time vs O(m+n), no extra space.

Time Complexity:

- **Brute Force:** O(m+n) where m,n are array lengths
- **Optimized:** O(log(min(m,n))) where m,n are array lengths

Space Complexity:

- **Brute Force:** O(m+n) for merged array
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(m+n) for brute force, O(1) for optimized if arrays already partitioned
- **Average Case:** O(m+n) for brute force, O(log(min(m,n))) for optimized
- **Worst Case:** O(m+n) for brute force, O(log(min(m,n))) for optimized

Q126. Priority queue using heapq

Question: Implement priority queue (min-heap) using heapq module.

Solution 1 – Brute Force Approach:

```
class PriorityQueueBF:
    """Priority queue using list and sorting."""
    def __init__(self):
        self.items = []

    def push(self, item, priority):
        """Add item with priority."""
        self.items.append((priority, item))
        # Sort by priority (ascending)
        self.items.sort()

    def pop(self):
        """Remove and return highest priority item."""
        if not self.is_empty():
            return self.items.pop(0)[1]
        raise IndexError("pop from empty priority queue")

    def is_empty(self):
        return len(self.items) == 0

    def size(self):
        return len(self.items)
```

Logic: Store (priority, item) pairs, sort list after each push.

Inefficiencies: O(n log n) for each push due to sorting.

Solution 2 – Optimized Approach:

```
import heapq

class PriorityQueueOpt:
    """Priority queue using heapq."""
    def __init__(self):
        self.heap = []

    def push(self, item, priority):
        """Add item with priority - O(log n)."""
        heapq.heappush(self.heap, (priority, item))

    def pop(self):
        """Remove and return highest priority item - O(log n)."""
        if not self.is_empty():
            return heapq.heappop(self.heap)[1]
        raise IndexError("pop from empty priority queue")

    def peek(self):
        """Return highest priority item without removing -
           O(1)."""
        if not self.is_empty():
```

```
            return self.heap[0][1]
        raise IndexError("peek from empty priority queue")

    def is_empty(self):
        return len(self.heap) == 0

    def size(self):
        return len(self.heap)

    def clear(self):
        """Clear priority queue."""
        self.heap.clear()
```

Optimization: Use heapq module which implements binary heap with O(log n) push/pop.
Justification: O(log n) operations vs O(n log n). heapq is C-optimized.
Time Complexity:

- **Brute Force:** O(n log n) push, O(n) pop
- **Optimized:** O(log n) push and pop

Space Complexity:

- **Brute Force:** O(n) where n is queue size
- **Optimized:** O(n) where n is queue size

Best / Average / Worst Case Analysis:

- **Best Case:** O(log n) for optimized operations
- **Average Case:** O(n log n) for brute force push, O(log n) for optimized
- **Worst Case:** O(n log n) for brute force push, O(log n) for optimized

Q127. Detect cycle in directed graph using DFS

Question: Detect if directed graph contains cycle using depth-first search.

Solution 1 – Brute Force Approach:

```python
def has_cycle_directed_bf(graph):
    """Detect cycle by checking all paths."""
    def dfs_visit_all(start, target, visited):
        """DFS to check if path exists from start to target."""
        if start == target:
            return True

        visited.add(start)
        for neighbor in graph.get(start, []):
            if neighbor not in visited:
                if dfs_visit_all(neighbor, target, visited):
                    return True

        visited.remove(start)
        return False

    # For each edge u->v, check if path v->u exists
    for u in graph:
        for v in graph.get(u, []):
            if dfs_visit_all(v, u, set()):
                return True

    return False
```

Logic: For each edge u→v, check if path exists from v back to u.

Inefficiencies: O(V×(V+E)) worst case, checks all pairs.

Solution 2 – Optimized Approach:

```python
def has_cycle_directed_opt(graph):
    """Detect cycle using DFS with three colors."""
    # Colors: 0=unvisited, 1=visiting, 2=visited
    color = {}

    def dfs(node):
        if node not in color:
            color[node] = 0  # unvisited

        if color[node] == 1:  # visiting -> cycle
            return True
        if color[node] == 2:  # already processed
            return False

        # Mark as visiting
        color[node] = 1

        # Visit all neighbors
        for neighbor in graph.get(node, []):
            if dfs(neighbor):
```

```
                return True

        # Mark as visited
        color[node] = 2
        return False

    # Check all nodes
    for node in graph:
        if dfs(node):
            return True

    return False
```

Optimization: Three-color DFS: white=unvisited, gray=visiting, black=visited. Cycle if encounter gray node.

Justification: O(V+E) time vs O(V×(V+E)), standard algorithm for cycle detection.

Time Complexity:

- **Brute Force:** O(V×(V+E)) where V vertices, E edges
- **Optimized:** O(V+E) where V vertices, E edges

Space Complexity:

- **Brute Force:** O(V) recursion stack
- **Optimized:** O(V) for color dict + O(V) recursion stack

Best / Average / Worst Case Analysis:

- **Best Case:** O(V+E) for optimized
- **Average Case:** O(V×(V+E)) for brute force, O(V+E) for optimized
- **Worst Case:** O(V×(V+E)) for brute force, O(V+E) for optimized

Q128. BFS traversal of graph

Question: Perform breadth-first search traversal of graph starting from given node.

Solution 1 – Brute Force Approach:

```python
def bfs_traversal_bf(graph, start):
    """BFS using list as queue (inefficient dequeue)."""
    if start not in graph:
        return []

    visited = set()
    result = []
    queue = [start]

    while queue:
        # pop(0) is O(n)
        node = queue.pop(0)

        if node not in visited:
            visited.add(node)
            result.append(node)

            # Add unvisited neighbors
            for neighbor in graph.get(node, []):
                if neighbor not in visited:
                    queue.append(neighbor)

    return result
```

Logic: Use FIFO queue, process nodes level by level. Mark visited when processing.

Inefficiencies: pop(0) is O(n) for list, making BFS $O(V^2)$ instead of O(V+E).

Solution 2 – Optimized Approach:

```python
from collections import deque

def bfs_traversal_opt(graph, start):
    """BFS using deque for efficient queue operations."""
    if start not in graph:
        return []

    visited = set([start])
    result = []
    queue = deque([start])

    while queue:
        # popleft() is O(1)
        node = queue.popleft()
        result.append(node)

        # Process neighbors
        for neighbor in graph.get(node, []):
            if neighbor not in visited:
                visited.add(neighbor)
```

```
                queue.append(neighbor)

    return result

# Alternative: BFS with level information
def bfs_levels(graph, start):
    """BFS returning nodes by level."""
    if start not in graph:
        return []

    visited = set([start])
    result = []
    queue = deque([start])

    while queue:
        level_size = len(queue)
        level = []

        for _ in range(level_size):
            node = queue.popleft()
            level.append(node)

            for neighbor in graph.get(node, []):
                if neighbor not in visited:
                    visited.add(neighbor)
                    queue.append(neighbor)

        result.append(level)

    return result
```

Optimization: Use deque for O(1) popleft operations. Mark visited when enqueueing to avoid duplicates.

Justification: O(V+E) time vs $O(V^2)$, proper BFS implementation.

Time Complexity:

- **Brute Force:** $O(V^2)$ where V vertices (due to pop(0))
- **Optimized:** O(V+E) where V vertices, E edges

Space Complexity:

- **Brute Force:** O(V) for queue and visited
- **Optimized:** O(V) for queue and visited

Best / Average / Worst Case Analysis:

- **Best Case:** O(V+E) for optimized
- **Average Case:** $O(V^2)$ for brute force, O(V+E) for optimized
- **Worst Case:** $O(V^2)$ for brute force, O(V+E) for optimized

Q129. DFS traversal of graph

Question: Perform depth-first search traversal of graph starting from given node.

Solution 1 – Brute Force Approach:

```python
def dfs_traversal_bf(graph, start):
    """DFS using recursion without visited tracking
       optimization."""
    visited = set()
    result = []

    def dfs(node):
        if node in visited:
            return

        visited.add(node)
        result.append(node)

        # Visit all neighbors
        for neighbor in graph.get(node, []):
            dfs(neighbor)

    dfs(start)
    return result
```

Logic: Recursive DFS: mark node visited, add to result, recursively visit all unvisited neighbors.

Inefficiencies: Standard DFS is already optimal. Stack overflow possible for deep graphs.

Solution 2 – Optimized Approach:

```python
def dfs_traversal_opt(graph, start):
    """DFS using iterative approach with stack."""
    if start not in graph:
        return []

    visited = set([start])
    result = []
    stack = [start]

    while stack:
        node = stack.pop()
        result.append(node)

        # Add unvisited neighbors to stack
        # Reverse to maintain same order as recursive
        for neighbor in reversed(graph.get(node, [])):
            if neighbor not in visited:
                visited.add(neighbor)
                stack.append(neighbor)

    return result

```

```python
# Alternative: DFS with pre/post order
def dfs_pre_post(graph, start):
    """DFS returning pre-order and post-order traversals."""
    visited = set()
    pre_order, post_order = [], []

    def dfs(node):
        visited.add(node)
        pre_order.append(node)  # Pre-order

        for neighbor in graph.get(node, []):
            if neighbor not in visited:
                dfs(neighbor)

        post_order.append(node)  # Post-order

    dfs(start)
    return pre_order, post_order
```

Optimization: Iterative DFS using stack avoids recursion limit. Can return pre/post order.

Justification: Avoids recursion stack overflow for deep graphs. More control over traversal.

Time Complexity:

- **Brute Force:** O(V+E) where V vertices, E edges
- **Optimized:** O(V+E) where V vertices, E edges

Space Complexity:

- **Brute Force:** O(V) recursion stack + O(V) visited
- **Optimized:** O(V) stack + O(V) visited

Best / Average / Worst Case Analysis:

- **Best Case:** O(V+E) for both
- **Average Case:** O(V+E) for both
- **Worst Case:** O(V+E) for both

Q130. Shortest path in unweighted graph using BFS

Question: Find shortest path from source to target in unweighted graph using BFS.

Solution 1 – Brute Force Approach:

```python
def shortest_path_bf(graph, start, end):
    """Find shortest path using BFS exploring all paths."""
    if start == end:
        return [start]

    # BFS to find shortest path length
    queue = [(start, [start])]
    visited = set([start])

    while queue:
        node, path = queue.pop(0)  # O(n)

        for neighbor in graph.get(node, []):
            if neighbor == end:
                return path + [neighbor]

            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))

    return []  # No path
```

Logic: BFS storing paths. When target found, return path.

Inefficiencies: pop(0) is O(n). Stores full paths in queue O(V×V) memory.

Solution 2 – Optimized Approach:

```python
from collections import deque

def shortest_path_opt(graph, start, end):
    """Find shortest path using BFS with parent tracking."""
    if start == end:
        return [start]

    visited = set([start])
    queue = deque([start])
    parent = {start: None}

    # BFS to find target
    while queue:
        node = queue.popleft()

        if node == end:
            # Reconstruct path
            path = []
            while node is not None:
                path.append(node)
                node = parent[node]
            return path[::-1]
```

```
23
24          for neighbor in graph.get(node, []):
25              if neighbor not in visited:
26                  visited.add(neighbor)
27                  parent[neighbor] = node
28                  queue.append(neighbor)
29
30      return []  # No path
```

Optimization: Use deque for O(1) popleft. Track parent nodes to reconstruct path.
Justification: O(V+E) time vs O(V^2). O(V) memory vs O(V^2).
Time Complexity:

- **Brute Force:** O(V^2) where V vertices (due to pop(0))
- **Optimized:** O(V+E) where V vertices, E edges

Space Complexity:

- **Brute Force:** O(V^2) for storing all paths
- **Optimized:** O(V) for parent dict and visited

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if start == end
- **Average Case:** O(V^2) for brute force, O(V+E) for optimized
- **Worst Case:** O(V^2) for brute force, O(V+E) for optimized

Q131. Dijkstra's algorithm for shortest path

Question: Find shortest path in weighted graph with non-negative weights using Dijkstra's algorithm.

Solution 1 – Brute Force Approach:

```
def dijkstra_bf(graph, start):
    """Dijkstra without priority queue - O(V  )."""
    n = len(graph)
    dist = {node: float('inf') for node in graph}
    dist[start] = 0
    visited = set()

    for _ in range(n):
        # Find unvisited node with minimum distance
        min_node = None
        min_dist = float('inf')

        for node in graph:
            if node not in visited and dist[node] < min_dist:
                min_dist = dist[node]
                min_node = node

        if min_node is None:
            break

        visited.add(min_node)

        # Update distances to neighbors
        for neighbor, weight in graph.get(min_node, {}).items():
            new_dist = dist[min_node] + weight
            if new_dist < dist[neighbor]:
                dist[neighbor] = new_dist

    return dist
```

Logic: Maintain distances. Each iteration find unvisited node with min distance, update neighbors.

Inefficiencies: O(V^2) time due to linear scan for min node each iteration.

Solution 2 – Optimized Approach:

```
import heapq

def dijkstra_opt(graph, start):
    """Dijkstra with priority queue - O((V+E) log V)."""
    dist = {node: float('inf') for node in graph}
    dist[start] = 0

    # Priority queue: (distance, node)
    pq = [(0, start)]

    while pq:
        current_dist, node = heapq.heappop(pq)
```

```
13
14          # Skip if we found a better path already
15          if current_dist > dist[node]:
16              continue
17
18          # Update neighbors
19          for neighbor, weight in graph.get(node, {}).items():
20              new_dist = current_dist + weight
21
22              if new_dist < dist[neighbor]:
23                  dist[neighbor] = new_dist
24                  heapq.heappush(pq, (new_dist, neighbor))
25
26      return dist
27
28  # With path reconstruction
29  def dijkstra_path(graph, start, end):
30      """Dijkstra returning shortest path."""
31      dist = {node: float('inf') for node in graph}
32      dist[start] = 0
33      parent = {start: None}
34      pq = [(0, start)]
35
36      while pq:
37          current_dist, node = heapq.heappop(pq)
38
39          if current_dist > dist[node]:
40              continue
41
42          if node == end:
43              # Reconstruct path
44              path = []
45              while node is not None:
46                  path.append(node)
47                  node = parent[node]
48              return path[::-1], dist[end]
49
50          for neighbor, weight in graph.get(node, {}).items():
51              new_dist = current_dist + weight
52
53              if new_dist < dist[neighbor]:
54                  dist[neighbor] = new_dist
55                  parent[neighbor] = node
56                  heapq.heappush(pq, (new_dist, neighbor))
57
58      return [], float('inf')  # No path
```

Optimization: Use min-heap (priority queue) to get node with minimum distance efficiently.

Justification: O((V+E) log V) time vs $O(V^2)$, much faster for sparse graphs.

Time Complexity:

- **Brute Force:** $O(V^2)$ where V vertices
- **Optimized:** O((V+E) log V) where V vertices, E edges

Space Complexity:

- **Brute Force:** O(V) for dist and visited
- **Optimized:** O(V) for dist + O(V) for heap

Best / Average / Worst Case Analysis:

- **Best Case:** $O(V^2)$ for brute force, O((V+E) log V) for optimized
- **Average Case:** $O(V^2)$ for brute force, O((V+E) log V) for optimized
- **Worst Case:** $O(V^2)$ for brute force, O((V+E) log V) for optimized

Q132. Reverse a singly linked list

Question: Reverse a singly linked list in-place.

Solution 1 – Brute Force Approach:

```python
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def reverse_list_bf(head):
    """Reverse list by storing values and recreating."""
    if not head:
        return None

    # Store values in list
    values = []
    current = head
    while current:
        values.append(current.val)
        current = current.next

    # Create new list in reverse
    dummy = ListNode(0)
    current = dummy
    for val in reversed(values):
        current.next = ListNode(val)
        current = current.next

    return dummy.next
```

Logic: Traverse list to store values, create new list with values in reverse order.

Inefficiencies: O(n) extra space, creates new nodes instead of reversing in-place.

Solution 2 – Optimized Approach:

```python
def reverse_list_opt(head):
    """Reverse list in-place using three pointers."""
    prev = None
    current = head

    while current:
        # Store next node
        next_node = current.next
        # Reverse link
        current.next = prev
        # Move pointers
        prev = current
        current = next_node

    return prev  # New head

# Recursive version
def reverse_list_recursive(head):
```

```
    """Reverse list recursively."""
    if not head or not head.next:
        return head

    # Reverse rest of list
    new_head = reverse_list_recursive(head.next)

    # Reverse current node
    head.next.next = head
    head.next = None

    return new_head
```

Optimization: In-place reversal using three pointers (prev, current, next). O(1) extra space.

Justification: O(1) space vs O(n), modifies existing list instead of creating new one.

Time Complexity:

- **Brute Force:** O(n) where n is list length
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(n) for values list + new nodes
- **Optimized:** O(1) iterative, O(n) recursive stack

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) for empty or single-node list
- **Average Case:** O(n) for both
- **Worst Case:** O(n) for both

Q133. Detect cycle in linked list

Question: Detect if singly linked list contains cycle using Floyd's cycle detection.

Solution 1 – Brute Force Approach:

```
def has_cycle_bf(head):
    """Detect cycle by storing visited nodes."""
    visited = set()
    current = head

    while current:
        if current in visited:
            return True
        visited.add(current)
        current = current.next

    return False
```

Logic: Traverse list, store nodes in set, check if node visited before.

Inefficiencies: O(n) extra space for set.

Solution 2 – Optimized Approach:

```
def has_cycle_opt(head):
    """Detect cycle using Floyd's Tortoise and Hare algorithm."""
    if not head or not head.next:
        return False

    slow = head
    fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

        if slow == fast:
            return True

    return False

# Find start of cycle
def detect_cycle_start(head):
    """Find start node of cycle if exists."""
    if not head or not head.next:
        return None

    # Phase 1: Find meeting point
    slow = fast = head
    has_cycle = False

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

```

```
        if slow == fast:
            has_cycle = True
            break

    if not has_cycle:
        return None

    # Phase 2: Find start of cycle
    slow = head
    while slow != fast:
        slow = slow.next
        fast = fast.next

    return slow
```

Optimization: Floyd's algorithm (tortoise and hare) with O(1) space. Two pointers moving at different speeds.

Justification: O(1) space vs O(n), mathematical proof shows they meet if cycle exists.

Time Complexity:

- **Brute Force:** O(n) where n is list length
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(n) for visited set
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if no cycle and short list
- **Average Case:** O(n) for both
- **Worst Case:** O(n) for both

Q134. Middle element of linked list

Question: Find middle element of singly linked list (if even length, return second middle).

Solution 1 – Brute Force Approach:

```
def find_middle_bf(head):
    """Find middle by counting nodes first."""
    if not head:
        return None

    # Count nodes
    count = 0
    current = head
    while current:
        count += 1
        current = current.next

    # Find middle node
    middle_idx = count // 2
    current = head
    for _ in range(middle_idx):
        current = current.next

    return current
```

Logic: First pass to count nodes, second pass to get to middle node.

Inefficiencies: Two passes through list O(2n).

Solution 2 – Optimized Approach:

```
def find_middle_opt(head):
    """Find middle using fast and slow pointers."""
    if not head:
        return None

    slow = fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    return slow

# For even length, return first middle
def find_middle_first(head):
    """Find first middle for even length lists."""
    if not head:
        return None

    slow = head
    fast = head

    while fast.next and fast.next.next:
```

```
24          slow = slow.next
25          fast = fast.next.next
26
27      return slow
```

Optimization: Fast and slow pointers. Fast moves 2 steps, slow moves 1 step. When fast reaches end, slow at middle.

Justification: One pass O(n) vs two passes O(2n).

Time Complexity:

- **Brute Force:** O(2n) = O(n) where n is list length
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) for empty or single-node list
- **Average Case:** O(n) for both
- **Worst Case:** O(n) for both

Q135. Merge two sorted linked lists

Question: Merge two sorted linked lists into one sorted list.

Solution 1 – Brute Force Approach:

```python
def merge_two_lists_bf(l1, l2):
    """Merge by creating new list from values."""
    values = []

    # Collect all values
    current = l1
    while current:
        values.append(current.val)
        current = current.next

    current = l2
    while current:
        values.append(current.val)
        current = current.next

    # Sort values
    values.sort()

    # Create new list
    dummy = ListNode(0)
    current = dummy
    for val in values:
        current.next = ListNode(val)
        current = current.next

    return dummy.next
```

Logic: Collect all values from both lists, sort them, create new list.

Inefficiencies: O(n log n) time due to sorting, O(n) extra space.

Solution 2 – Optimized Approach:

```python
def merge_two_lists_opt(l1, l2):
    """Merge in-place using pointer manipulation."""
    dummy = ListNode(0)
    current = dummy

    while l1 and l2:
        if l1.val <= l2.val:
            current.next = l1
            l1 = l1.next
        else:
            current.next = l2
            l2 = l2.next
        current = current.next

    # Attach remaining nodes
    current.next = l1 if l1 else l2

```

```
    return dummy.next

# Recursive version
def merge_two_lists_recursive(l1, l2):
    """Merge recursively."""
    if not l1:
        return l2
    if not l2:
        return l1

    if l1.val <= l2.val:
        l1.next = merge_two_lists_recursive(l1.next, l2)
        return l1
    else:
        l2.next = merge_two_lists_recursive(l1, l2.next)
        return l2
```

Optimization: Merge in-place by comparing nodes and linking appropriately. O(n) time, O(1) space.

Justification: O(n) time vs O(n log n), O(1) space vs O(n).

Time Complexity:

- **Brute Force:** O(n log n) where n total nodes
- **Optimized:** O(n) where n total nodes

Space Complexity:

- **Brute Force:** O(n) for values list + new nodes
- **Optimized:** O(1) iterative, O(n) recursive stack

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for optimized
- **Average Case:** O(n log n) for brute force, O(n) for optimized
- **Worst Case:** O(n log n) for brute force, O(n) for optimized

Q136. Remove Nth node from end of linked list

Question: Remove Nth node from end of singly linked list (1-indexed from end).

Solution 1 – Brute Force Approach:

```python
def remove_nth_from_end_bf(head, n):
    """Remove Nth from end by counting nodes."""
    if not head:
        return None

    # Count nodes
    count = 0
    current = head
    while current:
        count += 1
        current = current.next

    # Find position from start
    pos_from_start = count - n

    # If removing head
    if pos_from_start == 0:
        return head.next

    # Find node before target
    current = head
    for _ in range(pos_from_start - 1):
        current = current.next

    # Remove node
    if current.next:
        current.next = current.next.next

    return head
```

Logic: First pass to count nodes, calculate position from start, second pass to remove.

Inefficiencies: Two passes through list.

Solution 2 – Optimized Approach:

```python
def remove_nth_from_end_opt(head, n):
    """Remove Nth from end using two pointers in one pass."""
    dummy = ListNode(0)
    dummy.next = head

    fast = slow = dummy

    # Move fast n+1 steps ahead
    for _ in range(n + 1):
        fast = fast.next

    # Move both until fast reaches end
    while fast:
        fast = fast.next
```

```
        slow = slow.next

    # Remove nth node
    slow.next = slow.next.next

    return dummy.next
```

Optimization: Two pointers with n+1 gap. When fast reaches end, slow is before node to remove.

Justification: One pass O(n) vs two passes O(2n).

Time Complexity:

- **Brute Force:** O(2n) = O(n) where n is list length
- **Optimized:** O(n) where n is list length

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if removing head and n=length
- **Average Case:** O(n) for both
- **Worst Case:** O(n) for both

Q137. Implement LRU Cache

Question: Implement Least Recently Used (LRU) cache with O(1) get and put operations.

Solution 1 – Brute Force Approach:

```python
class LRUCacheBF:
    """LRU cache using list to track usage (inefficient)."""
    def __init__(self, capacity):
        self.capacity = capacity
        self.cache = {}  # key -> value
        self.usage = []  # track usage order

    def get(self, key):
        if key not in self.cache:
            return -1

        # Update usage: move to end
        self.usage.remove(key)  # O(n)
        self.usage.append(key)

        return self.cache[key]

    def put(self, key, value):
        if key in self.cache:
            # Update value and usage
            self.cache[key] = value
            self.usage.remove(key)  # O(n)
            self.usage.append(key)
        else:
            # Check capacity
            if len(self.cache) >= self.capacity:
                # Remove LRU
                lru_key = self.usage.pop(0)  # O(n)
                del self.cache[lru_key]

            # Add new key
            self.cache[key] = value
            self.usage.append(key)
```

Logic: Use dict for cache, list to track usage order. On access, move key to end of list.

Inefficiencies: remove() and pop(0) are O(n) operations, making get/put O(n).

Solution 2 – Optimized Approach:

```python
class ListNode:
    def __init__(self, key=0, val=0, prev=None, next=None):
        self.key = key
        self.val = val
        self.prev = prev
        self.next = next

class LRUCacheOpt:
    """LRU cache using hash map + doubly linked list."""
```

```
    def __init__(self, capacity):
        self.capacity = capacity
        self.cache = {}  # key -> node
        # Dummy head and tail
        self.head = ListNode()
        self.tail = ListNode()
        self.head.next = self.tail
        self.tail.prev = self.head

    def _add_node(self, node):
        """Add node right after head."""
        node.prev = self.head
        node.next = self.head.next

        self.head.next.prev = node
        self.head.next = node

    def _remove_node(self, node):
        """Remove node from linked list."""
        prev_node = node.prev
        next_node = node.next

        prev_node.next = next_node
        next_node.prev = prev_node

    def _move_to_head(self, node):
        """Move node to head (most recently used)."""
        self._remove_node(node)
        self._add_node(node)

    def _pop_tail(self):
        """Pop tail node (least recently used)."""
        node = self.tail.prev
        self._remove_node(node)
        return node

    def get(self, key):
        if key not in self.cache:
            return -1

        node = self.cache[key]
        self._move_to_head(node)
        return node.val

    def put(self, key, value):
        if key in self.cache:
            node = self.cache[key]
            node.val = value
            self._move_to_head(node)
        else:
            # Create new node
```

```
            node = ListNode(key, value)
            self.cache[key] = node
            self._add_node(node)

            # Check capacity
            if len(self.cache) > self.capacity:
                # Remove LRU
                tail = self._pop_tail()
                del self.cache[tail.key]
```

Optimization: Use hash map + doubly linked list. Head = most recent, tail = least recent.

Justification: O(1) get/put operations vs O(n). Linked list operations O(1).

Time Complexity:

- **Brute Force:** O(n) for get and put
- **Optimized:** O(1) for get and put

Space Complexity:

- **Brute Force:** O(capacity)
- **Optimized:** O(capacity)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) for optimized
- **Average Case:** O(n) for brute force, O(1) for optimized
- **Worst Case:** O(n) for brute force, O(1) for optimized

Q138. XOR of all elements in array

Question: Compute XOR of all elements in array.

Solution 1 – Brute Force Approach:

```python
def xor_all_bf(arr):
    """Compute XOR using loop."""
    result = 0
    for num in arr:
        result ^= num
    return result
```

Logic: Initialize result to 0, XOR each element sequentially.

Inefficiencies: Already optimal O(n) time, O(1) space.

Solution 2 – Optimized Approach:

```python
def xor_all_opt(arr):
    """Compute XOR using reduce or property optimizations."""
    # Method 1: Using functools.reduce
    from functools import reduce
    return reduce(lambda x, y: x ^ y, arr, 0)

    # Method 2: For special cases
    # If array is range 1..n: O(1) formula exists
    # def xor_1_to_n(n):
    #     if n % 4 == 0: return n
    #     if n % 4 == 1: return 1
    #     if n % 4 == 2: return n + 1
    #     return 0
```

Optimization: Use reduce for functional style. For range 1..n, O(1) formula exists.

Justification: Same time complexity but cleaner code with reduce. Special case optimization.

Time Complexity:

- **Brute Force:** O(n) where n is array length
- **Optimized:** O(n) where n is array length

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) for empty array
- **Average Case:** O(n) for both
- **Worst Case:** O(n) for both

Q139. Element appearing once where others appear twice

Question: Find element that appears once in array where every other element appears twice.

Solution 1 – Brute Force Approach:

```python
def single_number_bf(arr):
    """Find single number using frequency count."""
    freq = {}
    for num in arr:
        freq[num] = freq.get(num, 0) + 1

    for num, count in freq.items():
        if count == 1:
            return num

    return None
```

Logic: Count frequency of each element, return element with count 1.

Inefficiencies: O(n) time and space. Can be done with O(1) space.

Solution 2 – Optimized Approach:

```python
def single_number_opt(arr):
    """Find single number using XOR property."""
    result = 0
    for num in arr:
        result ^= num
    return result

# Extension: two numbers appear once
def two_single_numbers(arr):
    """Find two numbers that appear once."""
    # XOR all numbers
    xor_all = 0
    for num in arr:
        xor_all ^= num

    # Find rightmost set bit
    rightmost_bit = xor_all & -xor_all

    # Partition numbers based on that bit
    num1 = num2 = 0
    for num in arr:
        if num & rightmost_bit:
            num1 ^= num
        else:
            num2 ^= num

    return [num1, num2]
```

Optimization: Use XOR property: a $^{a} = 0, a^{0} = a.XORallnumbers, duplicatescancel.$

Justification: O(1) space vs O(n), clever mathematical property.

Time Complexity:

- **Brute Force:** O(n) where n is array length
- **Optimized:** O(n) where n is array length

Space Complexity:

- **Brute Force:** O(n) for frequency dict
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(n) for both
- **Average Case:** O(n) for both
- **Worst Case:** O(n) for both

Q140. Count set bits in number

Question: Count number of 1 bits (set bits) in binary representation of integer.

Solution 1 – Brute Force Approach:

```python
def count_set_bits_bf(n):
    """Count set bits by checking each bit."""
    count = 0
    while n > 0:
        if n & 1:  # Check least significant bit
            count += 1
        n >>= 1  # Right shift
    return count

# For negative numbers
def count_set_bits_signed(n):
    """Count set bits for signed integers."""
    count = 0
    # Use 32-bit mask
    mask = 0xFFFFFFFF
    n &= mask
    while n:
        count += n & 1
        n >>= 1
    return count
```

Logic: Check each bit from LSB to MSB by right shifting and checking LSB.

Inefficiencies: O(b) where b is number of bits (32 or 64). Can be optimized.

Solution 2 – Optimized Approach:

```python
def count_set_bits_opt(n):
    """Count set bits using Brian Kernighan's algorithm."""
    count = 0
    while n:
        n &= n - 1  # Clear lowest set bit
        count += 1
    return count

# Using built-in
def count_set_bits_builtin(n):
    """Count set bits using bin() or bit_count()."""
    # Python 3.10+
    # return n.bit_count()

    # Alternative
    return bin(n).count('1')

# Lookup table method (for repeated queries)
class PopCount:
    """Precomputed popcount using lookup table."""
    def __init__(self):
        self.table = [0] * 256
        for i in range(256):
```

```
            self.table[i] = (i & 1) + self.table[i >> 1]

    def count(self, n):
        count = 0
        while n:
            count += self.table[n & 0xFF]
            n >>= 8
        return count
```

Optimization: Brian Kernighan's algorithm: n &= n-1 clears lowest set bit. O(number of set bits).

Justification: O(number of set bits) vs O(total bits). Much faster for sparse bits.

Time Complexity:

- **Brute Force:** O(b) where b is number of bits (32 or 64)
- **Optimized:** O(k) where k is number of set bits

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1) for algorithm, O(256) for lookup table

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) if n=0
- **Average Case:** O(b) for brute force, O(k) for optimized
- **Worst Case:** O(b) for brute force (all bits set), O(b) for optimized (all bits set)

Q141. Check if number is power of two

Question: Check if integer is power of two (2 for some n 0).

Solution 1 – Brute Force Approach:

```python
def is_power_of_two_bf(n):
    """Check by repeatedly dividing by 2."""
    if n <= 0:
        return False

    while n > 1:
        if n % 2 != 0:
            return False
        n //= 2

    return True
```

Logic: Keep dividing by 2 until 1, check if any remainder during division.

Inefficiencies: O(log n) time, multiple divisions.

Solution 2 – Optimized Approach:

```python
def is_power_of_two_opt(n):
    """Check using bit manipulation."""
    if n <= 0:
        return False

    # Power of two has exactly one bit set
    # n & (n-1) clears lowest set bit
    return n & (n - 1) == 0

# Alternative: count set bits
def is_power_of_two_count(n):
    """Check by counting set bits."""
    if n <= 0:
        return False
    return bin(n).count('1') == 1
```

Optimization: Use bit property: power of two has exactly one 1-bit. n & (n-1) == 0 test.

Justification: O(1) time vs O(log n), single bit operation.

Time Complexity:

- **Brute Force:** O(log n) where n is the number
- **Optimized:** O(1)

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1)

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) for optimized

- **Average Case:** O(log n) for brute force, O(1) for optimized
- **Worst Case:** O(log n) for brute force, O(1) for optimized

Q142. Sieve of Eratosthenes

Question: Generate all prime numbers up to N using Sieve of Eratosthenes.

Solution 1 – Brute Force Approach:

```python
def sieve_bf(n):
    """Generate primes by checking each number individually."""
    if n < 2:
        return []

    primes = []
    for num in range(2, n + 1):
        is_prime = True
        # Check divisibility by numbers up to sqrt(num)
        for i in range(2, int(num**0.5) + 1):
            if num % i == 0:
                is_prime = False
                break
        if is_prime:
            primes.append(num)

    return primes
```

Logic: For each number from 2 to N, check if prime by testing divisibility up to n.

Inefficiencies: O(N N) time approximately, checks each number separately.

Solution 2 – Optimized Approach:

```python
def sieve_opt(n):
    """Generate primes using Sieve of Eratosthenes."""
    if n < 2:
        return []

    # Initialize sieve: True = prime, False = not prime
    sieve = [True] * (n + 1)
    sieve[0] = sieve[1] = False

    for i in range(2, int(n**0.5) + 1):
        if sieve[i]:
            # Mark multiples of i as not prime
            for j in range(i*i, n + 1, i):
                sieve[j] = False

    # Collect primes
    primes = [i for i in range(2, n + 1) if sieve[i]]
    return primes

# Optimized memory version (bit array)
def sieve_bit(n):
    """Sieve using bit array for memory efficiency."""
    if n < 2:
        return []

    # Each integer represents 32 bits
```

```
    size = (n + 31) // 32
    sieve = [0xFFFFFFFF] * size

    def clear_bit(i):
        sieve[i >> 5] &= ~(1 << (i & 31))

    def is_set(i):
        return sieve[i >> 5] & (1 << (i & 31))

    clear_bit(0)
    clear_bit(1)

    for i in range(2, int(n**0.5) + 1):
        if is_set(i):
            for j in range(i*i, n + 1, i):
                clear_bit(j)

    return [i for i in range(2, n + 1) if is_set(i)]
```

Optimization: Mark multiples of primes as non-prime starting from i^2. O(N log log N) time.

Justification: O(N log log N) vs O(N N), much faster for large N.

Time Complexity:

- **Brute Force:** O(N N) approximately
- **Optimized:** O(N log log N)

Space Complexity:

- **Brute Force:** O(1) excluding output
- **Optimized:** O(N) for sieve array

Best / Average / Worst Case Analysis:

- **Best Case:** O(N log log N) for optimized
- **Average Case:** O(N N) for brute force, O(N log log N) for optimized
- **Worst Case:** O(N N) for brute force, O(N log log N) for optimized

Q143. GCD using Euclid's algorithm

Question: Find greatest common divisor of two numbers using Euclid's algorithm.

Solution 1 – Brute Force Approach:

```python
def gcd_bf(a, b):
    """Find GCD by checking all possible divisors."""
    a, b = abs(a), abs(b)
    if a == 0:
        return b
    if b == 0:
        return a

    gcd_val = 1
    for i in range(1, min(a, b) + 1):
        if a % i == 0 and b % i == 0:
            gcd_val = i

    return gcd_val
```

Logic: Check all numbers from 1 to min(a,b), update when common divisor found.

Inefficiencies: O(min(a,b)) time, checks all numbers.

Solution 2 – Optimized Approach:

```python
def gcd_opt(a, b):
    """Find GCD using Euclidean algorithm."""
    a, b = abs(a), abs(b)

    while b != 0:
        a, b = b, a % b

    return a

# Recursive version
def gcd_recursive(a, b):
    """Recursive Euclidean algorithm."""
    a, b = abs(a), abs(b)

    if b == 0:
        return a
    return gcd_recursive(b, a % b)

# Extended Euclidean algorithm (also finds coefficients)
def extended_gcd(a, b):
    """Extended Euclidean algorithm: returns (g, x, y) such that
       ax + by = g."""
    if b == 0:
        return a, 1, 0

    g, x1, y1 = extended_gcd(b, a % b)
    x = y1
    y = x1 - (a // b) * y1

```

```
29      return g, x, y
```

Optimization: Euclidean algorithm: gcd(a,b) = gcd(b, a mod b). O(log(min(a,b))) time.

Justification: O(log(min(a,b))) vs O(min(a,b)), much faster for large numbers.

Time Complexity:

- **Brute Force:** O(min(a,b))
- **Optimized:** O(log(min(a,b)))

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1) iterative, O(log(min(a,b))) recursive stack

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) when b=0
- **Average Case:** O(min(a,b)) for brute force, O(log(min(a,b))) for optimized
- **Worst Case:** O(min(a,b)) for brute force, O(log(min(a,b))) for optimized (consecutive Fibonacci numbers)

Q144. Fast exponentiation

Question: Compute x raised to power n using fast exponentiation (exponentiation by squaring).

Solution 1 – Brute Force Approach:

```
def power_bf(x, n):
    """Compute power using repeated multiplication."""
    if n == 0:
        return 1
    if n < 0:
        return 1 / power_bf(x, -n)

    result = 1
    for _ in range(n):
        result *= x
    return result
```

Logic: Multiply x by itself n times.

Inefficiencies: O(n) multiplications for positive n.

Solution 2 – Optimized Approach:

```
def power_opt(x, n):
    """Compute power using exponentiation by squaring."""
    if n == 0:
        return 1

    # Handle negative exponent
    if n < 0:
        x = 1 / x
        n = -n

    result = 1
    current = x

    while n > 0:
        if n % 2 == 1:  # If n is odd
            result *= current
        current *= current  # Square the base
        n //= 2  # Halve the exponent

    return result

# Recursive version
def power_recursive(x, n):
    """Recursive fast exponentiation."""
    if n == 0:
        return 1
    if n < 0:
        return 1 / power_recursive(x, -n)

    half = power_recursive(x, n // 2)

```

```
    if n % 2 == 0:
        return half * half
    else:
        return x * half * half

# Modular exponentiation
def mod_power(x, n, m):
    """Compute (x^n) mod m using fast exponentiation."""
    if n == 0:
        return 1 % m

    result = 1
    x = x % m

    while n > 0:
        if n % 2 == 1:
            result = (result * x) % m
        x = (x * x) % m
        n //= 2

    return result
```

Optimization: Exponentiation by squaring: $x = (x^2)^{n/2} if even, x(x)^{(n-1)/2} if odd$.
Justification: O(log n) multiplications vs O(n), much faster for large n.
Time Complexity:

- **Brute Force:** O(n) where n is exponent
- **Optimized:** O(log n) where n is exponent

Space Complexity:

- **Brute Force:** O(1)
- **Optimized:** O(1) iterative, O(log n) recursive stack

Best / Average / Worst Case Analysis:

- **Best Case:** O(1) when n=0
- **Average Case:** O(n) for brute force, O(log n) for optimized
- **Worst Case:** O(n) for brute force, O(log n) for optimized

Q145. 0/1 Knapsack using DP

Question: Solve 0/1 knapsack problem: maximize value with weight constraint, each item can be taken at most once.

Solution 1 – Brute Force Approach:

```python
def knapsack_bf(weights, values, capacity):
    """Solve knapsack by trying all subsets."""
    n = len(weights)
    max_value = 0

    # Try all 2^n subsets
    for mask in range(1 << n):
        total_weight = 0
        total_value = 0

        for i in range(n):
            if mask & (1 << i):
                total_weight += weights[i]
                total_value += values[i]

        if total_weight <= capacity:
            max_value = max(max_value, total_value)

    return max_value
```

Logic: Generate all subsets of items, check weight constraint, track maximum value.

Inefficiencies: O(2) exponential time, checks all subsets.

Solution 2 – Optimized Approach:

```python
def knapsack_opt(weights, values, capacity):
    """Solve knapsack using dynamic programming."""
    n = len(weights)

    # DP table: dp[i][w] = max value with first i items and
       weight w
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(1, capacity + 1):
            # If current item's weight <= current capacity
            if weights[i-1] <= w:
                # Max of including or excluding item
                dp[i][w] = max(
                    values[i-1] + dp[i-1][w - weights[i-1]],  #
                       Include
                    dp[i-1][w]  # Exclude
                )
            else:
                # Can't include this item
                dp[i][w] = dp[i-1][w]

    return dp[n][capacity]
```

```python

# Space optimized version (1D array)
def knapsack_space_opt(weights, values, capacity):
    """Knapsack with O(capacity) space."""
    n = len(weights)
    dp = [0] * (capacity + 1)

    for i in range(n):
        # Iterate backwards to avoid overwriting
        for w in range(capacity, weights[i] - 1, -1):
            dp[w] = max(dp[w], values[i] + dp[w - weights[i]])

    return dp[capacity]

# With item reconstruction
def knapsack_with_items(weights, values, capacity):
    """Knapsack returning selected items."""
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(1, capacity + 1):
            if weights[i-1] <= w:
                dp[i][w] = max(values[i-1] + dp[i-1][w -
                   weights[i-1]], dp[i-1][w])
            else:
                dp[i][w] = dp[i-1][w]

    # Reconstruct items
    selected = []
    w = capacity
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i-1][w]:
            selected.append(i-1)
            w -= weights[i-1]

    return dp[n][capacity], selected[::-1]
```

Optimization: Dynamic programming. dp[i][w] = max value with first i items and weight w.

Justification: O(n×capacity) time vs O(2). Pseudo-polynomial time.

Time Complexity:

- **Brute Force:** O(2) where n is number of items
- **Optimized:** O(n×capacity) where n items, capacity constraint

Space Complexity:

- **Brute Force:** O(1) excluding masks
- **Optimized:** O(n×capacity) for DP table, O(capacity) space-optimized

Best / Average / Worst Case Analysis:

- **Best Case:** O(n×capacity) for optimized
- **Average Case:** O(2) for brute force, O(n×capacity) for optimized
- **Worst Case:** O(2) for brute force, O(n×capacity) for optimized

Q146. Minimum coins for amount (Coin Change)

Question: Find minimum number of coins needed to make amount, given coin denominations.

Solution 1 – Brute Force Approach:

```
def min_coins_bf(coins, amount):
    """Find min coins using recursion exploring all
       combinations."""
    def dfs(remaining):
        if remaining == 0:
            return 0
        if remaining < 0:
            return float('inf')

        min_coins = float('inf')
        for coin in coins:
            result = dfs(remaining - coin)
            if result != float('inf'):
                min_coins = min(min_coins, 1 + result)

        return min_coins

    result = dfs(amount)
    return result if result != float('inf') else -1
```

Logic: Recursively try all coin combinations, track minimum coins.

Inefficiencies: Exponential time due to overlapping subproblems.

Solution 2 – Optimized Approach:

```
def min_coins_opt(coins, amount):
    """Find min coins using dynamic programming."""
    # DP array: dp[i] = min coins for amount i
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0  # 0 coins needed for amount 0

    for i in range(1, amount + 1):
        for coin in coins:
            if coin <= i:
                dp[i] = min(dp[i], 1 + dp[i - coin])

    return dp[amount] if dp[amount] != float('inf') else -1

# BFS approach (finds minimum quickly)
def min_coins_bfs(coins, amount):
    """Find min coins using BFS (optimal for coin change)."""
    if amount == 0:
        return 0

    visited = [False] * (amount + 1)
    queue = [(0, 0)]  # (current amount, coins used)

    while queue:
```

```
        curr_amount, coins_used = queue.pop(0)

        for coin in coins:
            next_amount = curr_amount + coin

            if next_amount == amount:
                return coins_used + 1

            if next_amount < amount and not visited[next_amount]:
                visited[next_amount] = True
                queue.append((next_amount, coins_used + 1))

    return -1
```

Optimization: Dynamic programming: dp[i] = min coins for amount i. Or BFS for shortest path.

Justification: O(amount × coins) time vs exponential. BFS finds minimum quickly.

Time Complexity:

- **Brute Force:** O(coins^{amount})*exponential*
- **Optimized:** O(amount × coins) for DP

Space Complexity:

- **Brute Force:** O(amount) recursion stack
- **Optimized:** O(amount) for DP array

Best / Average / Worst Case Analysis:

- **Best Case:** O(amount × coins) for optimized
- **Average Case:** Exponential for brute force, O(amount × coins) for optimized
- **Worst Case:** Exponential for brute force, O(amount × coins) for optimized

Q147. Number of ways for coin change (Coin Change II)

Question: Count number of ways to make amount using given coin denominations (unlimited coins).

Solution 1 – Brute Force Approach:

```
def coin_change_ways_bf(coins, amount):
    """Count ways using recursion."""
    def dfs(idx, remaining):
        if remaining == 0:
            return 1
        if remaining < 0 or idx >= len(coins):
            return 0

        # Include current coin
        include = dfs(idx, remaining - coins[idx])
        # Exclude current coin
        exclude = dfs(idx + 1, remaining)

        return include + exclude

    return dfs(0, amount)
```

Logic: For each coin, either include it (can use again) or exclude it.

Inefficiencies: Exponential time due to overlapping subproblems.

Solution 2 – Optimized Approach:

```
def coin_change_ways_opt(coins, amount):
    """Count ways using dynamic programming."""
    # DP array: dp[i] = ways to make amount i
    dp = [0] * (amount + 1)
    dp[0] = 1  # 1 way to make amount 0 (use no coins)

    # Process coins one by one (unbounded knapsack)
    for coin in coins:
        for i in range(coin, amount + 1):
            dp[i] += dp[i - coin]

    return dp[amount]

# Alternative: 2D DP for clarity
def coin_change_ways_2d(coins, amount):
    """2D DP: dp[i][j] = ways with first i coins for amount j."""
    n = len(coins)
    dp = [[0] * (amount + 1) for _ in range(n + 1)]

    # Base case: 1 way to make amount 0 (no coins)
    for i in range(n + 1):
        dp[i][0] = 1

    for i in range(1, n + 1):
        for j in range(1, amount + 1):
            if coins[i-1] <= j:
```

```
                # Include + Exclude
                dp[i][j] = dp[i][j - coins[i-1]] + dp[i-1][j]
            else:
                # Can't include this coin
                dp[i][j] = dp[i-1][j]

    return dp[n][amount]
```

Optimization: Dynamic programming: dp[i] += dp[i - coin] for each coin. Unbounded knapsack.

Justification: O(amount × coins) time vs exponential.

Time Complexity:

- **Brute Force:** O(coins^{amount})$exponential$
- **Optimized:** O(amount × coins)

Space Complexity:

- **Brute Force:** O(amount) recursion stack
- **Optimized:** O(amount) for 1D DP, O(amount × coins) for 2D

Best / Average / Worst Case Analysis:

- **Best Case:** O(amount × coins) for optimized
- **Average Case:** Exponential for brute force, O(amount × coins) for optimized
- **Worst Case:** Exponential for brute force, O(amount × coins) for optimized

Q148. Longest common subsequence

Question: Find length of longest common subsequence (LCS) of two strings.

Solution 1 – Brute Force Approach:

```
def lcs_bf(s1, s2):
    """Find LCS by checking all subsequences."""
    def generate_subsequences(s):
        """Generate all subsequences of string."""
        n = len(s)
        result = []
        for mask in range(1 << n):
            subseq = []
            for i in range(n):
                if mask & (1 << i):
                    subseq.append(s[i])
            result.append(''.join(subseq))
        return result

    # Generate all subsequences
    subs1 = generate_subsequences(s1)
    subs2 = generate_subsequences(s2)

    # Find longest common
    max_len = 0
    for sub1 in subs1:
        for sub2 in subs2:
            if sub1 == sub2:
                max_len = max(max_len, len(sub1))

    return max_len
```

Logic: Generate all subsequences of both strings, find longest matching pair.

Inefficiencies: $O(2^{m+n}) time, generate all subsequences.$

Solution 2 – Optimized Approach:

```
def lcs_opt(s1, s2):
    """Find LCS using dynamic programming."""
    m, n = len(s1), len(s2)

    # DP table: dp[i][j] = LCS length for s1[:i], s2[:j]
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    # Reconstruct LCS
    lcs = []
    i, j = m, n
```

```
    while i > 0 and j > 0:
        if s1[i-1] == s2[j-1]:
            lcs.append(s1[i-1])
            i -= 1
            j -= 1
        elif dp[i-1][j] > dp[i][j-1]:
            i -= 1
        else:
            j -= 1

    return dp[m][n], ''.join(reversed(lcs))

# Space optimized version
def lcs_space_opt(s1, s2):
    """LCS with O(min(m,n)) space."""
    if len(s1) < len(s2):
        s1, s2 = s2, s1  # Ensure s2 is shorter

    m, n = len(s1), len(s2)
    prev = [0] * (n + 1)

    for i in range(1, m + 1):
        curr = [0] * (n + 1)
        for j in range(1, n + 1):
            if s1[i-1] == s2[j-1]:
                curr[j] = prev[j-1] + 1
            else:
                curr[j] = max(prev[j], curr[j-1])
        prev = curr

    return prev[n]
```

Optimization: Dynamic programming. dp[i][j] = LCS length for prefixes s1[:i], s2[:j].
Justification: O(m×n) time vs exponential, standard LCS algorithm.
Time Complexity:

- **Brute Force:** $O(2^{m+n}) where m, n are string lengths$
- **Optimized:** O(m×n) where m,n are string lengths

Space Complexity:

- **Brute Force:** $O(2^{m+n}) for all subsequences$
- **Optimized:** O(m×n) for DP table, O(min(m,n)) space-optimized

Best / Average / Worst Case Analysis:

- **Best Case:** O(m×n) for optimized
- **Average Case:** Exponential for brute force, O(m×n) for optimized
- **Worst Case:** Exponential for brute force, O(m×n) for optimized

Q149. Edit distance between strings

Question: Compute minimum edit distance (Levenshtein distance) between two strings.

Solution 1 – Brute Force Approach:

```python
def edit_distance_bf(s1, s2):
    """Compute edit distance using recursion."""
    def dfs(i, j):
        if i == 0:
            return j  # Insert j characters
        if j == 0:
            return i  # Delete i characters

        if s1[i-1] == s2[j-1]:
            return dfs(i-1, j-1)  # No cost

        # Minimum of insert, delete, replace
        return 1 + min(
            dfs(i, j-1),    # Insert
            dfs(i-1, j),    # Delete
            dfs(i-1, j-1)   # Replace
        )

    return dfs(len(s1), len(s2))
```

Logic: Recursively try all operations: insert, delete, replace. Base cases when one string empty.

Inefficiencies: Exponential time due to overlapping subproblems.

Solution 2 – Optimized Approach:

```python
def edit_distance_opt(s1, s2):
    """Compute edit distance using dynamic programming."""
    m, n = len(s1), len(s2)

    # DP table: dp[i][j] = edit distance for s1[:i], s2[:j]
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Initialize first row and column
    for i in range(m + 1):
        dp[i][0] = i  # Delete all characters
    for j in range(n + 1):
        dp[0][j] = j  # Insert all characters

    # Fill DP table
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = dp[i-1][j-1]  # No cost
            else:
                dp[i][j] = 1 + min(
                    dp[i][j-1],    # Insert
                    dp[i-1][j],    # Delete
                    dp[i-1][j-1]   # Replace
```

```
                )

    return dp[m][n]

# Space optimized version
def edit_distance_space_opt(s1, s2):
    """Edit distance with O(min(m,n)) space."""
    if len(s1) < len(s2):
        s1, s2 = s2, s1  # Ensure s1 is longer

    m, n = len(s1), len(s2)
    prev = list(range(n + 1))

    for i in range(1, m + 1):
        curr = [i] + [0] * n
        for j in range(1, n + 1):
            if s1[i-1] == s2[j-1]:
                curr[j] = prev[j-1]
            else:
                curr[j] = 1 + min(prev[j], curr[j-1], prev[j-1])
        prev = curr

    return prev[n]
```

Optimization: Dynamic programming. dp[i][j] = min edit distance for prefixes.
Justification: O(m×n) time vs exponential, standard edit distance algorithm.
Time Complexity:

- **Brute Force:** O(3^{m+n})*exponential*
- **Optimized:** O(m×n) where m,n are string lengths

Space Complexity:

- **Brute Force:** O(m+n) recursion stack
- **Optimized:** O(m×n) for DP table, O(min(m,n)) space-optimized

Best / Average / Worst Case Analysis:

- **Best Case:** O(m×n) for optimized
- **Average Case:** Exponential for brute force, O(m×n) for optimized
- **Worst Case:** Exponential for brute force, O(m×n) for optimized

Q150. Unique paths in grid

Question: Count number of unique paths from top-left to bottom-right in m×n grid, moving only right or down.

Solution 1 – Brute Force Approach:

```
def unique_paths_bf(m, n):
    """Count paths using recursion (DFS)."""
    def dfs(i, j):
        if i == m - 1 and j == n - 1:
            return 1
        if i >= m or j >= n:
            return 0

        # Move right + move down
        return dfs(i, j + 1) + dfs(i + 1, j)

    return dfs(0, 0)
```

Logic: From each cell, recursively explore right and down paths, count when reach destination.

Inefficiencies: Exponential time $O(2^{m+n})$ *duetooverlappingsubproblems*.

Solution 2 – Optimized Approach:

```
def unique_paths_opt(m, n):
    """Count paths using dynamic programming."""
    # DP table: dp[i][j] = paths to reach (i,j)
    dp = [[0] * n for _ in range(m)]

    # Initialize first row and column
    for i in range(m):
        dp[i][0] = 1
    for j in range(n):
        dp[0][j] = 1

    # Fill DP table
    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i-1][j] + dp[i][j-1]

    return dp[m-1][n-1]

# Space optimized version (1D array)
def unique_paths_space_opt(m, n):
    """Count paths with O(n) space."""
    dp = [1] * n

    for i in range(1, m):
        for j in range(1, n):
            dp[j] += dp[j-1]

    return dp[n-1]

```

```python
# Using combinatorics: C(m+n-2, m-1)
def unique_paths_math(m, n):
    """Count paths using combinatorial formula."""
    from math import comb
    return comb(m + n - 2, m - 1)

# With obstacles
def unique_paths_with_obstacles(grid):
    """Count paths with obstacles (1=obstacle, 0=empty)."""
    m, n = len(grid), len(grid[0])
    dp = [[0] * n for _ in range(m)]

    # Initialize first cell
    dp[0][0] = 0 if grid[0][0] == 1 else 1

    # Initialize first column
    for i in range(1, m):
        dp[i][0] = dp[i-1][0] if grid[i][0] == 0 else 0

    # Initialize first row
    for j in range(1, n):
        dp[0][j] = dp[0][j-1] if grid[0][j] == 0 else 0

    # Fill DP table
    for i in range(1, m):
        for j in range(1, n):
            if grid[i][j] == 0:
                dp[i][j] = dp[i-1][j] + dp[i][j-1]
            else:
                dp[i][j] = 0

    return dp[m-1][n-1]
```

Optimization: Dynamic programming: paths(i,j) = paths(i-1,j) + paths(i,j-1). Or combinatorial formula.

Justification: O(m×n) time vs exponential. Combinatorial formula O(1) after precomputation.

Time Complexity:

- **Brute Force:** O(2^{m+n})*exponential*
- **Optimized:** O(m×n) for DP, O(1) for combinatorial

Space Complexity:

- **Brute Force:** O(m+n) recursion stack
- **Optimized:** O(m×n) for DP table, O(n) space-optimized

Best / Average / Worst Case Analysis:

- **Best Case:** O(m×n) for optimized DP

- **Average Case:** Exponential for brute force, O(m×n) for optimized
- **Worst Case:** Exponential for brute force, O(m×n) for optimized